

Linear Algebra & Probability: Minimum Lessons for CS221, EE261, and EE278

Rigorous theory (after Axler, and after Bertsekas–Tsitsiklis) meets hands-on
computation (after Strang)

A four-part refresher

August 2026 (revised)

How to use this booklet

Stanford’s CS221 lists linear algebra (Math 51) as a required prerequisite, but the course itself uses a surprisingly small slice of the subject — and the staff explicitly note that eigenvectors, a pillar of most linear algebra courses, are not used. What CS221 actually leans on, constantly, is: vectors as feature representations, dot products as scores, norms as loss magnitudes, matrices as linear maps and datasets, spans/rank when reasoning about what a model can express, projections and least squares behind linear regression, and gradients of vector functions for gradient descent.

Part I covers exactly that slice, in seven lessons plus a capstone, sized to fit before the course starts.

CS221 also lists *probability* (CS109-level) as a required prerequisite, and unlike the eigenvector story, it uses this one heavily: Bayes’ rule and conditional independence power the Bayesian networks and HMM lectures, expectations define the utilities that MDPs and game-playing maximize, and every loss function is an expectation over data. **Part III** covers that probability core — probabilistic models, conditioning and Bayes’ rule, discrete and continuous random variables, expectation — in four lessons plus a capstone, following Bertsekas & Tsitsiklis, *Introduction to Probability* (2nd ed.), Chapters 1–3. Do it right after Part I, before CS221 begins.

Part II continues the linear algebra through the rest of Axler’s book (Chapters 5–9): complex vector spaces, eigenvalues and diagonalization, the spectral theorem, unitary matrices and the discrete Fourier transform, the singular value decomposition, and determinants and trace. Its target is Stanford’s **EE261**: *The Fourier Transform and Its Applications* — a course whose linear-algebra backbone is precisely orthonormal expansions, eigen-analysis of linear systems, and the DFT as a unitary matrix. Part II is not needed for CS221 and can be done at a relaxed pace in the weeks before EE261 begins.

Part IV continues the probability toward Stanford’s **EE278**: *Probability and Statistical Inference* (in earlier years, *Introduction to Statistical Signal Processing*). EE278 assumes a full first course in probability (EE178-level) plus linear systems and Fourier transforms at the EE102/EE261 level, and then builds: random vectors and random processes, convergence and limit theorems, Markov and Gaussian processes, stationarity and autocorrelation, minimum-mean-square-error and linear estimation, Kalman filtering, hypothesis testing, and PCA. Part

IV is the prerequisite half of that list, from Bertsekas & Tsitsiklis Chapters 4–8: derived distributions, covariance, conditional expectation as an estimator, random vectors and Gaussian vectors, limit theorems, the Bernoulli and Poisson processes, Markov chains, and least-mean-squares estimation. (The Fourier half of EE278’s prerequisites is EE261 itself — and Part II is its linear-algebra spine.) Part IV leans on Part III throughout and, at marked moments, on Part II: Lesson 22 reuses Lesson 6’s projection geometry, Lesson 23 reuses Lesson 11’s spectral theorem, and Lesson 26 reuses Lesson 10’s eigenvalues.

Each lesson has three interleaved layers:

- **Theory** in the style of Axler’s *Linear Algebra Done Right* (LADR, 4th ed.) for Parts I–II, and of Bertsekas–Tsitsiklis (B&T) for Parts III–IV: precise definitions and theorems with complete proofs. Understanding *why* a fact is true is what makes it stick.
- **Concrete practice** in the style of Strang’s *Introduction to Linear Algebra*: actual small examples — 2- and 3-dimensional vectors, dice, coins, three-state chains — computed by hand, with geometric pictures in words.
- **NumPy**: the same objects on a computer, since CS221 homework is in Python and EE278 problem sets routinely simulate.

Exercises at the end of each lesson are tagged **[Proof]**, **[Hand]**, or **[NumPy]**. Do all of the [Hand] ones, most of the [NumPy] ones, and as many [Proof] ones as time allows — the starred proofs are the most valuable. Hints and answers to computational exercises are in Appendix A.

What each course needs from this booklet

CS221 (*Artificial Intelligence: Principles and Techniques*) covers, in order: machine learning (linear models, SGD, features, neural networks), state-based search, Markov decision processes, game playing, constraint satisfaction, graphical models (Bayesian networks and hidden Markov models), and logic. Its prerequisites are CS103 (discrete math), CS106B (programming), CS109 (probability), and CS161 (algorithms). From *this* booklet it needs Part I (the ML lectures) and Part III (the MDP, game-playing, and graphical-models lectures) — plus the Markov-chain lesson of Part IV if you want the MDP and HMM lectures to feel like review rather than news.

EE278 (*Probability and Statistical Inference*) assumes all of Part III as a *floor* and then builds on precisely the material of Part IV; its Fourier/linear-systems prerequisite is EE261, whose linear-algebra spine is Part II.

Every lesson below carries a **NEEDED FOR:** tag. The booklet-wide map:

Lessons	CS221	EE261 / EE278
1–8 (Part I: linear algebra core)	required	assumed background for both
9–15 (Part II: eigen/Fourier/SVD)	not needed	EE261 core ; EE278 (PCA, DFT)
16–20 (Part III: probability core)	required	EE278 assumes it (EE178-level)
21 (derived dists, covariance)	not needed	EE278
22 (conditional expectation, LMS)	not needed	EE278 (MMSE, Kalman lead-in)
23 (random vectors, Gaussians)	not needed	EE278 (PCA, Gaussian processes)
24 (limit theorems)	helpful (Monte Carlo)	EE278 (convergence, concentration)
25 (Bernoulli/Poisson processes)	not needed	EE278 (first random processes)
26 (Markov chains)	helpful (MDPs, HMMs)	EE278 (Markov processes)
27 (Capstone IV)	not needed	EE278

In one sentence: *for CS221 do Lessons 1–8 and 16–20 (and skim 26); everything else is extra, for EE261 (Part II) and EE278 (Part IV).*

Schedule

Part I — the CS221 linear algebra sprint:

Day 1 (Sat 8/8)	Lesson 1: Vectors and linear combinations
Day 2 (Sun 8/9)	Lesson 2: Dot products, norms, orthogonality
Day 3 (Mon 8/10)	Lesson 3: Span, independence, basis, dimension
Day 4 (Tue 8/11)	Lesson 4: Matrices and linear maps
Day 5 (Wed 8/12)	Lesson 5: Null space, rank, and solving $Ax = b$
Day 6 (Thu 8/13)	Lesson 6: Projections and least squares
Day 7 (Fri 8/14)	Lesson 7: Gradients for machine learning
Day 8 (Sat 8/15)	Capstone: linear regression from scratch in NumPy
Day 9 (Sun 8/16)	Buffer: unfinished exercises; skim the self-check list

Part III — the CS221 probability core, one lesson per day, starting 8/17:

Day 1	Lesson 16: Probability models, conditioning, and Bayes' rule
Day 2	Lesson 17: Discrete random variables and expectation
Day 3	Lesson 18: Multiple random variables and conditioning
Day 4	Lesson 19: Continuous random variables and the normal
Day 5	Capstone III: a naive Bayes classifier from scratch

Part II — one lesson per day, any week before EE261 starts:

Day 1	Lesson 9: Complex vectors and complex exponentials
Day 2	Lesson 10: Eigenvalues, eigenvectors, diagonalization
Day 3	Lesson 11: Self-adjoint operators and the spectral theorem
Day 4	Lesson 12: Unitary matrices and the discrete Fourier transform
Day 5	Lesson 13: The singular value decomposition
Day 6	Lesson 14: Determinants and trace
Day 7	Capstone II: the DFT in NumPy

Part IV — one lesson per day, after Parts II & III, before EE278 starts:

Day 1	Lesson 21: Derived distributions, covariance, correlation
Day 2	Lesson 22: Conditional expectation and least mean squares
Day 3	Lesson 23: Random vectors and Gaussian vectors
Day 4	Lesson 24: Limit theorems
Day 5	Lesson 25: The Bernoulli and Poisson processes
Day 6	Lesson 26: Markov chains
Day 7	Capstone IV: estimation and simulation in NumPy

Deeper reading. Pointers appear at the end of each lesson: LADR sections by number (e.g. 1A, 3C), Strang by chapter name (editions of Strang differ in page numbering), and B&T by section number (e.g. B&T 1.3–1.5). Everything here is self-contained, though — the pointers are optional.

Notation. Following Axler, $\mathbb{F}$ denotes $\mathbb{R}$ or $\mathbb{C}$; in Part I you can always read $\mathbb{F} = \mathbb{R}$; Part II uses $\mathbb{F} = \mathbb{C}$ where it matters and says so. Vectors in $\mathbb{R}^n$ are written as columns; $x = (x_1, \dots, x_n)$

is shorthand for the column vector with entries $x_1, \dots, x_n$. In Parts III–IV, following B&T, $\mathbf{P}(A)$ is the probability of an event, random variables are capital letters $X, Y, \dots$, p_X is a probability mass function, f_X a density, $\mathbf{E}[X]$ an expectation, and $\text{var}(X)$, $\text{cov}(X, Y)$ variance and covariance.

Contents

Part I: The CS221 Core	9
1 Vectors and Linear Combinations	9
1.1 Theory: the space $\mathbb{R}^n$	9
1.2 Practice: vectors in 2D and 3D	10
1.3 NumPy	10
1.4 Exercises	11
2 Dot Products, Norms, and Orthogonality	11
2.1 Theory: inner products	11
2.2 Practice: 2D/3D computations	13
2.3 NumPy	13
2.4 Exercises	13
3 Span, Linear Independence, Basis, Dimension	14
3.1 Theory	14
3.2 Practice: 2D/3D	16
3.3 NumPy	16
3.4 Exercises	16
4 Matrices and Linear Maps	17
4.1 Theory: linear maps first	17
4.2 Practice: 2D matrices you can see	18
4.3 NumPy	19
4.4 Exercises	19
5 Null Space, Rank, and Solving $Ax = b$	20
5.1 Theory: the fundamental theorem	20
5.2 Practice: elimination by hand	21
5.3 NumPy	22
5.4 Exercises	22
6 Orthogonal Projections and Least Squares	23
6.1 Theory	23
6.2 Practice: fitting a line by hand	24
6.3 NumPy	25
6.4 Exercises	25
7 Gradients for Machine Learning	26
7.1 Theory	26
7.2 Practice: by hand	27
7.3 NumPy	28
7.4 Exercises	28
8 Capstone: Linear Regression from Scratch	29

Part II: Through Chapter 9, Toward EE261	31
9 Complex Vectors and Complex Exponentials	31
9.1 Theory	31
9.2 Practice: by hand	32
9.3 NumPy	32
9.4 Exercises	33
10 Eigenvalues, Eigenvectors, and Diagonalization	33
10.1 Theory	33
10.2 Practice: by hand	35
10.3 NumPy	36
10.4 Exercises	36
11 Self-Adjoint Operators and the Spectral Theorem	37
11.1 Theory	37
11.2 Practice: by hand	38
11.3 NumPy	39
11.4 Exercises	39
12 Unitary Matrices and the Discrete Fourier Transform	40
12.1 Theory	40
12.2 Practice: by hand	41
12.3 NumPy	42
12.4 Exercises	42
13 The Singular Value Decomposition	43
13.1 Theory	43
13.2 Practice: by hand	44
13.3 NumPy	45
13.4 Exercises	45
14 Determinants and Trace	46
14.1 Theory	46
14.2 Practice: by hand	47
14.3 NumPy	48
14.4 Exercises	48
15 Capstone II: The DFT in NumPy	49
Part III: The CS221 Probability Core	51
16 Probability Models, Conditioning, and Bayes' Rule	51
16.1 Theory	51
16.2 Practice: by hand	53
16.3 NumPy	53
16.4 Exercises	54

17 Discrete Random Variables and Expectation	54
17.1 Theory	55
17.2 Practice: by hand	56
17.3 NumPy	57
17.4 Exercises	57
18 Multiple Random Variables and Conditioning	58
18.1 Theory	58
18.2 Practice: by hand	59
18.3 NumPy	60
18.4 Exercises	60
19 Continuous Random Variables and the Normal	61
19.1 Theory	61
19.2 Practice: by hand	62
19.3 NumPy	63
19.4 Exercises	63
20 Capstone III: A Naive Bayes Classifier from Scratch	64
Part IV: Toward EE278	66
21 Derived Distributions, Covariance, and Correlation	66
21.1 Theory	66
21.2 Practice: by hand	67
21.3 NumPy	68
21.4 Exercises	68
22 Conditional Expectation and Least Mean Squares	69
22.1 Theory	69
22.2 Practice: by hand	70
22.3 NumPy	71
22.4 Exercises	72
23 Random Vectors and Gaussian Vectors	72
23.1 Theory	72
23.2 Practice: by hand	74
23.3 NumPy	74
23.4 Exercises	75
24 Limit Theorems	75
24.1 Theory	76
24.2 Practice: by hand	77
24.3 NumPy	77
24.4 Exercises	78

25 The Bernoulli and Poisson Processes	78
25.1 Theory	78
25.2 Practice: by hand	79
25.3 NumPy	80
25.4 Exercises	80
26 Markov Chains	81
26.1 Theory	81
26.2 Practice: by hand	82
26.3 NumPy	83
26.4 Exercises	83
27 Capstone IV: Estimation and Simulation in NumPy	84
A Hints and Answers to Computational Exercises	87

Part I: The CS221 Core

1 Vectors and Linear Combinations

NEEDED FOR: CS221 (core); assumed by everything later

1.1 Theory: the space $\mathbb{R}^n$

Definition 1.1 ($\mathbb{R}^n$). For a positive integer n , $\mathbb{R}^n$ is the set of all lists of n real numbers:

$$\mathbb{R}^n = \{(x_1, \dots, x_n) : x_k \in \mathbb{R} \text{ for } k = 1, \dots, n\}.$$

Elements of $\mathbb{R}^n$ are called *vectors*; the number x_k is the k th *coordinate*.

Definition 1.2 (Addition and scalar multiplication). For $x = (x_1, \dots, x_n)$, $y = (y_1, \dots, y_n)$ in $\mathbb{R}^n$ and $\lambda \in \mathbb{R}$:

$$x + y = (x_1 + y_1, \dots, x_n + y_n), \quad \lambda x = (\lambda x_1, \dots, \lambda x_n).$$

These two operations obey the familiar rules of arithmetic. Axler abstracts these rules into the definition of a *vector space*: a set V with an addition and scalar multiplication satisfying commutativity, associativity, existence of an additive identity 0 and additive inverses, a multiplicative identity ($1v = v$), and the distributive properties. $\mathbb{R}^n$ is the prototypical example, and for CS221, $\mathbb{R}^n$ is essentially the only vector space you need. But two quick payoffs of the abstract view are worth internalizing, because their proofs teach the style of reasoning used everywhere below.

Proposition 1.3 (Unique additive identity). *A vector space has exactly one additive identity.*

Proof. Suppose 0 and $0'$ are both additive identities, meaning $v + 0 = v$ and $v + 0' = v$ for every v . Then

$$0' = 0' + 0 = 0 + 0' = 0,$$

where the first equality holds because 0 is an additive identity, the second by commutativity of addition, and the third because $0'$ is an additive identity. Hence $0' = 0$. $\square$

Proposition 1.4 (0 times a vector). *In any vector space, $0v = 0$ for every vector v . (On the left, 0 is the number zero; on the right, the zero vector.)*

Proof. Using the distributive property with $0 = 0 + 0$ in $\mathbb{R}$:

$$0v = (0 + 0)v = 0v + 0v.$$

Adding the additive inverse $-(0v)$ to both sides gives $0 = 0v$. $\square$

Why prove obvious things? These proofs use only the listed axioms, never a picture. That discipline is exactly what you need later when statements stop being obvious — e.g., “the set of solutions of $Ax = 0$ is a subspace” or “a projection minimizes distance.” The habit: every step cites a definition or a previously proved fact.

Definition 1.5 (Linear combination). A *linear combination* of vectors $v_1, \dots, v_m \in \mathbb{R}^n$ is a vector of the form

$$a_1v_1 + \dots + a_mv_m, \quad a_1, \dots, a_m \in \mathbb{R}.$$

1.2 Practice: vectors in 2D and 3D

A vector $v = (2, 1)$ in $\mathbb{R}^2$ is drawn as an arrow from the origin to the point $(2, 1)$. Addition is the parallelogram rule: place the tail of w at the head of v ; the sum $v + w$ is the arrow from the origin to the final head. Scalar multiplication stretches ($\lambda > 1$), shrinks ($0 < \lambda < 1$), or flips ($\lambda < 0$) the arrow.

Example 1.6 (All combinations of one or two vectors). Let $v = (1, 2)$ and $w = (3, 1)$ in $\mathbb{R}^2$.

- The set of all multiples $\{\lambda v : \lambda \in \mathbb{R}\}$ is the line through the origin and $(1, 2)$.
- The set of all combinations $\{av + bw\}$ is all of $\mathbb{R}^2$: given any target (t_1, t_2) , we need $a + 3b = t_1$ and $2a + b = t_2$; solving gives $b = (2t_1 - t_2)/5$ and $a = t_1 - 3b$, which always works. (Lesson 3 explains why it always works: v and w are linearly independent.)

In $\mathbb{R}^3$, the combinations of two (non-parallel) vectors fill a *plane* through the origin — not all of $\mathbb{R}^3$. Try to reach $(0, 0, 1)$ using only $u = (1, 0, 0)$ and $v = (0, 1, 0)$: every combination $au + bv = (a, b, 0)$ has third coordinate 0, so you cannot. Combinations of a third vector pointing out of that plane, e.g. $w = (0, 0, 1)$, fill all of $\mathbb{R}^3$.

Example 1.7 (Hand computation). With $u = (1, 0, -1)$, $v = (2, 1, 0)$, $w = (0, 1, 2)$:

$$2u - v + 3w = (2, 0, -2) - (2, 1, 0) + (0, 3, 6) = (0, 2, 4).$$

Notice $(0, 2, 4) = 2w + 0u + 0v$ as well — the same vector can arise from different combinations. Lesson 3 turns this observation into the definition of linear *dependence*: indeed $2u - v + w = 0$ here (check it!), so these three vectors are dependent.

CS221 connection. A feature extractor ϕ maps an input (an email, a sentence, a game state) to a feature vector $\phi(x) \in \mathbb{R}^n$. Adding evidence and scaling importance are literally vector addition and scalar multiplication. A linear model's weight vector $w \in \mathbb{R}^n$ lives in the same space.

1.3 NumPy

```
import numpy as np

v = np.array([1.0, 2.0])          # a vector in R^2
w = np.array([3.0, 1.0])
print(v + w)                     # [4. 3.]      componentwise
print(2.5 * v)                   # [2.5 5. ]      scalar multiple
print(2*v - 0.5*w)               # any linear combination

u = np.zeros(3)                  # the zero vector in R^3
e1 = np.eye(3)[0]                # standard basis vector (1,0,0)
print(v.shape)                   # (2,) -- a 1-D array of length 2
```

A NumPy 1-D array of length n plays the role of a vector in $\mathbb{R}^n$. Operations are componentwise and *vectorized*: $v + w$ adds whole arrays at machine speed with no Python loop. In complexity terms, adding or scaling length- n vectors costs $O(n)$ arithmetic operations — linear in the length — but NumPy's constant factor is tiny compared to a Python `for` loop, which matters when n is in the millions.

1.4 Exercises

Exercise 1.1 (Hand). Let $v = (2, -1)$ and $w = (1, 3)$. Compute $3v + 2w$, $v - w$, and find scalars a, b with $av + bw = (0, 7)$.

Exercise 1.2 (Hand). Which vectors in $\mathbb{R}^3$ are linear combinations of $u = (1, 1, 0)$ and $v = (0, 1, 1)$? Describe the set geometrically, and exhibit one specific vector in $\mathbb{R}^3$ that is *not* a combination of u and v , with justification.

Exercise 1.3 (Proof, ★). Prove that in any vector space, $(-1)v = -v$ for every v ; that is, $(-1)v$ is the additive inverse of v . (Use Proposition 1.4 and the distributive property. This is LADR 1.B territory.)

Exercise 1.4 (Proof). Prove that the additive inverse of each vector is unique (mirror the proof of the unique-additive-identity proposition).

Exercise 1.5 (NumPy). Generate $x = \text{np.random.randn}(5)$ and $y = \text{np.random.randn}(5)$. Verify numerically that $2(x + y) = 2x + 2y$ using `np.allclose`. Then time $x + y$ for length 10^7 arrays against a Python loop doing the same additions (use `time.perf_counter`); both are $O(n)$, so explain in one sentence where the speed difference comes from.

Deeper reading. LADR 1A ($\mathbb{R}^n$ and $\mathbb{C}^n$), 1B (vector space axioms); Strang, ch. “Introduction to Vectors” (vectors and linear combinations).

2 Dot Products, Norms, and Orthogonality

NEEDED FOR: CS221 (core)

2.1 Theory: inner products

Definition 2.1 (Dot product). For $x = (x_1, \dots, x_n)$ and $y = (y_1, \dots, y_n)$ in $\mathbb{R}^n$, the *dot product* (Axler: *Euclidean inner product*) is

$$x \cdot y = \langle x, y \rangle = x_1 y_1 + \dots + x_n y_n.$$

The dot product on $\mathbb{R}^n$ satisfies, for all $x, y, z \in \mathbb{R}^n$ and $\lambda \in \mathbb{R}$ (each is a one-line check from the definition):

- *positivity*: $\langle x, x \rangle \geq 0$, with equality if and only if $x = 0$;
- *symmetry*: $\langle x, y \rangle = \langle y, x \rangle$;
- *linearity in each slot*: $\langle x + z, y \rangle = \langle x, y \rangle + \langle z, y \rangle$ and $\langle \lambda x, y \rangle = \lambda \langle x, y \rangle$.

Every proof in this lesson uses only these three properties — never the coordinate formula. That is the Axler discipline, and it means the results hold for *any* inner product.

Definition 2.2 (Norm, orthogonality). The *norm* of x is $\|x\| = \sqrt{\langle x, x \rangle} = \sqrt{x_1^2 + \dots + x_n^2}$. Vectors x, y are *orthogonal* if $\langle x, y \rangle = 0$.

Theorem 2.3 (Pythagorean theorem). If $x, y \in \mathbb{R}^n$ are orthogonal, then $\|x + y\|^2 = \|x\|^2 + \|y\|^2$.

Proof. Expanding with linearity and symmetry,

$$\|x + y\|^2 = \langle x + y, x + y \rangle = \langle x, x \rangle + 2\langle x, y \rangle + \langle y, y \rangle = \|x\|^2 + 2\langle x, y \rangle + \|y\|^2,$$

and orthogonality makes the middle term vanish. $\square$

The next lemma is the engine of Lessons 2 and 6: any vector can be split into a piece along y and a piece orthogonal to y .

Lemma 2.4 (Orthogonal decomposition). *Let $y \neq 0$. For any $x \in \mathbb{R}^n$, set*

$$c = \frac{\langle x, y \rangle}{\|y\|^2} \quad \text{and} \quad z = x - cy.$$

Then $\langle z, y \rangle = 0$ and $x = cy + z$.

Proof. The identity $x = cy + z$ holds by the definition of z . For orthogonality,

$$\langle z, y \rangle = \langle x - cy, y \rangle = \langle x, y \rangle - c \langle y, y \rangle = \langle x, y \rangle - \frac{\langle x, y \rangle}{\|y\|^2} \|y\|^2 = 0. \quad \square$$

The vector cy is the *projection of x onto the line spanned by y* .

Theorem 2.5 (Cauchy–Schwarz inequality). *For all $x, y \in \mathbb{R}^n$,*

$$|\langle x, y \rangle| \leq \|x\| \|y\|,$$

with equality if and only if one of x, y is a scalar multiple of the other.

Proof. If $y = 0$ both sides are 0. Otherwise decompose $x = cy + z$ as in Lemma 2.4. Since cy and z are orthogonal, the Pythagorean theorem gives

$$\|x\|^2 = \|cy\|^2 + \|z\|^2 = \frac{\langle x, y \rangle^2}{\|y\|^2} + \|z\|^2 \geq \frac{\langle x, y \rangle^2}{\|y\|^2},$$

because $\|z\|^2 \geq 0$. Multiplying by $\|y\|^2$ and taking square roots yields the inequality. Equality holds exactly when $\|z\| = 0$, i.e. $z = 0$, i.e. $x = cy$ is a multiple of y (and if x is a multiple of y , direct computation gives equality). $\square$

Theorem 2.6 (Triangle inequality). $\|x + y\| \leq \|x\| + \|y\|$ for all $x, y \in \mathbb{R}^n$.

Proof. Using Cauchy–Schwarz in the middle step,

$$\|x + y\|^2 = \|x\|^2 + 2\langle x, y \rangle + \|y\|^2 \leq \|x\|^2 + 2\|x\|\|y\| + \|y\|^2 = (\|x\| + \|y\|)^2. \quad \square$$

By Cauchy–Schwarz, for nonzero x, y the ratio $\langle x, y \rangle / (\|x\|\|y\|)$ lies in $[-1, 1]$, so it is the cosine of a unique angle $\theta \in [0, \pi]$:

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\| \|y\|}.$$

In $\mathbb{R}^2$ and $\mathbb{R}^3$ this matches the geometric angle between the arrows; in higher dimensions it is the *definition* of angle, and Cauchy–Schwarz is precisely what makes the definition legal.

2.2 Practice: 2D/3D computations

Example 2.7. Let $x = (3, 4)$ and $y = (4, -3)$. Then $\langle x, y \rangle = 12 - 12 = 0$: orthogonal. Norms: $\|x\| = \sqrt{9 + 16} = 5 = \|y\|$. Pythagoras check: $x + y = (7, 1)$, and $\|x + y\|^2 = 50 = 25 + 25$. ✓

Example 2.8 (Projection by hand). Project $x = (2, 3)$ onto the line spanned by $y = (1, 1)$:

$$c = \frac{\langle x, y \rangle}{\|y\|^2} = \frac{2 + 3}{2} = \frac{5}{2}, \quad cy = \left(\frac{5}{2}, \frac{5}{2}\right), \quad z = x - cy = \left(-\frac{1}{2}, \frac{1}{2}\right).$$

Check: $\langle z, y \rangle = -\frac{1}{2} + \frac{1}{2} = 0$. ✓ The point $(\frac{5}{2}, \frac{5}{2})$ is the closest point to $(2, 3)$ on the line $y = x$ — Lesson 6 proves that projections are always the closest points.

Example 2.9 (Angles). For $u = (1, 0, 1)$ and $v = (1, 1, 0)$: $\langle u, v \rangle = 1$, $\|u\| = \|v\| = \sqrt{2}$, so $\cos \theta = 1/2$ and $\theta = 60^\circ$.

CS221 connection. A linear classifier predicts with the *score* $w \cdot \phi(x)$: positive score, predict +1; negative, predict -1. The decision boundary is the set $\{v : w \cdot v = 0\}$ — the hyperplane orthogonal to w . The *margin* of an example involves $\|w\|$, and cosine similarity between feature vectors is exactly $\cos \theta$ above. You will compute dot products constantly.

2.3 NumPy

```
import numpy as np
x = np.array([2.0, 3.0]); y = np.array([1.0, 1.0])

np.dot(x, y)           # 5.0 -- also: x @ y
np.linalg.norm(x)      # sqrt(13)
c = (x @ y) / (y @ y)  # projection coefficient: 2.5
p = c * y              # projection of x onto span(y)
z = x - p
np.allclose(z @ y, 0)  # True: residual is orthogonal

cos_theta = (x @ y) / (np.linalg.norm(x) * np.linalg.norm(y))
```

A dot product of length- n vectors is n multiplications and $n - 1$ additions: $O(n)$, one pass over the data. `x @ y` is the idiom you will see in CS221 starter code.

2.4 Exercises

Exercise 2.1 (Hand). For $x = (1, 2, 2)$ and $y = (2, -2, 1)$: compute $\langle x, y \rangle$, $\|x\|$, $\|y\|$, and $\cos \theta$. Are x and y orthogonal?

Exercise 2.2 (Hand). Project $x = (4, 1)$ onto $y = (2, 1)$; write $x = cy + z$ and verify $\langle z, y \rangle = 0$. Then compute $\|cy\|^2 + \|z\|^2$ and confirm it equals $\|x\|^2$.

Exercise 2.3 (Proof, ★). (Parallelogram equality, LADR 6A) Prove that for all $x, y \in \mathbb{R}^n$,

$$\|x + y\|^2 + \|x - y\|^2 = 2\|x\|^2 + 2\|y\|^2,$$

using only the three inner-product properties. Interpret the identity for the parallelogram with sides x and y .

Exercise 2.4 (Proof). Show that if $\langle x, v \rangle = 0$ for every $v \in \mathbb{R}^n$, then $x = 0$. (Hint: choose v wisely.)

Exercise 2.5 (NumPy). Draw 10,000 pairs of random unit vectors in $\mathbb{R}^{100}$ (generate with `randn` and `normalize`), compute their cosines, and report the mean and standard deviation. In one or two sentences, explain what this says about “random directions in high dimensions are nearly orthogonal” and why that is reassuring for high-dimensional feature vectors.

Deeper reading. LADR 6A (inner products and norms — the proofs above follow Axler’s); Strang, ch. “Introduction to Vectors” (lengths and dot products).

3 Span, Linear Independence, Basis, Dimension

NEEDED FOR: CS221 (core)

3.1 Theory

Definition 3.1 (Span). The *span* of $v_1, \dots, v_m \in \mathbb{R}^n$ is the set of all their linear combinations:

$$\text{span}(v_1, \dots, v_m) = \{a_1v_1 + \dots + a_mv_m : a_1, \dots, a_m \in \mathbb{R}\}.$$

The span of the empty list is defined to be $\{0\}$. If $\text{span}(v_1, \dots, v_m) = V$, we say the list *spans* V .

A *subspace* of $\mathbb{R}^n$ is a subset containing 0 that is closed under addition and scalar multiplication. Spans are exactly the subspaces you can describe by a finite list: $\text{span}(v_1, \dots, v_m)$ is the smallest subspace containing $v_1, \dots, v_m$ (Exercise 3.4).

Definition 3.2 (Linear independence). A list $v_1, \dots, v_m$ is *linearly independent* if the only choice of scalars with

$$a_1v_1 + \dots + a_mv_m = 0$$

is $a_1 = \dots = a_m = 0$. Otherwise the list is *linearly dependent*.

Equivalently (and this is the useful reading): independence means every vector in the span has *exactly one* representation as a combination. For if $a_1v_1 + \dots + a_mv_m = b_1v_1 + \dots + b_mv_m$, subtracting gives $\sum_k (a_k - b_k)v_k = 0$, so independence forces $a_k = b_k$ for all k .

Lemma 3.3 (Linear dependence lemma, LADR 2.19). *If $v_1, \dots, v_m$ is linearly dependent and $v_1 \neq 0$, then there exists an index $k \in \{2, \dots, m\}$ such that $v_k \in \text{span}(v_1, \dots, v_{k-1})$; moreover, removing v_k from the list leaves the span unchanged.*

Proof. By dependence there are scalars $a_1, \dots, a_m$, not all zero, with $a_1v_1 + \dots + a_mv_m = 0$. Let k be the *largest* index with $a_k \neq 0$. If $k = 1$ we would have $a_1v_1 = 0$ with $a_1 \neq 0$, hence $v_1 = 0$, contradicting our hypothesis; so $k \geq 2$. Solving for v_k :

$$v_k = -\frac{a_1}{a_k}v_1 - \dots - \frac{a_{k-1}}{a_k}v_{k-1} \in \text{span}(v_1, \dots, v_{k-1}).$$

For the second claim: any combination using v_k can substitute this expression for v_k , producing the same vector as a combination of the others. Hence the span without v_k contains the span with it; the reverse containment is clear. $\square$

Theorem 3.4 (Independent lists are no longer than spanning lists, LADR 2.22). *In $\mathbb{R}^n$, if $u_1, \dots, u_p$ is linearly independent and $w_1, \dots, w_q$ spans $\mathbb{R}^n$ (or any subspace containing the u 's), then $p \leq q$.*

Proof. We show that we can trade the w 's for the u 's one at a time, keeping a spanning list of length q at every step.

Step 1. The list $u_1, w_1, \dots, w_q$ is linearly dependent (because u_1 is already in the span of the w 's, it has a nontrivial representation, giving a dependence relation) and its first vector $u_1 \neq 0$ (independent lists cannot contain 0). By Lemma 3.3 we may remove some w_j — the lemma yields an index $k \geq 2$ whose vector lies in the span of its predecessors, and that vector must be a w , not a u , since the u 's alone will always be independent — leaving a spanning list of length q containing u_1 .

Step j . Suppose after $j - 1$ steps we have a spanning list of length q containing $u_1, \dots, u_{j-1}$ (listed first) and $q - j + 1$ of the w 's. Adjoin u_j right after $u_1, \dots, u_{j-1}$: the resulting list of length $q + 1$ is dependent (u_j is in the span of a spanning list). By Lemma 3.3, some vector is in the span of its predecessors; it cannot be one of $u_1, \dots, u_j$, since those are independent. So it is one of the remaining w 's, and removing it keeps a spanning list of length q .

For this process to run through step p , there must be a w available to remove at each step; hence $q \geq p$. $\square$

Definition 3.5 (Basis). A *basis* of a subspace V is a list of vectors in V that is linearly independent and spans V .

Theorem 3.6 (Dimension is well defined, LADR 2.34). *Any two bases of a subspace V have the same length. This common length is called the dimension of V , written $\dim V$.*

Proof. Let B_1 (length p) and B_2 (length q) be bases of V . Since B_1 is independent and B_2 spans V , Theorem 3.4 gives $p \leq q$. Swapping roles gives $q \leq p$. Hence $p = q$. $\square$

Theorem 3.7 (Criterion for a basis). *The list $v_1, \dots, v_m$ is a basis of V if and only if every $v \in V$ can be written in exactly one way as $v = a_1v_1 + \dots + a_mv_m$.*

Proof. $(\Rightarrow)$ Spanning gives at least one representation; independence gives at most one, by the uniqueness argument following the definition of independence. $(\Leftarrow)$ Existence of representations for every v is spanning. Uniqueness applied to $v = 0$ says the only combination equal to 0 is the all-zeros one — independence. $\square$

The *standard basis* of $\mathbb{R}^n$ is $e_1 = (1, 0, \dots, 0), \dots, e_n = (0, \dots, 0, 1)$; hence $\dim \mathbb{R}^n = n$, and now “dimension” finally has a theorem-backed meaning matching the intuition: a line through 0 has dimension 1, a plane through 0 has dimension 2.

Why this matters. Theorem 3.7 is the licence to use *coordinates*: once you fix a basis, every vector *is* a unique list of numbers. The dimension theorem says “number of degrees of freedom” is intrinsic — it does not depend on which basis you happened to pick. These two facts are silently invoked every time CS221 says “represent the state as a vector in $\mathbb{R}^d$.”

3.2 Practice: 2D/3D

Example 3.8 (Checking independence by hand). Are $v_1 = (1, 2, 0)$, $v_2 = (0, 1, 1)$, $v_3 = (1, 4, 2)$ independent? Solve $av_1 + bv_2 + cv_3 = 0$:

$$\begin{aligned} a + c &= 0 \\ 2a + b + 4c &= 0 \implies a = -c, \quad b = -2c, \quad \text{2nd eq: } -2c - 2c + 4c = 0. \checkmark \\ b + 2c &= 0 \end{aligned}$$

The second equation is automatic, so c is free: taking $c = 1$ gives $-v_1 - 2v_2 + v_3 = 0$. *Dependent*, and indeed $v_3 = v_1 + 2v_2$: the third vector lies in the plane spanned by the first two, exactly as the linear dependence lemma predicts.

Example 3.9 (A basis of a plane). The plane $P = \{(x_1, x_2, x_3) : x_1 + x_2 + x_3 = 0\}$ in $\mathbb{R}^3$ is a subspace (check closure!). The vectors $u = (1, -1, 0)$ and $v = (1, 0, -1)$ lie in P , are independent (neither is a multiple of the other), and span P : given (x_1, x_2, x_3) with $x_1 + x_2 + x_3 = 0$, write it as $-x_2u - x_3v$ — check: $-x_2(1, -1, 0) - x_3(1, 0, -1) = (-x_2 - x_3, x_2, x_3) = (x_1, x_2, x_3)$, using $x_1 = -x_2 - x_3$. So $\dim P = 2$: a plane, as geometry demands.

CS221 connection. The set of functions a linear model can represent is the span of its features: with features $\phi_1, \dots, \phi_d$, the hypothesis class is $\{\sum_k w_k \phi_k\}$. Redundant (dependent) features do not enlarge the span — they add parameters without adding expressive power. “Add a feature to make the model more expressive” means “add a vector outside the current span.”

3.3 NumPy

Rank (Lesson 5 defines it properly) counts the dimension of the span, which makes it the quick machine test of independence: m vectors are independent exactly when the matrix having them as columns has rank m .

```
import numpy as np
v1 = np.array([1., 2., 0.]); v2 = np.array([0., 1., 1.])
v3 = np.array([1., 4., 2.])
A = np.column_stack([v1, v2, v3]) # 3x3, columns are the vectors
np.linalg.matrix_rank(A)          # 2 -> dependent (rank < 3)

B = np.column_stack([v1, v2])
np.linalg.matrix_rank(B)          # 2 -> v1, v2 independent
```

`matrix_rank` works via singular values; for an $n \times m$ matrix it costs $O(nm \min(n, m))$ — for our tiny examples, effectively instant; the point is that the test is numerical, not symbolic.

3.4 Exercises

Exercise 3.1 (Hand). Determine whether each list is independent; if dependent, exhibit a dependence relation. (a) $(2, 1), (4, 3)$ in $\mathbb{R}^2$. (b) $(1, 1, 1), (1, 2, 3), (0, 1, 2)$ in $\mathbb{R}^3$. (c) any list containing the zero vector.

Exercise 3.2 (Hand). Find a basis for the line $\{(t, 2t, -t) : t \in \mathbb{R}\}$ and for the plane $x_1 - x_3 = 0$ in $\mathbb{R}^3$. State the dimension of each.

Exercise 3.3 (Proof, ★). Prove that any list of $n + 1$ vectors in $\mathbb{R}^n$ is linearly dependent. (One line, given a theorem from this lesson.) Conclude that at most n features on a dataset of n -dimensional inputs can be “genuinely new” in the sense of enlarging the span at every step.

Exercise 3.4 (Proof). Prove that $\text{span}(v_1, \dots, v_m)$ is a subspace, and that any subspace containing $v_1, \dots, v_m$ contains $\text{span}(v_1, \dots, v_m)$.

Exercise 3.5 (NumPy). Generate a random 3×5 matrix $M = \text{np.random.randn}(3, 5)$. What is `matrix_rank(M)` (and why was that predictable from Theorem 3.4)? Now build a 5-column matrix whose rank is 2 *by construction*, and confirm with NumPy.

Deeper reading. LADR 2A (span and linear independence), 2B (bases), 2C (dimension); Strang, ch. “Vector Spaces and Subspaces” (independence, basis and dimension).

4 Matrices and Linear Maps

NEEDED FOR: CS221 (core)

4.1 Theory: linear maps first

Definition 4.1 (Linear map). A function $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a *linear map* if for all $u, v \in \mathbb{R}^n$ and $\lambda \in \mathbb{R}$:

$$T(u + v) = Tu + Tv \quad \text{and} \quad T(\lambda v) = \lambda Tv.$$

Rotations about the origin, reflections through lines through the origin, scalings, shears, and projections onto subspaces are linear. Translation $v \mapsto v + b$ (for $b \neq 0$) is *not*: it moves 0.

Proposition 4.2. *If T is linear, then $T(0) = 0$.*

Proof. $T(0) = T(0 + 0) = T(0) + T(0)$; add $-T(0)$ to both sides. □

Theorem 4.3 (A linear map is determined by its values on a basis, LADR 3.4). *Let $v_1, \dots, v_n$ be a basis of $\mathbb{R}^n$ and $w_1, \dots, w_n$ any vectors in $\mathbb{R}^m$. Then there exists a unique linear map $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $Tv_k = w_k$ for each k .*

Proof. Existence. Every $x \in \mathbb{R}^n$ has a unique representation $x = a_1v_1 + \dots + a_nv_n$ (Theorem 3.7); define $Tx = a_1w_1 + \dots + a_nw_n$. This is well defined precisely because the representation is unique. Linearity: if $x = \sum a_kv_k$ and $y = \sum b_kv_k$, then $x + y = \sum (a_k + b_k)v_k$ is *the* representation of $x + y$, so $T(x + y) = \sum (a_k + b_k)w_k = Tx + Ty$; homogeneity is similar. Also $Tv_k = w_k$ by construction (take $a_k = 1$, others 0).

Uniqueness. If S is linear with $Sv_k = w_k$, then for $x = \sum a_kv_k$, linearity forces $Sx = \sum a_kSv_k = \sum a_kw_k = Tx$. □

The point. A linear map on $\mathbb{R}^n$ is pinned down by n vectors’ worth of information — its values on a basis. Choosing the standard basis, those values are the *columns of a matrix*. This theorem is why matrices and linear maps are interchangeable.

Definition 4.4 (Matrix of a map; matrix–vector product). For linear $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the *matrix of T* (with respect to standard bases) is the $m \times n$ array A whose k th column is $Te_k \in \mathbb{R}^m$. For $x = (x_1, \dots, x_n)$ we then have, by linearity,

$$Tx = T\left(\sum_k x_k e_k\right) = \sum_k x_k (Te_k),$$

which we *define* to be the product Ax . In words:

Ax = the linear combination of the *columns* of A with coefficients $x_1, \dots, x_n$.

Entrywise, $(Ax)_i = \sum_k A_{i,k} x_k$: row i of A dotted with x . Both readings are used daily; the column reading is the more illuminating one.

Definition 4.5 (Matrix multiplication). For linear maps $S : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $T : \mathbb{R}^p \rightarrow \mathbb{R}^n$, the composition $S \circ T : \mathbb{R}^p \rightarrow \mathbb{R}^m$ is linear (check!). The product AB of the matrix A of S ($m \times n$) and the matrix B of T ($n \times p$) is *defined* as the matrix of $S \circ T$; it is $m \times p$. Column k of AB is $(S \circ T)e_k = A(Be_k) = A(\text{column } k \text{ of } B)$, which yields the familiar formula

$$(AB)_{i,j} = \sum_{k=1}^n A_{i,k} B_{k,j}.$$

Because composition of functions is associative but not commutative, matrix multiplication is associative — $(AB)C = A(BC)$, no computation needed — and in general $AB \neq BA$.

Definition 4.6 (Transpose). The *transpose* of an $m \times n$ matrix A is the $n \times m$ matrix $A^\top$ with $(A^\top)_{i,j} = A_{j,i}$. Two facts used constantly: $(AB)^\top = B^\top A^\top$, and the dot product is a matrix product: $x \cdot y = x^\top y$. The defining property tying transpose to the dot product is

$$\langle Ax, y \rangle = \langle x, A^\top y \rangle \quad \text{for all } x \in \mathbb{R}^n, y \in \mathbb{R}^m,$$

which follows by expanding both sides as $\sum_{i,k} A_{i,k} x_k y_i$.

4.2 Practice: 2D matrices you can see

Example 4.7 (Reading a matrix by its columns).

$$A = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} : \quad Ae_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad Ae_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}.$$

$e_1 \mapsto e_2$ and $e_2 \mapsto -e_1$: this is rotation by 90° counterclockwise. In general, rotation by angle θ has matrix $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ — its columns are where e_1, e_2 land, which is all Theorem 4.3 says you need to know.

Example 4.8 (Hand computation, both readings).

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 0 & 1 \end{pmatrix}, \quad x = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

Column reading: $Ax = 2 \begin{pmatrix} 1 \\ 3 \\ 0 \end{pmatrix} - 1 \begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \\ -1 \end{pmatrix}$. Row reading: $(Ax)_1 = 1 \cdot 2 + 2 \cdot (-1) = 0$, $(Ax)_2 = 6 - 4 = 2$, $(Ax)_3 = 0 - 1 = -1$. Same answer, as it must be.

Example 4.9 (Composition order matters). Let R = rotation by 90° (above) and $S = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ = projection onto the x_1 -axis. Then

$$SR = \begin{pmatrix} 0 & -1 \\ 0 & 0 \end{pmatrix}, \quad RS = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}.$$

Track e_1 through each composition (rightmost map acts first): $SRe_1 = S(Re_1) = S(e_2) = 0$, while $RS e_1 = R(Se_1) = R(e_1) = e_2$. Rotate-then-project kills e_1 ; project-then-rotate sends it to e_2 . Different maps, different matrices: $AB \neq BA$.

CS221 connection. A dataset of m examples with n features is an $m \times n$ matrix X whose *rows* are feature vectors. Then Xw computes all m scores $w \cdot \phi(x_i)$ in one matrix–vector product. A tiny neural network layer is $x \mapsto \sigma(Wx + b)$: a linear map, a shift, a nonlinearity. Reading Ax fluently in both the row picture and the column picture is the single highest-value matrix skill for the course.

4.3 NumPy

```
import numpy as np
A = np.array([[1., 2.], [3., 4.], [0., 1.]]) # 3x2
x = np.array([2., -1.])

A @ x          # matrix-vector product -> array([ 0.,  2., -1.])
A.T            # transpose, 2x3
B = np.random.randn(2, 5)
(A @ B).shape  # (3, 5): (3x2)(2x5) -- inner dims must match
np.allclose((A @ B).T, B.T @ A.T) # True

# Row picture vs column picture, explicitly:
np.allclose(A @ x, x[0]*A[:, 0] + x[1]*A[:, 1]) # columns
np.allclose(A @ x, np.array([A[i, :] @ x for i in range(3)])) # rows
```

Complexity worth knowing cold: Ax for an $m \times n$ matrix is $O(mn)$ (touch every entry once); AB for $(m \times n)(n \times p)$ is $O(mnp)$ by the definition — mp entries, each an n -term dot product. Multiplying two 1000×1000 matrices is a billion operations; NumPy does it in well under a second, a Python triple loop would take minutes. This is why CS221 code is written matrix-first.

4.4 Exercises

Exercise 4.1 (Hand). Write down the 2×2 matrices of: (a) reflection through the line $x_2 = x_1$; (b) scaling by 3 in the x_1 direction only; (c) rotation by 180° . (Use the columns-are-images-of- e_k principle; no formulas to memorize.)

Exercise 4.2 (Hand). With $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ (a *shear*) and $B = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$, compute AB , BA , and $A^T B^T$. Check $(BA)^T = A^T B^T$.

Exercise 4.3 (Proof, $\star$). Prove that the composition of linear maps is linear, and that $\langle Ax, y \rangle = \langle x, A^T y \rangle$ for all x, y by expanding both sides in coordinates.

Exercise 4.4 (Proof). Using Theorem 4.3, prove: if two linear maps $S, T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ agree on a basis, they are equal. Then explain in one sentence why an $m \times n$ matrix and a linear map $\mathbb{R}^n \rightarrow \mathbb{R}^m$ carry exactly the same information.

Exercise 4.5 (NumPy). Build the rotation matrix $R(\theta)$ for $\theta = 30^\circ$ and verify numerically that $R(30^\circ)R(30^\circ) = R(60^\circ)$ and that $R^\top R = I$. What does $R^\top R = I$ say about what rotations do to lengths and dot products? (Hint: $\langle Rx, Ry \rangle = \langle x, R^\top Ry \rangle$.)

Deeper reading. LADR 3A (linear maps), 3C (matrices, matrix multiplication); Strang, chs. “Multiplying Matrices” / “Solving Linear Equations” (the idea of elimination, matrix operations).

5 Null Space, Rank, and Solving $Ax = b$

NEEDED FOR: CS221 (core)

5.1 Theory: the fundamental theorem

Throughout, $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is linear with matrix A .

Definition 5.1 (Null space and range).

$$\text{null } T = \{x \in \mathbb{R}^n : Tx = 0\}, \quad \text{range } T = \{Tx : x \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

For the matrix A , $\text{range } T$ is the span of the columns of A (by the column reading of Ax), also called the *column space*; $\text{null } T$ is the solution set of $Ax = 0$. Both are subspaces (Exercise 5.4). The *rank* of A is $\text{rank } A = \dim \text{range } T$.

Proposition 5.2 (Injectivity $\Leftrightarrow$ trivial null space, LADR 3.15). *T is injective (one-to-one) if and only if $\text{null } T = \{0\}$.*

Proof. $(\Rightarrow)$ $T0 = 0$; if $Tx = 0$ too, injectivity gives $x = 0$. $(\Leftarrow)$ If $Tu = Tv$, linearity gives $T(u - v) = 0$, so $u - v \in \text{null } T = \{0\}$, i.e. $u = v$. $\square$

This tiny proposition is used constantly: to check whether a linear map ever collapses two inputs together, you only have to look at what it sends to zero.

Theorem 5.3 (Fundamental theorem of linear maps / rank–nullity, LADR 3.21). *For linear $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$,*

$$\dim \text{null } T + \dim \text{range } T = n.$$

Proof. Let $u_1, \dots, u_k$ be a basis of $\text{null } T$ (so $\dim \text{null } T = k$). Extend it to a basis $u_1, \dots, u_k, v_1, \dots, v_r$ of $\mathbb{R}^n$ (add vectors outside the current span one at a time until spanning; independence is preserved at each step by the linear dependence lemma). Then $n = k + r$, and it suffices to show $Tv_1, \dots, Tv_r$ is a basis of $\text{range } T$.

Spanning: any $x \in \mathbb{R}^n$ is $x = \sum a_i u_i + \sum b_j v_j$, so

$$Tx = \sum a_i \underbrace{Tu_i}_{=0} + \sum b_j Tv_j = \sum b_j Tv_j,$$

hence every vector of range T is a combination of the Tv_j .

Independence: suppose $\sum_j c_j Tv_j = 0$. By linearity $T(\sum_j c_j v_j) = 0$, so $\sum_j c_j v_j \in \text{null } T$, hence $\sum_j c_j v_j = \sum_i d_i u_i$ for some scalars d_i . Rearranged, $\sum_j c_j v_j - \sum_i d_i u_i = 0$ is a linear relation on the basis of $\mathbb{R}^n$, so all c_j (and d_i) are zero. $\square$

Corollary 5.4 (The square case is all-or-nothing). *For $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$: injective $\Leftrightarrow$ surjective $\Leftrightarrow$ invertible.*

Proof. By Theorem 5.3, $\dim \text{null } T = 0 \iff \dim \text{range } T = n \iff \text{range } T = \mathbb{R}^n$ (a subspace of $\mathbb{R}^n$ of dimension n must be $\mathbb{R}^n$: take a basis of the subspace; it is an independent list of length n in $\mathbb{R}^n$, and extending it to a basis of $\mathbb{R}^n$ cannot add anything by Theorem 3.4). So injective $\iff$ surjective, and a bijective linear map has a linear inverse (Exercise 5.5). A square matrix A is *invertible* when such A^{-1} exists: $A^{-1}A = AA^{-1} = I$. $\square$

What the fundamental theorem says. An $m \times n$ matrix consumes n input dimensions. Each dimension either survives into the output (contributing to rank) or is crushed to zero (contributing to the null space) — and nothing is lost or created: crushed + surviving = n . Consequences you can now read off instantly: a map to a smaller space ($m < n$) must crush something ($\dim \text{null } T \geq n - m > 0$), so it cannot be injective; a map from a smaller space ($n < m$) has rank at most $n < m$, so it cannot be surjective.

Solving $Ax = b$. The complete picture, now provable in three lines:

1. $Ax = b$ has a solution $\iff b \in \text{range } T$ (that is what range means).
2. If x_p is one particular solution, the full solution set is $x_p + \text{null } T = \{x_p + h : Ah = 0\}$: for if $Ax = b = Ax_p$ then $A(x - x_p) = 0$; conversely $A(x_p + h) = b + 0 = b$.
3. Hence the solution is unique iff $\text{null } T = \{0\}$; for square A , Corollary 5.4 says a solution then exists for every b , namely $x = A^{-1}b$.

5.2 Practice: elimination by hand

Gaussian elimination is Strang's centerpiece: subtract multiples of rows to reach triangular form, then back-substitute. Row operations do not change the solution set (each is reversible and preserves the equations).

Example 5.5 (A unique solution). Solve $Ax = b$:

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 5 & 4 \\ 1 & 1 & 0 \end{pmatrix} x = \begin{pmatrix} 3 \\ 8 \\ 1 \end{pmatrix}.$$

$R_2 \leftarrow R_2 - 2R_1$ and $R_3 \leftarrow R_3 - R_1$:

$$\begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ 0 & -1 & -1 \end{pmatrix} x = \begin{pmatrix} 3 \\ 2 \\ -2 \end{pmatrix}; \quad R_3 \leftarrow R_3 + R_2 : \quad \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix} x = \begin{pmatrix} 3 \\ 2 \\ 0 \end{pmatrix}.$$

Back-substitute: $x_3 = 0$, then $x_2 = 2 - 2x_3 = 2$, then $x_1 = 3 - 2x_2 - x_3 = -1$. Solution $x = (-1, 2, 0)$; three pivots, so rank 3, null space $\{0\}$, solution unique — the theory and the algorithm agree.

Example 5.6 (Null space by hand). Find $\text{null } A$ for $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$. The second row is twice the first, so $Ax = 0$ reduces to the single equation $x_1 + 2x_2 + 3x_3 = 0$: solve $x_1 = -2x_2 - 3x_3$ with x_2, x_3 free. Null space = $\text{span}((-2, 1, 0), (-3, 0, 1))$, dimension 2. Rank check: the columns of A are all multiples of $(1, 2)$, so $\text{rank} = 1$, and $1 + 2 = 3 = n$. ✓

CS221 connection. “Does a weight vector exist that fits these constraints, and is it unique?” is exactly existence/uniqueness for a linear system. A nontrivial null space of the feature matrix means different weight vectors produce identical predictions on the training set — the model is over-parameterized on that data. Rank tells you the number of effective directions your data spans.

5.3 NumPy

```
import numpy as np
A = np.array([[1., 2., 1.], [2., 5., 4.], [1., 1., 0.]])
b = np.array([3., 8., 1.])

x = np.linalg.solve(A, b)          # [-1.,  2.,  0.]
np.allclose(A @ x, b)              # True
np.linalg.matrix_rank(A)           # 3

# Never do this in real code:
x_bad = np.linalg.inv(A) @ b       # works, but slower & less accurate
```

`solve` runs elimination (LU factorization) in $O(n^3)$ for an $n \times n$ system — the same arithmetic you did by hand, industrialized. Prefer `solve` over `inv`: computing the full inverse costs about three times as much and amplifies rounding error; you almost never need the inverse itself, only the solution.

5.4 Exercises

Exercise 5.1 (Hand). Solve by elimination, tracking pivots: $\begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix} x = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$, then $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} x = \begin{pmatrix} 3 \\ 6 \end{pmatrix}$, then the same left side with right side $(3, 7)$. Describe all solutions in each case (unique / infinitely many / none) and reconcile with rank and the three-step picture above.

Exercise 5.2 (Hand). For $A = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \end{pmatrix}$, find a basis of $\text{null } A$ and a basis of the column space; verify rank-nullity.

Exercise 5.3 (Proof, ★). Prove Proposition 5.2’s consequence for systems: $Ax = b$ has *at most one* solution for every b if and only if $Ax = 0$ has only the zero solution.

Exercise 5.4 (Proof). Prove that $\text{null } T$ and $\text{range } T$ are subspaces.

Exercise 5.5 (Proof). Suppose $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is linear and bijective. Prove that its inverse function T^{-1} is linear. (Apply T to both sides of what you want to show.)

Exercise 5.6 (NumPy). Create the 2×3 matrix from the null-space example. Use `np.linalg.lstsq` or SciPy-free reasoning: verify that both basis vectors you found by hand satisfy $Ax = 0$ with

`np.allclose`, and that `matrix_rank(A)` is 1. Then generate a random $b \in \mathbb{R}^2$ and decide numerically whether $Ax = b$ is solvable (hint: compare `matrix_rank(A)` with the rank of the augmented matrix `np.column_stack([A, b])`).

Deeper reading. LADR 3B (null spaces and ranges, fundamental theorem), 3D (invertibility); Strang, chs. “Solving Linear Equations” (elimination) and “Vector Spaces and Subspaces” (the null space, rank).

6 Orthogonal Projections and Least Squares

NEEDED FOR: CS221 (core); reused in Lesson 22 for MMSE estimation (EE278)

6.1 Theory

Definition 6.1 (Orthonormal list; orthogonal complement). A list $e_1, \dots, e_k$ is *orthonormal* if $\langle e_i, e_j \rangle = 0$ for $i \neq j$ and $\|e_i\| = 1$ for all i . For a subspace $U \subseteq \mathbb{R}^n$, the *orthogonal complement* is

$$U^\perp = \{x \in \mathbb{R}^n : \langle x, u \rangle = 0 \text{ for all } u \in U\}.$$

Proposition 6.2 (Orthonormal lists are independent, LADR 6.25). *Every orthonormal list is linearly independent, and if $v = a_1 e_1 + \dots + a_k e_k$ then $a_j = \langle v, e_j \rangle$.*

Proof. Take the inner product of $v = \sum_i a_i e_i$ with e_j : by linearity, $\langle v, e_j \rangle = \sum_i a_i \langle e_i, e_j \rangle = a_j$, since all terms vanish except $i = j$, where $\langle e_j, e_j \rangle = 1$. For independence, apply this with $v = 0$: every coefficient equals $\langle 0, e_j \rangle = 0$. $\square$

So orthonormal bases make coordinates *free*: no system-solving, just k dot products.

Theorem 6.3 (Orthogonal projection). *Let $e_1, \dots, e_k$ be an orthonormal basis of a subspace $U \subseteq \mathbb{R}^n$ (one always exists: apply the Gram–Schmidt procedure below to any basis). Define $P_U : \mathbb{R}^n \rightarrow \mathbb{R}^n$ by*

$$P_U x = \langle x, e_1 \rangle e_1 + \dots + \langle x, e_k \rangle e_k.$$

Then for every x : (a) $P_U x \in U$ and $x - P_U x \in U^\perp$; (b) $P_U x$ is the unique closest point of U to x :

$$\|x - P_U x\| < \|x - u\| \quad \text{for every } u \in U \text{ with } u \neq P_U x.$$

Proof. (a) $P_U x \in U$ since it is a combination of the e_i . For the second part, for each j ,

$$\langle x - P_U x, e_j \rangle = \langle x, e_j \rangle - \sum_i \langle x, e_i \rangle \langle e_i, e_j \rangle = \langle x, e_j \rangle - \langle x, e_j \rangle = 0,$$

and a vector orthogonal to each e_j is orthogonal to every combination of them, i.e. to all of U .

(b) Let $u \in U$. Write

$$x - u = \underbrace{(x - P_U x)}_{\in U^\perp} + \underbrace{(P_U x - u)}_{\in U}.$$

The two pieces are orthogonal, so by the Pythagorean theorem

$$\|x - u\|^2 = \|x - P_U x\|^2 + \|P_U x - u\|^2 \geq \|x - P_U x\|^2,$$

with equality iff $\|P_U x - u\| = 0$, i.e. $u = P_U x$. $\square$

One picture to keep. Drop a perpendicular from x to the subspace U ; its foot is $P_U x$. The proof of (b) is the Pythagorean theorem applied to the right triangle with vertices x , $P_U x$, and any other candidate u . Every least-squares fact below is this picture.

Gram–Schmidt in brief (LADR 6.32). Given an independent list $v_1, \dots, v_k$, define $e_1 = v_1/\|v_1\|$ and, inductively, subtract from v_j its projection onto the span of the previous e 's, then normalize:

$$f_j = v_j - \sum_{i < j} \langle v_j, e_i \rangle e_i, \quad e_j = f_j / \|f_j\|.$$

Each $f_j \neq 0$ (else v_j would be in the span of its predecessors, contradicting independence), each e_j is orthogonal to the previous ones by the same computation as in Theorem 6.3(a), and the spans agree at every stage.

Theorem 6.4 (Least squares / normal equations). *Let A be $m \times n$ with linearly independent columns, and $b \in \mathbb{R}^m$. Then the minimization problem*

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|^2$$

has the unique solution given by the normal equations

$$A^T A \hat{x} = A^T b.$$

Proof. As x ranges over $\mathbb{R}^n$, the vector Ax ranges exactly over $U = \text{range } A$ (the column space). By Theorem 6.3(b), the closest point of U to b is the unique vector $P_U b$; so minimizers are exactly the solutions of $A\hat{x} = P_U b$. Independence of the columns means A is injective on $\mathbb{R}^n$ (Proposition 5.2 applied to combinations of columns), so exactly one $\hat{x}$ satisfies this.

It remains to convert “ $A\hat{x}$ is the projection of b ” into equations. By Theorem 6.3(a), $A\hat{x} = P_U b$ iff the residual $b - A\hat{x}$ is orthogonal to U , i.e. orthogonal to every column of A , i.e.

$$A^T(b - A\hat{x}) = 0 \iff A^T A \hat{x} = A^T b.$$

(For the converse direction of the iff: if $A^T(b - A\hat{x}) = 0$, then $b - A\hat{x}$ is orthogonal to each column of A , hence to all of U , and adding $A\hat{x} \in U$ shows $A\hat{x}$ is the orthogonal projection of b , by the uniqueness of the decomposition $b = P_U b + (b - P_U b)$ into a U -part plus a $U^\perp$ -part.)

Finally, $A^T A$ is invertible when A has independent columns: if $A^T A x = 0$ then $0 = \langle A^T A x, x \rangle = \langle Ax, Ax \rangle = \|Ax\|^2$, so $Ax = 0$, so $x = 0$ by injectivity; now apply Corollary 5.4 to the square matrix $A^T A$. $\square$

6.2 Practice: fitting a line by hand

Example 6.5 (Least-squares line through three points). Fit $y \approx c + dt$ to the data points $(t, y) = (0, 1), (1, 2), (2, 4)$. In matrix form, we want $Ax \approx b$ with unknowns $x = (c, d)$:

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 4 \end{pmatrix}.$$

No exact solution exists (the three points are not collinear — check: slopes 1 and 2). Normal equations:

$$A^T A = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad A^T b = \begin{pmatrix} 7 \\ 10 \end{pmatrix} \implies \begin{cases} 3c + 3d = 7 \\ 3c + 5d = 10 \end{cases} \implies d = \frac{3}{2}, \quad c = \frac{7-3d}{3} = \frac{5}{6}.$$

Best line: $y = \frac{5}{6} + \frac{3}{2}t$. Residual: $b - A\hat{x} = (1, 2, 4) - (\frac{5}{6}, \frac{7}{3}, \frac{23}{6}) = (\frac{1}{6}, -\frac{1}{3}, \frac{1}{6})$. Sanity checks: the residual sums to zero (orthogonal to the all-ones first column) and $\langle \text{residual}, (0, 1, 2) \rangle = 0 - \frac{1}{3} + \frac{1}{3} = 0$ (orthogonal to the second column). The residual is orthogonal to the column space — the picture made numerical.

CS221 connection. Linear regression with the squared loss *is* this theorem: training loss $= \frac{1}{m} \|Xw - y\|^2$, and the closed-form minimizer solves $X^T X w = X^T y$. CS221 will usually minimize by gradient descent instead (Lesson 7) — but knowing the geometry (prediction = projection of the label vector onto the span of the features; residual $\perp$ features) is what makes regression make sense.

6.3 NumPy

```
import numpy as np
A = np.array([[1., 0.], [1., 1.], [1., 2.]])
b = np.array([1., 2., 4.])

xhat, res, rank, sv = np.linalg.lstsq(A, b, rcond=None)
xhat                                     # [0.8333..., 1.5] = (5/6, 3/2)

# Normal equations by hand (fine for small, well-conditioned problems):
xhat2 = np.linalg.solve(A.T @ A, A.T @ b)
np.allclose(xhat, xhat2)                # True
r = b - A @ xhat
np.allclose(A.T @ r, 0)                 # True: residual orthogonal to columns
```

`lstsq` uses an orthogonalization-based method (QR/SVD) rather than forming $A^T A$, which roughly squares the conditioning of the problem; for $m \times n$ with $m \geq n$ the cost is $O(mn^2)$. Rule of thumb: `lstsq` in code, normal equations on paper.

6.4 Exercises

Exercise 6.1 (Hand). Run Gram–Schmidt on $v_1 = (1, 1, 0)$, $v_2 = (1, 0, 1)$ to produce an orthonormal basis e_1, e_2 of their span U . Then compute $P_U x$ for $x = (0, 0, 3)$ and verify $x - P_U x$ is orthogonal to both e_1 and e_2 .

Exercise 6.2 (Hand). Fit the least-squares line to $(0, 0)$, $(1, 1)$, $(2, 1)$, $(3, 2)$ via the normal equations. Verify the residual is orthogonal to both columns of your A .

Exercise 6.3 (Proof, ★). Prove that $\mathbb{R}^n = U \oplus U^\perp$ for every subspace U ; that is, every x is uniquely a sum of a vector in U and a vector in $U^\perp$. (Existence: Theorem 6.3(a). Uniqueness: show $U \cap U^\perp = \{0\}$ — what is $\langle v, v \rangle$ for v in both?)

Exercise 6.4 (Proof). Show $\langle P_U x, y \rangle = \langle x, P_U y \rangle$ for all x, y (projections are self-adjoint), and $P_U(P_U x) = P_U x$ (idempotent), directly from the formula in Theorem 6.3.

Exercise 6.5 (NumPy). Generate noisy data with `t = np.linspace(0, 1, 50)` and `y = 2 + 3*t + 0.1*np.random.randn(50)`. Build A with columns $(\mathbf{1}, t)$, solve with `lstsq`, and check the recovered coefficients are near $(2, 3)$. Then add a duplicated column (t again) and observe what happens to `matrix_rank(A)` and to the normal-equations route — explain the failure using Theorem 6.4’s independence hypothesis.

Deeper reading. LADR 6B (orthonormal bases, Gram–Schmidt), 6C (orthogonal complements, minimization — Axler solves this exact least-squares problem); Strang, ch. “Orthogonality” (projections; least squares approximations).

7 Gradients for Machine Learning

NEEDED FOR: CS221 (core)

This lesson is the bridge from linear algebra to CS221’s first four lectures. It is Math 51’s multivariable differentiation in the minimal dose: gradients of the two function shapes machine learning actually optimizes.

7.1 Theory

Definition 7.1 (Gradient). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. The *partial derivative* $\frac{\partial f}{\partial x_k}(x)$ is the ordinary derivative of $t \mapsto f(x + te_k)$ at $t = 0$. If all partials exist and are continuous, the *gradient* is the vector

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x), \dots, \frac{\partial f}{\partial x_n}(x) \right) \in \mathbb{R}^n,$$

and f admits the first-order approximation $f(x + h) \approx f(x) + \langle \nabla f(x), h \rangle$ for small h .

Proposition 7.2 (Steepest ascent). *Suppose $\nabla f(x) \neq 0$. Among all unit vectors u , the directional rate of change $\langle \nabla f(x), u \rangle$ is maximized uniquely at $u = \nabla f(x) / \|\nabla f(x)\|$ and minimized uniquely at its negative.*

Proof. By Cauchy–Schwarz (Theorem 2.5), $|\langle \nabla f(x), u \rangle| \leq \|\nabla f(x)\| \|u\| = \|\nabla f(x)\|$, with equality iff u is a scalar multiple of $\nabla f(x)$; the unit multiples are $\pm \nabla f(x) / \|\nabla f(x)\|$, giving the maximum with sign $+$ and the minimum with sign $-$. $\square$

That is the entire justification for *gradient descent*: step against the gradient,

$$x^{(t+1)} = x^{(t)} - \eta \nabla f(x^{(t)}),$$

because $-\nabla f$ is locally the fastest way downhill. ($\eta > 0$ is the *step size* or *learning rate*.)

Proposition 7.3 (The two gradients to memorize). *Fix $a \in \mathbb{R}^n$ and a symmetric $n \times n$ matrix M (meaning $M^\top = M$). Then*

$$(i) \nabla_x \langle a, x \rangle = a; \quad (ii) \nabla_x (x^\top M x) = 2Mx.$$

Proof. (i) $\langle a, x \rangle = \sum_k a_k x_k$, so $\partial/\partial x_k$ picks out a_k ; assembling the partials gives a .

(ii) Write $g(x) = x^\top Mx = \sum_{i,j} M_{i,j} x_i x_j$. Differentiating with respect to x_k : the terms containing x_k are those with $i = k$ or $j = k$ (the term $i = j = k$ is $M_{k,k} x_k^2$, derivative $2M_{k,k} x_k$, consistent with counting it in both groups once each):

$$\frac{\partial g}{\partial x_k} = \sum_j M_{k,j} x_j + \sum_i M_{i,k} x_i = (Mx)_k + (M^\top x)_k = 2(Mx)_k,$$

using symmetry in the last step. Assembling over k gives $2Mx$. $\square$

Corollary 7.4 (Gradient of the squared-error loss). *For $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, the function $f(x) = \|Ax - b\|^2$ has*

$$\nabla f(x) = 2A^\top(Ax - b).$$

Consequently $\nabla f(\hat{x}) = 0 \iff A^\top A\hat{x} = A^\top b$: the normal equations of Lesson 6 are exactly the vanishing-gradient condition.

Proof. Expand using $\|v\|^2 = \langle v, v \rangle$, linearity, and $\langle Ax, b \rangle = \langle x, A^\top b \rangle$ (Lesson 4):

$$f(x) = \langle Ax - b, Ax - b \rangle = x^\top (A^\top A)x - 2\langle A^\top b, x \rangle + \|b\|^2.$$

The matrix $A^\top A$ is symmetric since $(A^\top A)^\top = A^\top A$. Now apply Proposition 7.3: the quadratic term contributes $2A^\top Ax$, the linear term $-2A^\top b$, the constant nothing. Sum: $2A^\top(Ax - b)$. $\square$

Two roads, one summit. Lesson 6 found the least-squares solution by geometry (projection: residual $\perp$ columns). This lesson found it by calculus (set the gradient to zero). They agree — $A^\top(A\hat{x} - b) = 0$ either way — and CS221 takes a third road to the same place: iterate $w \leftarrow w - \eta \nabla f(w)$, which for this convex f converges to the same $\hat{x}$ (for suitably small η). Recognizing all three as one is the payoff of this booklet.

7.2 Practice: by hand

Example 7.5 (A full gradient computation). $f(w_1, w_2) = (3w_1 + 2w_2 - 5)^2$. Chain rule per coordinate: with $r = 3w_1 + 2w_2 - 5$,

$$\frac{\partial f}{\partial w_1} = 2r \cdot 3, \quad \frac{\partial f}{\partial w_2} = 2r \cdot 2, \quad \nabla f = 2r(3, 2).$$

This matches Corollary 7.4 with $A = \begin{pmatrix} 3 & 2 \end{pmatrix}$ (one row), $b = 5$: $\nabla f = 2A^\top(Aw - b)$. Note the gradient is $2r$ times the feature vector $(3, 2)$ — *residual times features*, the shape of every gradient-descent update in CS221's learning lectures.

Example 7.6 (Two steps of gradient descent by hand). Minimize $f(w) = (w_1 - 1)^2 + 4(w_2)^2$, i.e. $w^\top Mw - 2w_1 + 1$ with $M = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}$; so $\nabla f(w) = (2(w_1 - 1), 8w_2)$. Start at $w^{(0)} = (0, 1)$ with $\eta = 0.1$:

$$\begin{aligned} \nabla f(w^{(0)}) &= (-2, 8) \Rightarrow w^{(1)} = (0, 1) - 0.1(-2, 8) = (0.2, 0.2); \\ \nabla f(w^{(1)}) &= (-1.6, 1.6) \Rightarrow w^{(2)} = (0.2, 0.2) - 0.1(-1.6, 1.6) = (0.36, 0.04). \end{aligned}$$

Both coordinates march toward the minimizer $(1, 0)$ — the steep w_2 direction (curvature 4) converges faster at this η , and would oscillate if η were too large (check: the w_2 update is $w_2 \leftarrow (1 - 0.8)w_2$ here, and $\eta > 0.25$ makes $|1 - 8\eta| > 1$). Step-size sensitivity is a lived experience in CS221 homework; this two-line example is its essence.

CS221 connection. The learning lectures define the training loss as an average of per-example losses,

$$\text{TrainLoss}(w) = \frac{1}{m} \sum_i \text{Loss}(x_i, y_i, w),$$

and minimize it by (stochastic) gradient descent. For the squared loss, $\nabla_w \text{Loss} = 2(w \cdot \phi(x) - y)\phi(x)$: residual times feature vector, exactly the pattern above. For the hinge loss on a misclassified example, $\nabla_w \text{Loss} = -y\phi(x)$. Every update you will implement has the form $w \leftarrow w - \eta(\text{scalar})\phi(x)$: a scalar–vector product plus a vector subtraction, both $O(n)$: one cheap pass over the features per example, which is why SGD scales to huge datasets.

7.3 NumPy

```
import numpy as np

# Gradient descent on  $f(x) = \|Ax - b\|^2$ 
A = np.array([[1., 0.], [1., 1.], [1., 2.]])
b = np.array([1., 2., 4.])

def grad(x):
    return 2 * A.T @ (A @ x - b)          # Corollary 7.4

x = np.zeros(2); eta = 0.05
for t in range(500):
    x = x - eta * grad(x)

x                                     # -> [0.8333, 1.5], same as lstsq!
np.allclose(x, np.linalg.lstsq(A, b, rcond=None)[0], atol=1e-4) # True
```

Each iteration costs two matrix–vector products, $O(mn)$ — compare $O(mn^2)$ for the direct `lstsq`. When n is large and you only need modest accuracy, iterating wins; this trade-off is the reason machine learning runs on gradient descent rather than on exact solvers.

7.4 Exercises

Exercise 7.1 (Hand). Compute ∇f for: (a) $f(w) = \langle (1, -2, 3), w \rangle$; (b) $f(w) = \|w\|^2$; (c) $f(w) = (w \cdot \phi - y)^2$ for fixed $\phi \in \mathbb{R}^n$, $y \in \mathbb{R}$ (answer in terms of the residual).

Exercise 7.2 (Hand). Run two iterations of gradient descent on $f(w) = (2w_1 + w_2 - 3)^2$ from $w^{(0)} = (0, 0)$ with $\eta = 0.1$, by hand. Does the loss decrease at each step? (Compute it.)

Exercise 7.3 (Proof, $\star$). Prove Corollary 7.4 a second way, without Proposition 7.3: expand $f(x + h) = \|A(x + h) - b\|^2$ into $f(x) + \langle g, h \rangle + \|Ah\|^2$ for an explicit vector g , and identify $\nabla f(x) = g$ from the first-order approximation. (This perturbation technique is how gradients are computed cleanly in general.)

Exercise 7.4 (Proof). Show that for symmetric M , the function $g(x) = x^\top Mx$ satisfies $g(x + h) - g(x) - \langle 2Mx, h \rangle = h^\top Mh$ exactly (no approximation), and explain how this re-proves Proposition 7.3(ii).

Exercise 7.5 (NumPy, ★). Implement *stochastic* gradient descent for the data in the NumPy block: at each step pick one random row a_i of A and update $x \leftarrow x - \eta \cdot 2(a_i \cdot x - b_i) a_i$. Compare its trajectory (loss vs. iteration) with full gradient descent. One sentence: why is each SGD step m times cheaper?

Deeper reading. Strang, ch. “Orthogonality” again (least squares) — and for the calculus, any Math 51-style text; the CS221 foundations homework (HW1) exercises exactly these gradient patterns.

8 Capstone: Linear Regression from Scratch

NEEDED FOR: CS221 (core)

One project, forty-five minutes, everything at once. Type it, don’t copy-paste.

Task. Generate synthetic data $y = w^* \cdot \phi(t) + \text{noise}$ with $\phi(t) = (1, t, t^2)$ and $w^* = (1, -2, 0.5)$; recover w^* three ways; verify the three answers coincide.

```
import numpy as np
rng = np.random.default_rng(0)

# --- data ---
m = 200
t = rng.uniform(-3, 3, size=m)
Phi = np.column_stack([np.ones(m), t, t**2]) # m x 3 feature matrix
w_star = np.array([1.0, -2.0, 0.5])
y = Phi @ w_star + 0.3 * rng.standard_normal(m)

# --- way 1: normal equations (Lesson 6, geometry) ---
w1 = np.linalg.solve(Phi.T @ Phi, Phi.T @ y)

# --- way 2: library least squares ---
w2 = np.linalg.lstsq(Phi, y, rcond=None)[0]

# --- way 3: gradient descent (Lesson 7, calculus) ---
def loss(w): return np.mean((Phi @ w - y)**2)
def grad(w): return 2/m * Phi.T @ (Phi @ w - y)

w3 = np.zeros(3)
for step in range(20000):
    w3 -= 0.01 * grad(w3)

for name, w in [("normal eq", w1), ("lstsq", w2), ("grad desc", w3)]:
    print(f"{name:10s} {np.round(w, 3)} loss={loss(w):.4f}")
print("true", w_star)
```

Checkpoints (do all of these).

1. All three w 's agree to ~ 3 decimals and sit *near* (not exactly at) w^* . Why not exactly? The noise: you recover the projection of y onto the span of the features, and y itself left that span when noise was added.
2. Compute the residual $r = y - \Phi w_1$ and check `np.allclose(Phi.T @ r, 0, atol=1e-8)`: the residual is orthogonal to all three feature columns (Lesson 6's picture).
3. `np.linalg.matrix_rank(Phi)` is 3 — the features are independent (Lesson 3), which is what licensed `solve` on $\Phi^T \Phi$ (Theorem 6.4).
4. Break it on purpose: append a fourth feature column equal to `2*t` (dependent!). Watch `matrix_rank` stay 3 while the normal-equations route fails or misbehaves, and `lstsq` still returns an answer — now one of infinitely many minimizers, since the null space is nontrivial (Lesson 5: solution set $= \hat{x} + \text{null } \Phi$).
5. Raise η to 0.2: gradient descent diverges (Lesson 7's step-size discussion, now lived).

If every checkpoint makes sense — not just runs, but *makes sense* — you are ready for CS221's learning lectures and HW1.

Final self-check list for Part I

You should be able to, from memory:

- compute dot products, norms, cosines; project a vector onto a line (Lessons 1–2);
- decide independence of small lists; produce a basis of a described line/plane (Lesson 3);
- multiply matrices; read Ax as a combination of columns; state why matrices = linear maps (Lesson 4);
- state and prove rank-nullity; describe the full solution set of $Ax = b$ (Lesson 5);
- derive the normal equations two ways: residual-orthogonality and $\nabla f = 0$ (Lessons 6–7);
- compute $\nabla \langle a, x \rangle$, $\nabla \|Ax - b\|^2$, and hand-run two steps of gradient descent (Lesson 7);
- implement all of the above in NumPy without looking anything up (Capstone).

Part I deliberately omitted determinants, eigenvalues, diagonalization, SVD, and complex vector spaces because CS221 does not need them. They are exactly what EE261 does need — so they are Part II. When the CS221 linear-algebra sprint is done, go to Part III (probability, which CS221 *does* need) next; return to Part II in the weeks before EE261.

Part II: Through Chapter 9, Toward EE261

9 Complex Vectors and Complex Exponentials

NEEDED FOR: EE261 (core); EE278 background. Not needed for CS221.

Fourier analysis lives over the complex numbers: the basic signals are the complex exponentials $e^{i\omega t}$, and the clean statements of orthogonality, eigenvalues, and the DFT all require $\mathbb{C}$. This lesson upgrades Lessons 1–2 from $\mathbb{R}$ to $\mathbb{C}$.

9.1 Theory

Recall $\mathbb{C} = \{a + bi : a, b \in \mathbb{R}\}$ with $i^2 = -1$; the *conjugate* of $z = a + bi$ is $\bar{z} = a - bi$ and the *modulus* is $|z| = \sqrt{a^2 + b^2} = \sqrt{z\bar{z}}$. Euler's formula $e^{i\theta} = \cos \theta + i \sin \theta$ says $e^{i\theta}$ is the point on the unit circle at angle θ ; thus $|e^{i\theta}| = 1$, $\overline{e^{i\theta}} = e^{-i\theta}$, and $e^{i\theta} e^{i\varphi} = e^{i(\theta+\varphi)}$ (angles add — multiplication by $e^{i\theta}$ is rotation by θ).

The space $\mathbb{C}^n$ is defined exactly like $\mathbb{R}^n$ with complex coordinates and complex scalars; everything in Lessons 1 and 3–5 (span, independence, basis, dimension, linear maps, rank-nullity) goes through verbatim with $\mathbb{F} = \mathbb{C}$ — the proofs never used anything about $\mathbb{R}$ beyond field arithmetic. The one definition that must change is the inner product.

Definition 9.1 (Inner product on $\mathbb{C}^n$). For $x, y \in \mathbb{C}^n$,

$$\langle x, y \rangle = x_1 \bar{y}_1 + \cdots + x_n \bar{y}_n.$$

Why the conjugate? Because we need $\langle x, x \rangle = \sum_k x_k \bar{x}_k = \sum_k |x_k|^2$ to be a nonnegative real number, so that $\|x\| = \sqrt{\langle x, x \rangle}$ is a genuine length. Without the conjugate, $\langle x, x \rangle$ could be negative or non-real (try $x = (i)$ in $\mathbb{C}^1$: $i \cdot i = -1$).

The price is that symmetry becomes *conjugate symmetry* and linearity holds only in the first slot:

$$\langle y, x \rangle = \overline{\langle x, y \rangle}, \quad \langle \lambda x, y \rangle = \lambda \langle x, y \rangle, \quad \langle x, \lambda y \rangle = \bar{\lambda} \langle x, y \rangle.$$

(Each is a one-line check.) With these amendments, every proof of Lesson 2 survives unchanged in structure: positivity, the orthogonal decomposition lemma, Cauchy–Schwarz $|\langle x, y \rangle| \leq \|x\| \|y\|$, the triangle inequality, and the Pythagorean theorem for orthogonal vectors ($\langle x, y \rangle = 0$) all hold in $\mathbb{C}^n$, and Lesson 6's orthonormal-basis and projection theory holds verbatim with $\langle \cdot, \cdot \rangle$ in this complex sense. This is LADR 6A's actual setting.

Lemma 9.2 (Roots of unity; the key to the DFT). *Fix an integer $N \geq 1$ and let $\omega = e^{-2\pi i/N}$. Then $\omega^N = 1$, and for any integer m :*

$$\sum_{j=0}^{N-1} \omega^{jm} = \begin{cases} N & \text{if } N \mid m, \\ 0 & \text{otherwise.} \end{cases}$$

Proof. $\omega^N = e^{-2\pi i} = 1$. If $N \mid m$, then $\omega^m = (\omega^N)^{m/N} = 1$ and every term of the sum is 1: total N . If $N \nmid m$, then $\omega^m \neq 1$ (since $\omega^m = e^{-2\pi i m/N} = 1$ exactly when m/N is an integer), so the finite geometric series gives

$$\sum_{j=0}^{N-1} (\omega^m)^j = \frac{(\omega^m)^N - 1}{\omega^m - 1} = \frac{(\omega^N)^m - 1}{\omega^m - 1} = \frac{1 - 1}{\omega^m - 1} = 0. \quad \square$$

Geometrically: the numbers ω^{jm} for $j = 0, \dots, N-1$ are equally spaced points on the unit circle (when $N \nmid m$), and equally spaced points on a circle sum to zero by symmetry. The lemma is that picture made algebra, and Lesson 12 will use it to prove that the Fourier basis is orthonormal.

9.2 Practice: by hand

Example 9.3 (Complex arithmetic warm-up). $(1+i)^2 = 1+2i+i^2 = 2i$. Also $|1+i| = \sqrt{2}$ and $1+i = \sqrt{2}e^{i\pi/4}$, so $(1+i)^2 = 2e^{i\pi/2} = 2i$: polar form turns powers into arithmetic on angles. Similarly $(1+i)^8 = 16e^{2\pi i} = 16$.

Example 9.4 (Fourth roots of unity). For $N = 4$: $\omega = e^{-2\pi i/4} = -i$, and the powers $\omega^0, \omega^1, \omega^2, \omega^3 = 1, -i, -1, i$: the four compass points of the unit circle. Check the lemma for $m = 1$: $1 - i - 1 + i = 0$. ✓ For $m = 4$: $1 + 1 + 1 + 1 = 4$. ✓

Example 9.5 (A complex inner product). $x = (1, i)$, $y = (i, 1)$ in $\mathbb{C}^2$: $\langle x, y \rangle = 1 \cdot \bar{i} + i \cdot \bar{1} = -i + i = 0$ — orthogonal. And $\|x\|^2 = |1|^2 + |i|^2 = 2$. Note $\langle y, x \rangle = \bar{0} = 0$ too, but in general the order matters up to conjugation.

EE261 connection. EE261's protagonist is $e^{2\pi i st}$. Fourier series write a periodic signal as $f(t) = \sum_k c_k e^{2\pi i kt}$, with coefficients $c_k = \int_0^1 f(t) \overline{e^{2\pi i kt}} dt$ — read that as $\langle f, e_k \rangle$ for the inner product $\langle f, g \rangle = \int_0^1 f \bar{g}$ on functions. It is literally Proposition 6.2's formula $a_j = \langle v, e_j \rangle$, with integrals replacing sums: Fourier coefficients are coordinates in an orthonormal basis, and partial Fourier sums are orthogonal projections (Theorem 6.3) onto the span of finitely many exponentials — hence the best least-squares approximants. Carrying that dictionary into EE261 makes the first three weeks feel familiar.

9.3 NumPy

```
import numpy as np
z = 1 + 1j                                # Python's imaginary unit is 1j
abs(z), np.angle(z)                       # (1.414..., 0.785... = pi/4)
np.conj(z)                                # (1-1j)

x = np.array([1, 1j]); y = np.array([1j, 1])
np.vdot(y, x)                             # complex inner product <x,y>:
                                           # vdot conjugates its FIRST argument
np.linalg.norm(x)                         # sqrt(2)

N = 4; w = np.exp(-2j * np.pi / N)
sum(w**(np.arange(N)*1))                  # ~0      (lemma, m = 1)
sum(w**(np.arange(N)*4))                  # 4        (lemma, m = 4)
```

Beware the conjugation convention: `np.vdot(a, b)` computes $\sum \overline{a_k} b_k$ (conjugates the *first* argument), while our $\langle x, y \rangle$ conjugates the second — so $\langle x, y \rangle$ is `np.vdot(y, x)`. Plain `x @ y` does no conjugation and is wrong for complex norms.

9.4 Exercises

Exercise 9.1 (Hand). Write $z = -1 + i\sqrt{3}$ in polar form $re^{i\theta}$, then compute z^3 two ways (polar and direct expansion) and confirm they agree.

Exercise 9.2 (Hand). List the sixth roots of unity $e^{-2\pi ij/6}$, $j = 0, \dots, 5$, in the form $a + bi$. Verify by hand that they sum to 0, and that the sum of their squares is also 0 (which case of Lemma 9.2 is that?).

Exercise 9.3 (Proof, ★). Prove the complex Pythagorean theorem and Cauchy–Schwarz: adapt the proofs of Theorem 2.3 and Theorem 2.5 to $\mathbb{C}^n$, flagging exactly where conjugate symmetry and conjugate-linearity in the second slot are used. (In the decomposition lemma, take $c = \langle x, y \rangle / \|y\|^2$ and verify $\langle z, y \rangle = 0$ still holds.)

Exercise 9.4 (Proof). Show that for all $x, y \in \mathbb{C}^n$: $\langle x, y \rangle + \langle y, x \rangle = 2\operatorname{Re}\langle x, y \rangle$, and deduce $\|x + y\|^2 = \|x\|^2 + 2\operatorname{Re}\langle x, y \rangle + \|y\|^2$.

Exercise 9.5 (NumPy). Plot (or just tabulate) the real and imaginary parts of the array `np.exp(2j*np.pi*3*np.linspace(0,1,100))` — a frequency-3 complex exponential over one period. Confirm numerically that it is orthogonal (in the `vdot` sense, scaled by 1/100) to the frequency-5 exponential sampled the same way.

Deeper reading. LADR 1A ($\mathbb{C}^n$), 4 (polynomials over $\mathbb{C}$: the fundamental theorem of algebra, used next lesson), 6A (complex inner products); Strang, ch. “Complex Vectors and Matrices.”

10 Eigenvalues, Eigenvectors, and Diagonalization

NEEDED FOR: EE261 (core); EE278 (Markov chains, PCA). Skimmable for CS221’s MDP lectures.

10.1 Theory

Throughout, $T : V \rightarrow V$ is a linear map from a space to itself (an *operator*); V is $\mathbb{R}^n$ or $\mathbb{C}^n$.

Definition 10.1 (Eigenvalue, eigenvector). A scalar $\lambda \in \mathbb{F}$ is an *eigenvalue* of T if there exists $v \neq 0$ with

$$Tv = \lambda v;$$

any such v is an *eigenvector* for λ . Equivalently (Proposition 5.2): λ is an eigenvalue iff $T - \lambda I$ is not injective; the eigenvectors for λ , together with 0, form the subspace $\operatorname{null}(T - \lambda I)$, the *eigenspace*.

An eigenvector is a direction the operator does not turn — only stretches by the factor λ . On such directions, applying T repeatedly is trivial: $T^k v = \lambda^k v$.

Theorem 10.2 (Independence of eigenvectors, LADR 5.11). *Eigenvectors corresponding to distinct eigenvalues are linearly independent.*

Proof. Let $v_1, \dots, v_m$ be eigenvectors for distinct eigenvalues $\lambda_1, \dots, \lambda_m$, and suppose for contradiction the list is dependent. Since each $v_i \neq 0$, the linear dependence lemma (Lemma 3.3) gives

a smallest index k with $v_k \in \text{span}(v_1, \dots, v_{k-1})$, and $v_1, \dots, v_{k-1}$ is independent by minimality of k . Write

$$v_k = a_1 v_1 + \dots + a_{k-1} v_{k-1}.$$

Apply T to both sides, and separately multiply both sides by λ_k :

$$\lambda_k v_k = a_1 \lambda_1 v_1 + \dots + a_{k-1} \lambda_{k-1} v_{k-1}, \quad \lambda_k v_k = a_1 \lambda_k v_1 + \dots + a_{k-1} \lambda_k v_{k-1}.$$

Subtracting, $0 = \sum_{i < k} a_i (\lambda_i - \lambda_k) v_i$. Independence of $v_1, \dots, v_{k-1}$ forces $a_i (\lambda_i - \lambda_k) = 0$ for each i ; since the eigenvalues are distinct, every $a_i = 0$, making $v_k = 0$ — contradicting that v_k is an eigenvector. $\square$

Corollary 10.3. *An operator on an n -dimensional space has at most n distinct eigenvalues.*

Proof. Eigenvectors for distinct eigenvalues form an independent list, and independent lists in an n -dimensional space have length at most n (Theorem 3.4). $\square$

Do eigenvalues exist at all? Over $\mathbb{R}$, not always: rotation of $\mathbb{R}^2$ by 90° turns every direction, so it has no real eigenvalue. Over $\mathbb{C}$, always — and Axler’s proof is a gem, using no determinants:

Theorem 10.4 (Existence of eigenvalues over $\mathbb{C}$, LADR 5.19). *Every operator on a nonzero finite-dimensional complex vector space has an eigenvalue.*

Proof. Let T be an operator on $\mathbb{C}^n$, $n \geq 1$, and pick any $v \neq 0$. The $n + 1$ vectors

$$v, Tv, T^2v, \dots, T^n v$$

must be linearly dependent (Theorem 3.4 again: $n + 1$ vectors in an n -dimensional space). So there are scalars $a_0, \dots, a_n$, not all zero, with $a_0 v + a_1 Tv + \dots + a_n T^n v = 0$. Let m be the largest index with $a_m \neq 0$; then $m \geq 1$, because $m = 0$ would give $a_0 v = 0$ with $a_0 \neq 0$ and $v \neq 0$.

Now the polynomial $p(z) = a_0 + a_1 z + \dots + a_m z^m$ has degree $m \geq 1$, so by the fundamental theorem of algebra it factors completely over $\mathbb{C}$:

$$p(z) = a_m (z - \lambda_1)(z - \lambda_2) \cdots (z - \lambda_m)$$

for some $\lambda_1, \dots, \lambda_m \in \mathbb{C}$. Because powers of T commute with each other, the same factorization holds with T substituted for z :

$$0 = a_0 v + a_1 Tv + \dots + a_m T^m v = a_m (T - \lambda_1 I)(T - \lambda_2 I) \cdots (T - \lambda_m I) v.$$

The right side is a composition of operators applied to the nonzero vector v , producing 0. If every factor $T - \lambda_j I$ were injective, the composition would be injective and could not send $v \neq 0$ to 0. Hence some $T - \lambda_j I$ is not injective: λ_j is an eigenvalue. $\square$

Why this proof is worth knowing. It explains *where eigenvalues come from*: dependence among $v, Tv, T^2v, \dots$ produces a polynomial identity in T , and eigenvalues are the roots. The fundamental theorem of algebra — every complex polynomial factors into linear pieces — is the sole reason $\mathbb{C}$ guarantees eigenvalues while $\mathbb{R}$ does not (over $\mathbb{R}$, $z^2 + 1$ refuses to factor, and that irreducible quadratic *is* the 90° rotation’s obstruction).

Definition 10.5 (Diagonalizable). An operator T on V ($\dim V = n$) is *diagonalizable* if V has a basis $v_1, \dots, v_n$ of eigenvectors of T . In matrix terms: $A = PDP^{-1}$, where D is diagonal with the eigenvalues $\lambda_1, \dots, \lambda_n$ on its diagonal and P is the invertible matrix whose columns are the corresponding eigenvectors.

The matrix identity is just bookkeeping for “ A acts diagonally on the eigenbasis”: $AP = PD$ says column-by-column that $Av_k = \lambda_k v_k$ (compute both sides’ k th columns: AP has k th column Av_k , and PD has k th column $\lambda_k v_k$). By Theorem 10.2, an operator on an n -dimensional space with n *distinct* eigenvalues is automatically diagonalizable. Repeated eigenvalues may or may not ruin it: I has one eigenvalue with an n -dimensional eigenspace (diagonalizable — it is diagonal), while the shear $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has the single eigenvalue 1 with only a one-dimensional eigenspace (not diagonalizable: no basis of eigenvectors exists).

Proposition 10.6 (Powers via diagonalization). If $A = PDP^{-1}$ then $A^k = PD^kP^{-1}$, and D^k is diagonal with entries λ_j^k .

Proof. $A^2 = PDP^{-1}PDP^{-1} = PD^2P^{-1}$; induction gives the rest. Diagonal matrices multiply entrywise on the diagonal. $\square$

Understanding a diagonalizable operator’s long-run behavior thus reduces to scalar arithmetic: components along eigenvectors with $|\lambda| < 1$ decay, $|\lambda| > 1$ grow, $|\lambda| = 1$ persist.

10.2 Practice: by hand

Example 10.7 (Full eigen-computation, 2×2). $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Find λ making $A - \lambda I$ singular.

A 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ is singular exactly when $ad - bc = 0$ (its columns are then parallel — Lesson 14 names this quantity the determinant). Here:

$$(2 - \lambda)^2 - 1 = 0 \iff 2 - \lambda = \pm 1 \iff \lambda = 1 \text{ or } 3.$$

Eigenvectors: for $\lambda = 3$, solve $(A - 3I)v = 0$: $\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} v = 0 \Rightarrow v = (1, 1)$. For $\lambda = 1$: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} v = 0 \Rightarrow v = (1, -1)$. Two distinct eigenvalues $\Rightarrow$ diagonalizable, and note $(1, 1) \perp (1, -1)$ — not an accident: A is symmetric, and Lesson 11 proves symmetric matrices always get orthogonal eigenvectors.

Geometrically, A stretches the diagonal direction $(1, 1)$ by 3 and the anti-diagonal $(1, -1)$ by 1: an anisotropic stretch along tilted axes.

Example 10.8 (Powers put to work). With A as above, compute $A^{10}(1, 0)$ without ten multiplications: decompose $(1, 0) = \frac{1}{2}(1, 1) + \frac{1}{2}(1, -1)$, then

$$A^{10}(1, 0) = \frac{1}{2} 3^{10}(1, 1) + \frac{1}{2} 1^{10}(1, -1) = \frac{1}{2}(3^{10} + 1, 3^{10} - 1) = (29525, 29524).$$

Eigen-coordinates turn matrix powers into scalar powers — the essential trick behind solving recurrences, Markov chains, and linear ODEs.

EE261 connection. A linear time-invariant (LTI) system — any circuit or filter EE261 studies — has the complex exponentials as its *eigenfunctions*: feed in $e^{2\pi i s t}$ and out comes $H(s)e^{2\pi i s t}$, the same signal scaled by a complex number. The scale factor $H(s)$ (the fre-

quency response / transfer function) is precisely an eigenvalue, indexed by frequency. “Diagonalize the system in the Fourier basis” is the slogan under all of EE261: convolution in time becomes multiplication (by eigenvalues) in frequency. Lesson 12 proves the finite-dimensional version of this exactly.

10.3 NumPy

```
import numpy as np
A = np.array([[2., 1.], [1., 2.]])
lams, P = np.linalg.eig(A)    # eigenvalues, eigenvector columns
lams                                # [3., 1.]
P                                # columns ~ (1,1)/sqrt2, (1,-1)/sqrt2

D = np.diag(lams)
np.allclose(A, P @ D @ np.linalg.inv(P))    # True: A = P D P^-1
np.allclose(np.linalg.matrix_power(A, 10),
             P @ np.diag(lams**10) @ np.linalg.inv(P))    # True

# For symmetric/Hermitian matrices prefer eigh: faster, more
# accurate, returns real eigenvalues sorted and orthonormal P.
lams_h, Q = np.linalg.eigh(A)
```

General `eig` on an $n \times n$ matrix costs $O(n^3)$ and may return complex output even for real input (rotation matrices!); `eigh` exploits symmetry for a guaranteed-real, orthonormal answer. Computing $A^k v$ by repeated multiplication is $O(kn^2)$; via eigendecomposition it is $O(n^3)$ once, then $O(n^2)$ for any k — the classic precompute-then-reuse trade.

10.4 Exercises

Exercise 10.1 (Hand). Find all eigenvalues and eigenvectors of (a) $\begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix}$, (b) $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (reflection through $x_2 = x_1$ — interpret the answer geometrically), (c) $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ over $\mathbb{R}$ and then over $\mathbb{C}$.

Exercise 10.2 (Hand). Diagonalize $A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ and use Proposition 10.6 to find a closed form for $A^k(1, 0)$.

Exercise 10.3 (Proof, ★). Prove that if T is invertible with eigenvalue λ (necessarily $\lambda \neq 0$ — why?), then λ^{-1} is an eigenvalue of T^{-1} , with the same eigenvectors. Also show: λ^k is an eigenvalue of T^k for every $k \geq 1$.

Exercise 10.4 (Proof). Show the shear $S = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ is not diagonalizable: find all eigenvalues, show the eigenspace is one-dimensional, and conclude no basis of eigenvectors exists.

Exercise 10.5 (NumPy). Let M be the “population matrix” $\begin{pmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{pmatrix}$ (columns sum to 1). Compute $M^k x_0$ for $x_0 = (1, 0)$ and $k = 1, 5, 50$ and observe convergence. Then compute the eigendecomposition and explain the limit via the eigenvalues (one should be 1; where does the limiting vector come from?).

Deeper reading. LADR 5A (invariant subspaces, eigenvalues), 5B (existence via minimal polynomials — a refinement of Theorem 10.4), 5D (diagonalizable operators); Strang, ch. “Eigenvalues and Eigenvectors.”

11 Self-Adjoint Operators and the Spectral Theorem

NEEDED FOR: EE261 (core); EE278 (covariance matrices, PCA — reused in Lesson 23)

11.1 Theory

Work in $\mathbb{F}^n$ ($\mathbb{F} = \mathbb{R}$ or $\mathbb{C}$) with the inner product of Lessons 2 and 9. Recall from Lesson 4 the defining property of the transpose; over $\mathbb{C}$ its correct analogue is the *conjugate transpose* (*adjoint*) $A^* = \overline{A}^T$, which satisfies $\langle Ax, y \rangle = \langle x, A^*y \rangle$ for all x, y (same expansion as before, tracking conjugates). Over $\mathbb{R}$, $A^* = A^T$.

Definition 11.1 (Self-adjoint). A is *self-adjoint* (*Hermitian*; over $\mathbb{R}$: *symmetric*) if $A^* = A$; equivalently $\langle Ax, y \rangle = \langle x, Ay \rangle$ for all x, y .

Proposition 11.2 (Real eigenvalues, LADR 7.12). *Every eigenvalue of a self-adjoint matrix is real.*

Proof. Let $Av = \lambda v$ with $v \neq 0$. Then

$$\lambda \|v\|^2 = \langle \lambda v, v \rangle = \langle Av, v \rangle = \langle v, Av \rangle = \langle v, \lambda v \rangle = \bar{\lambda} \|v\|^2.$$

Since $\|v\|^2 \neq 0$, $\lambda = \bar{\lambda}$: real. □

Proposition 11.3 (Orthogonal eigenspaces). *If A is self-adjoint, eigenvectors for distinct eigenvalues are orthogonal.*

Proof. Let $Av = \lambda v$, $Aw = \mu w$, $\lambda \neq \mu$ (both real by Proposition 11.2). Then

$$\lambda \langle v, w \rangle = \langle Av, w \rangle = \langle v, Aw \rangle = \bar{\mu} \langle v, w \rangle = \mu \langle v, w \rangle,$$

so $(\lambda - \mu) \langle v, w \rangle = 0$, forcing $\langle v, w \rangle = 0$. □

Theorem 11.4 (Spectral theorem, LADR 7.29/7.31). *Let A be a self-adjoint $n \times n$ matrix (real symmetric, or complex Hermitian). Then $\mathbb{F}^n$ has an orthonormal basis of eigenvectors of A , and all eigenvalues are real. Equivalently, $A = Q\Lambda Q^*$ with Q having orthonormal columns ($Q^*Q = I$) and Λ real diagonal.*

Proof. Induct on n . For $n = 1$ any unit vector works. For $n > 1$: A has an eigenvalue λ_1 . (Over $\mathbb{C}$ this is Theorem 10.4. Over $\mathbb{R}$: regard the real symmetric A as a complex matrix; it is Hermitian, so it has a complex eigenvalue λ_1 , real by Proposition 11.2; then $A - \lambda_1 I$ is a real matrix that is singular over $\mathbb{C}$, hence has rank $< n$, hence is singular over $\mathbb{R}$ — rank is the same whether we allow real or complex coefficients, since elimination never leaves the field the entries live in — so a real eigenvector exists.)

Pick a unit eigenvector e_1 for λ_1 , and let $U = \{x : \langle x, e_1 \rangle = 0\}$ be its orthogonal complement, of dimension $n - 1$ (Exercise 6.3). The key point is that A maps U into U : for $u \in U$,

$$\langle Au, e_1 \rangle = \langle u, Ae_1 \rangle = \langle u, \lambda_1 e_1 \rangle = \lambda_1 \langle u, e_1 \rangle = 0,$$

so $Au \in U$. The restriction of A to U is an operator on an $(n - 1)$ -dimensional inner product space, still self-adjoint (the identity $\langle Ax, y \rangle = \langle x, Ay \rangle$ holds in particular for $x, y \in U$). By

the induction hypothesis, U has an orthonormal basis $e_2, \dots, e_n$ of eigenvectors of A . Then $e_1, e_2, \dots, e_n$ is orthonormal (each later e_j lies in $U \perp e_1$) and consists of eigenvectors.

For the matrix form: let Q have columns $e_1, \dots, e_n$. Orthonormality of columns says exactly $Q^*Q = I$, and $Ae_k = \lambda_k e_k$ says $AQ = Q\Lambda$; since Q is square with $Q^*Q = I$, $Q^{-1} = Q^*$ and $A = Q\Lambda Q^*$. $\square$

What the spectral theorem buys you. Self-adjoint matrices are exactly the ones that are a *pure stretch along some perpendicular axes*: rotate into the right orthonormal frame (Q^*), scale each axis by a real number (Λ), rotate back (Q). No shearing, no hidden rotation. Because the eigenbasis is orthonormal, coordinates in it are free ($a_j = \langle x, e_j \rangle$, Proposition 6.2) — so analyzing A 's action on any vector costs n dot products. This is the finite-dimensional model of what Fourier analysis does for differential operators.

A useful companion definition: a self-adjoint A is *positive semidefinite* (PSD) if $\langle Ax, x \rangle \geq 0$ for all x . By the spectral theorem this holds iff all eigenvalues are ≥ 0 (expand x in the eigenbasis: $\langle Ax, x \rangle = \sum_j \lambda_j |\langle x, e_j \rangle|^2$). The matrices A^*A from least squares (Lesson 6) are always PSD: $\langle A^*Ax, x \rangle = \|Ax\|^2 \geq 0$ — this observation powers the SVD in Lesson 13 (and, in Lesson 23, the fact that covariance matrices are PSD).

11.2 Practice: by hand

Example 11.5 (Spectral decomposition, fully worked). From Example 10.7, $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ has eigenpairs $\lambda = 3$, $v = (1, 1)$ and $\lambda = 1$, $v = (1, -1)$, already orthogonal as Proposition 11.3 promised. Normalize:

$$Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \Lambda = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}, \quad A = Q\Lambda Q^T.$$

Verify one entry: $(Q\Lambda Q^T)_{11} = \frac{1}{2}(3 \cdot 1 \cdot 1 + 1 \cdot 1 \cdot 1) = 2$. $\checkmark$ Also read off positivity: eigenvalues $3, 1 > 0$, so $\langle Ax, x \rangle > 0$ for all $x \neq 0$ — A is *positive definite*.

Example 11.6 (Quadratic forms become sums of squares). For the same A , with new coordinates $y = Q^T x$ (the eigen-coordinates),

$$x^T A x = x^T Q \Lambda Q^T x = y^T \Lambda y = 3y_1^2 + y_2^2.$$

The level sets $x^T A x = 1$ are ellipses with axes along $(1, 1)$ and $(1, -1)$ and semi-axis lengths $1/\sqrt{3}$ and 1. Diagonalization untangles every quadratic form into independent squared terms.

EE261 connection. Symmetric structure is why real signals have the Fourier symmetries EE261 drills ($\hat{f}(-s) = \overline{\hat{f}(s)}$, etc.), and self-adjointness of the underlying operators is what makes Fourier eigen-expansions come with orthogonality — which in turn yields Parseval-Rayleigh identities: energy in time equals energy in frequency, i.e., $\|x\|$ is preserved when you switch to an orthonormal (eigen)basis. The next lesson makes the norm-preservation statement exact for the DFT.

11.3 NumPy

```
import numpy as np
A = np.array([[2., 1.], [1., 2.]])
lam, Q = np.linalg.eigh(A)          # eigh: for self-adjoint input
lam                                     # [1., 3.] (ascending, real)
np.allclose(Q.T @ Q, np.eye(2))    # True: orthonormal columns
np.allclose(A, Q @ np.diag(lam) @ Q.T) # True: A = Q Lam Q^T

# PSD check for a Gram matrix:
B = np.random.randn(5, 3)
G = B.T @ B
np.all(np.linalg.eigvalsh(G) >= -1e-12) # True (up to roundoff)
```

`eigh` costs $O(n^3)$ like `eig`, but with about half the work and a guarantee the output is real and orthonormal; always prefer it when you know the matrix is self-adjoint. Testing PSD via `eigvalsh` beats testing $\langle Ax, x \rangle \geq 0$ on random vectors — the eigenvalues certify all directions at once.

11.4 Exercises

Exercise 11.1 (Hand). Find the spectral decomposition $Q\Lambda Q^T$ of $\begin{pmatrix} 0 & 2 \\ 2 & 3 \end{pmatrix}$, and write the quadratic form $q(x) = 4x_1x_2 + 3x_2^2$ as a combination of squares in the eigen-coordinates. Is the matrix PSD?

Exercise 11.2 (Hand). The Hermitian matrix $H = \begin{pmatrix} 1 & i \\ -i & 1 \end{pmatrix}$ (note $H^* = H$): find its (real!) eigenvalues and a pair of orthogonal eigenvectors in $\mathbb{C}^2$; verify orthogonality with the complex inner product.

Exercise 11.3 (Proof, ★). Prove: a self-adjoint matrix with all eigenvalues in $\{0, 1\}$ is exactly an orthogonal projection P_U (as in Theorem 6.3) onto the span U of its $\lambda = 1$ eigenvectors. (Use the spectral theorem to write $A = \sum_j \lambda_j e_j e_j^*$, i.e. $Ax = \sum_j \lambda_j \langle x, e_j \rangle e_j$, and compare with the projection formula.)

Exercise 11.4 (Proof). Show that for any (not necessarily self-adjoint) real A , the matrix $S = \frac{1}{2}(A + A^T)$ is symmetric and $x^T Ax = x^T Sx$ for all x — so quadratic forms lose nothing by assuming symmetry (as Proposition 7.3 did).

Exercise 11.5 (NumPy). Build a random symmetric 6×6 matrix as $S = R + R.T$. Confirm numerically: eigenvalues real, eigenvectors orthonormal, and reconstruction $Q\Lambda Q^T$ exact to 10^{-10} . Then confirm Proposition 11.3 fails for a random *non*-symmetric matrix (compute `eig` and test $Q^T Q \approx I$).

Deeper reading. LADR 7A (self-adjoint and normal operators), 7B (spectral theorem — Axler also proves the normal case over $\mathbb{C}$, the full characterization of orthonormally diagonalizable operators), 7C (positive operators); Strang, ch. “Symmetric Matrices and Positive Definiteness.”

12 Unitary Matrices and the Discrete Fourier Transform

NEEDED FOR: EE261 (core); EE278 background (transform-domain reasoning)

12.1 Theory

Definition 12.1 (Unitary / orthogonal matrix). A square matrix Q over $\mathbb{C}$ is *unitary* if $Q^*Q = I$ (over $\mathbb{R}$, with $Q^T Q = I$: *orthogonal*).

Theorem 12.2 (Three faces of unitarity, LADR 7.51). *For a square matrix Q , the following are equivalent:*

- (a) $Q^*Q = I$ (columns are orthonormal);
- (b) $\langle Qx, Qy \rangle = \langle x, y \rangle$ for all x, y (inner products preserved);
- (c) $\|Qx\| = \|x\|$ for all x (lengths preserved).

Moreover a unitary Q is invertible with $Q^{-1} = Q^*$, which is again unitary.

Proof. (a) $\Rightarrow$ (b): $\langle Qx, Qy \rangle = \langle x, Q^*Qy \rangle = \langle x, y \rangle$, using the adjoint's defining property. (b) $\Rightarrow$ (c): take $y = x$ and square roots. (c) $\Rightarrow$ (a): first note entry (j, k) of Q^*Q is $\langle Qe_k, Qe_j \rangle$. From (c) and the expansion $\|Q(x+y)\|^2 = \|Qx\|^2 + 2\operatorname{Re}\langle Qx, Qy \rangle + \|Qy\|^2$ (Exercise 9.4), preserving all norms forces $\operatorname{Re}\langle Qx, Qy \rangle = \operatorname{Re}\langle x, y \rangle$ for all x, y ; replacing x by ix recovers the imaginary parts likewise (over $\mathbb{R}$, the real-part identity is already everything). So (b) holds, and applying it to standard basis vectors gives $\langle Qe_k, Qe_j \rangle = \langle e_k, e_j \rangle = \delta_{jk}$: the columns are orthonormal, which is (a).

Finally, $Q^*Q = I$ with Q square makes Q injective (if $Qx = 0$ then $x = Q^*Qx = 0$), hence invertible by Corollary 5.4, and multiplying $Q^*Q = I$ by Q^{-1} on the right gives $Q^* = Q^{-1}$; then $(Q^*)^*Q^* = QQ^{-1} = I$ shows Q^* is unitary. $\square$

Unitary matrices are the rigid motions of $\mathbb{C}^n$ fixing the origin: they preserve all lengths and angles. A change of basis between two orthonormal bases is always unitary, and conversely.

Definition 12.3 (The DFT matrix). Fix $N \geq 1$ and $\omega = e^{-2\pi i/N}$. The *discrete Fourier transform* of $x \in \mathbb{C}^N$ (indexing coordinates from 0) is the vector $\hat{x} = Wx$, where W is the $N \times N$ matrix with entries

$$W_{k,n} = \omega^{kn}, \quad \text{i.e.} \quad \hat{x}_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i kn/N}.$$

The number $\hat{x}_k$ measures the correlation of the signal x with the frequency- k oscillation.

Theorem 12.4 (The DFT is (essentially) unitary). $W^*W = NI$. Hence $F = W/\sqrt{N}$ is unitary, the DFT is invertible with

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} \hat{x}_k e^{+2\pi i kn/N} \quad (\text{the inverse DFT}),$$

and Parseval's identity holds: $\sum_n |x_n|^2 = \frac{1}{N} \sum_k |\hat{x}_k|^2$.

Proof. Entry (m, n) of W^*W is $\sum_k \overline{W_{k,m}} W_{k,n} = \sum_{k=0}^{N-1} \overline{\omega^{km}} \omega^{kn} = \sum_{k=0}^{N-1} \omega^{k(n-m)}$, which by the roots-of-unity lemma (Lemma 9.2) equals N if $m = n$ and 0 otherwise — here $|n - m| < N$, so $N \mid (n - m)$ occurs only for $m = n$. Thus $W^*W = NI$.

Consequently $F^*F = I$: F is unitary, so $W^{-1} = \frac{1}{N}W^*$, whose entry (n, k) is $\frac{1}{N}\overline{\omega^{kn}} = \frac{1}{N}e^{+2\pi i kn/N}$ — the displayed inversion formula. Parseval is Theorem 12.2(c) for F : $\|Fx\| = \|x\|$, i.e. $\frac{1}{N}\|Wx\|^2 = \|x\|^2$. $\square$

Definition 12.5 (Circular convolution). For $c, x \in \mathbb{C}^N$, with indices taken mod N ,

$$(c * x)_n = \sum_{j=0}^{N-1} c_j x_{n-j \bmod N}.$$

Equivalently $c * x = Cx$ where C is the *circulant* matrix whose column j is c cyclically shifted down by j .

Theorem 12.6 (Convolution theorem). $\widehat{c * x}_k = \hat{c}_k \hat{x}_k$ for every k . Equivalently: every circulant matrix is diagonalized by the DFT, $C = W^{-1} \text{diag}(\hat{c}) W$, with eigenvalues $\hat{c}_0, \dots, \hat{c}_{N-1}$.

Proof. Compute, substituting $m = n - j \bmod N$ (as n runs over residues mod N with j fixed, so does m , and $\omega^{kn} = \omega^{k(j+m)}$ since $\omega^{kN} = 1$ makes exponents matter only mod N):

$$\widehat{c * x}_k = \sum_n \sum_j c_j x_{n-j} \omega^{kn} = \sum_j c_j \omega^{kj} \sum_m x_m \omega^{km} = \hat{c}_k \hat{x}_k.$$

For the matrix form: the identity says $W(Cx) = \text{diag}(\hat{c})(Wx)$ for all x , i.e. $WC = \text{diag}(\hat{c})W$, i.e. $C = W^{-1} \text{diag}(\hat{c})W$. Reading it as a diagonalization: the columns of W^{-1} (the sampled complex exponentials) are eigenvectors of *every* circulant, with eigenvalues the DFT of the first column. $\square$

The punchline of Part II. Shift-invariant linear operations (circulants) are precisely the ones diagonalized by the Fourier basis, and their eigenvalues are the transform of their impulse response c . Filtering = multiply in frequency. This is Lesson 10's “diagonalization turns matrix powers into scalars” plus Lesson 11's “orthonormal eigenbases make coordinates free,” specialized to the one basis that never changes: the complex exponentials. EE261 spends a quarter developing the continuous version.

12.2 Practice: by hand

Example 12.7 (The 4-point DFT, fully). $N = 4$, $\omega = -i$ (Lesson 9). The matrix and a transform:

$$W = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{pmatrix}, \quad x = (1, 0, 1, 0) \Rightarrow \hat{x} = Wx = (2, 0, 2, 0).$$

Sense check: x alternates with period 2 — energy at frequency 0 (its mean is not zero) and frequency 2 (the Nyquist alternation), none at frequencies 1, 3. Parseval: $\sum |x_n|^2 = 2$ and $\frac{1}{4} \sum |\hat{x}_k|^2 = \frac{1}{4}(4 + 4) = 2$. $\checkmark$

Example 12.8 (Convolution theorem, by hand). Take the averaging filter $c = \frac{1}{2}(1, 1, 0, 0)$ and $x = (1, 0, 1, 0)$ with $N = 4$. Directly: $(c * x)_n = \frac{1}{2}(x_n + x_{n-1}) = \frac{1}{2}(1, 1, 1, 1)$. Via frequency: $\hat{c} = \frac{1}{2}(2, 1-i, 0, 1+i)$, and $\hat{c} \odot \hat{x} = (2, 0, 0, 0)$ (entrywise), whose inverse DFT is $\frac{1}{4} \cdot 2 \cdot (1, 1, 1, 1) = \frac{1}{2}(1, 1, 1, 1)$. ✓ The filter's zero at frequency 2 ($\hat{c}_2 = 0$) is why the alternating component of x is annihilated: averaging adjacent samples kills the fastest oscillation.

EE261 connection. This lesson is the last third of EE261 in miniature: the DFT section of the course defines exactly this matrix, proves exactly this inversion and convolution theorem, and then studies the FFT — an algorithm computing Wx in $O(N \log N)$ instead of the obvious $O(N^2)$, arguably the most important algorithm in engineering. Everything earlier in the course (Fourier series, the continuous transform, sampling) has the same skeleton with integrals in place of sums; if you own this lesson, EE261's DFT weeks are review.

12.3 NumPy

```
import numpy as np
N = 4; x = np.array([1., 0., 1., 0.])

xh = np.fft.fft(x)                # [2.+0j, 0.+0j, 2.+0j, 0.+0j]
np.allclose(np.fft.ifft(xh), x)    # True: inversion formula

# Build W explicitly and compare:
n = np.arange(N)
W = np.exp(-2j * np.pi * np.outer(n, n) / N)
np.allclose(W @ x, xh)            # True
np.allclose(W.conj().T @ W, N * np.eye(N)) # True: W* W = N I

# Convolution theorem:
c = np.array([.5, .5, 0., 0.])
lhs = np.fft.fft(np.array([(c * np.roll(x[:-1], k+1)).sum()
                           for k in range(N)])) # circular conv, direct
rhs = np.fft.fft(c) * np.fft.fft(x)
np.allclose(lhs, rhs)             # True
```

`np.fft.fft` uses the same sign/normalization convention as this lesson (no $1/\sqrt{N}$; the $1/N$ lives in `ifft`). The FFT's $O(N \log N)$ versus the matrix product's $O(N^2)$ is the difference between milliseconds and hours at $N = 10^6$ — and it is why convolving long signals is done by transforming, multiplying, and transforming back.

12.4 Exercises

Exercise 12.1 (Hand). Write out the 3-point DFT matrix using $\omega = e^{-2\pi i/3} = -\frac{1}{2} - \frac{\sqrt{3}}{2}i$, and compute $\hat{x}$ for $x = (1, 1, 1)$ and for $x = (1, \omega^{-1}, \omega^{-2})$. Explain both answers in one sentence each.

Exercise 12.2 (Hand). For $N = 4$, write the circulant matrix C for $c = (0, 1, 0, 0)$ (the cyclic shift). Verify directly that $v = (1, i^{-1}, i^{-2}, i^{-3}) = (1, -i, -1, i)$ satisfies $Cv = \lambda v$, and identify λ as an entry of $\hat{c}$.

Exercise 12.3 (Proof, ★). Prove that the product of two unitary matrices is unitary and that every eigenvalue λ of a unitary matrix has $|\lambda| = 1$. (For the latter: apply Theorem 12.2(c) to an eigenvector.) Conclude the DFT eigenvalue fact: $|\hat{c}_k| \leq \sum_j |c_j|$ for a circulant's eigenvalues, directly from the definition of $\hat{c}$ and the triangle inequality.

Exercise 12.4 (Proof). Show that the set of circulant $N \times N$ matrices is closed under products and that any two circulants commute. (Two lines with Theorem 12.6: simultaneous diagonalization.)

Exercise 12.5 (NumPy). Generate a signal of length $N = 256$: a sum of two sinusoids at frequencies 5 and 40 plus noise. Plot $|\hat{x}_k|$ and locate the peaks. Then zero out all DFT coefficients with k outside $\{0, \dots, 10\} \cup \{246, \dots, 255\}$, inverse-transform, and compare with the original: you have built a low-pass filter. Why must you keep the mirror band near N to get a real-valued output? (Hint: for real x , $\hat{x}_{N-k} = \overline{\hat{x}_k}$ — prove it from the definition.)

Deeper reading. LADR 7D (isometries and unitary operators); Strang, ch. “Complex Vectors and Matrices” (the Fast Fourier Transform); EE261 course reader (Osgood), chapter on the DFT.

13 The Singular Value Decomposition

NEEDED FOR: EE261-adjacent; EE278 (PCA). Not needed for CS221.

The spectral theorem needs a self-adjoint square matrix. The SVD is its extension to *every* matrix, square or not — LADR Chapter 7's finale, and the workhorse behind `lstsq`, `matrix_rank`, PCA, and compression.

13.1 Theory

Theorem 13.1 (Singular value decomposition, LADR 7.70). *Let A be a real $m \times n$ matrix of rank r . Then there exist orthonormal vectors $v_1, \dots, v_r \in \mathbb{R}^n$, orthonormal vectors $u_1, \dots, u_r \in \mathbb{R}^m$, and numbers $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ (the singular values) such that*

$$A = \sigma_1 u_1 v_1^\top + \dots + \sigma_r u_r v_r^\top, \quad \text{equivalently} \quad Av_i = \sigma_i u_i \ (i \leq r), \quad Av = 0 \text{ for } v \perp v_1, \dots, v_r.$$

(The same holds over $\mathbb{C}$ with $*$ in place of $\top$.)

Proof. The matrix $A^\top A$ is symmetric and PSD (Lesson 11: $\langle A^\top A x, x \rangle = \|Ax\|^2 \geq 0$). By the spectral theorem it has an orthonormal eigenbasis $v_1, \dots, v_n$ of $\mathbb{R}^n$ with real eigenvalues $\lambda_1 \geq \dots \geq \lambda_n \geq 0$; write $\lambda_i = \sigma_i^2$ with $\sigma_i \geq 0$, and let r' be the number of nonzero σ_i .

Key computation. For any i, j :

$$\langle Av_i, Av_j \rangle = \langle A^\top A v_i, v_j \rangle = \sigma_i^2 \langle v_i, v_j \rangle = \begin{cases} \sigma_i^2 & i = j \\ 0 & i \neq j. \end{cases}$$

So the vectors $Av_1, \dots, Av_n$ are mutually orthogonal with $\|Av_i\| = \sigma_i$; in particular $Av_i = 0$ exactly when $\sigma_i = 0$. For $i \leq r'$ define $u_i = Av_i / \sigma_i$: unit vectors, orthonormal by the computation above.

The formula. Both sides of the claimed identity are linear maps; by Theorem 4.3 it suffices to check them on the basis $v_1, \dots, v_n$. For $j \leq r'$: the right side gives $\sum_i \sigma_i u_i (v_i^\top v_j) = \sigma_j u_j = Av_j$. For $j > r'$: the right side gives $0 = Av_j$. They agree.

Rank. range A is spanned by the Av_j , i.e. by $\sigma_1 u_1, \dots, \sigma_{r'} u_{r'}$, an orthogonal (hence independent) list — so $\text{rank } A = r'$, forcing $r' = r$. $\square$

In matrix form, padding the v 's and u 's to full orthonormal bases: $A = U\Sigma V^T$ with U ($m \times m$) and V ($n \times n$) orthogonal and Σ ($m \times n$) diagonal with the σ_i 's. Read as a pipeline, every matrix is: rotate the input frame (V^T), stretch each axis by σ_i and embed into the output space (Σ), rotate the output frame (U). Rotation–stretch–rotation, no exceptions.

Corollary 13.2 (What the SVD reveals at a glance). *With notation as above:*

- (a) $\text{rank } A = \text{number of nonzero singular values (this is what `matrix_rank` counts, with a tolerance)}$;
- (b) $\|Av\| \leq \sigma_1 \|v\|$ for all v , with equality at $v = v_1$: σ_1 is the operator's maximum stretch factor;
- (c) the null space of A is $\text{span}(v_{r+1}, \dots, v_n)$ and the range is $\text{span}(u_1, \dots, u_r)$ — the SVD hands you orthonormal bases for both of Lesson 5's subspaces.

Proof. (a) was shown in the proof. (c): $Av = 0$ iff all eigen-coordinates of v along $v_1, \dots, v_r$ vanish (by the orthogonality of the Av_i and $\|Av_i\| = \sigma_i > 0$); the range statement was the last line of the proof. (b): expand $v = \sum_i c_i v_i$; then $Av = \sum_{i \leq r} c_i \sigma_i u_i$, and by orthonormality of the u_i (Pythagoras, repeatedly):

$$\|Av\|^2 = \sum_{i \leq r} \sigma_i^2 c_i^2 \leq \sigma_1^2 \sum_i c_i^2 = \sigma_1^2 \|v\|^2,$$

with equality when all weight sits on c_1 . $\square$

Dropping all but the k largest terms of $\sum_i \sigma_i u_i v_i^T$ gives the rank- k matrix A_k closest to A (Eckart–Young theorem; statement worth knowing, proof omitted) — the principle behind PCA and lossy compression: keep the directions with the biggest stretch, discard the rest.

13.2 Practice: by hand

Example 13.3 (A full 2×2 SVD). $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ (the shear that refused to diagonalize in Lesson 10 — but nothing refuses an SVD). Compute

$$A^T A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} : (1 - \lambda)(2 - \lambda) - 1 = 0 \iff \lambda^2 - 3\lambda + 1 = 0 \iff \lambda = \frac{3 \pm \sqrt{5}}{2}.$$

So $\sigma_1 = \sqrt{(3 + \sqrt{5})/2} \approx 1.618$ and $\sigma_2 = \sqrt{(3 - \sqrt{5})/2} \approx 0.618$ (the golden ratio and its reciprocal!). Eigenvector for λ_1 : solve $(1 - \lambda_1)a + b = 0$, giving $v_1 \propto (1, \lambda_1 - 1)$; then $u_1 = Av_1/\sigma_1$. The shear stretches one tilted direction by φ and compresses an orthogonal one by $1/\varphi$ — invisible to eigen-analysis, plain in the SVD.

EE261 connection. SVD is EE261-adjacent rather than EE261-core: it appears when Fourier methods meet data — e.g., analyzing which frequency patterns dominate a family of signals (PCA of spectrograms), or characterizing the maximum gain σ_1 of a non-shift-

invariant system where the convolution theorem does not apply. It also closes a Part I loop: `lstsq` (Lesson 6) is implemented via SVD because Corollary 13.2 exposes range and null space in orthonormal coordinates, making the projection of b trivial even when columns are dependent. (In EE278 it returns as PCA: Lesson 23 shows the covariance matrix's eigendecomposition *is* the SVD story for data.)

13.3 NumPy

```
import numpy as np
A = np.array([[1., 1.], [0., 1.]])
U, s, Vt = np.linalg.svd(A)
s                                     # [1.618..., 0.618...]
np.allclose(A, U @ np.diag(s) @ Vt)  # True
np.allclose(s[0], np.linalg.norm(A, 2)) # True: top stretch

# Low-rank approximation of an image-like matrix:
M = np.outer(np.arange(50), np.ones(50)) + np.random.randn(50,50)
U, s, Vt = np.linalg.svd(M)
M5 = (U[:, :5] * s[:5]) @ Vt[:5, :]      # keep top 5 components
np.linalg.norm(M - M5) / np.linalg.norm(M) # small relative error
```

Full SVD of $m \times n$ costs $O(mn \min(m, n))$ — the most expensive standard factorization, and the most informative: rank, norm, null space, range, and conditioning all fall out of one call.

13.4 Exercises

Exercise 13.1 (Hand). Compute the SVD ingredients of $A = \begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix}$: singular values, v_i 's, u_i 's. (Careful: singular values are positive; where did the sign of -2 go?) Compare with the eigenvalues from Exercise 10.1(a).

Exercise 13.2 (Hand). For $A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ (rank 1), find σ_1, u_1, v_1 and write $A = \sigma_1 u_1 v_1^T$ exactly.

Exercise 13.3 (Proof, ★). Prove that for symmetric PSD A , the singular values equal the eigenvalues and one may take $u_i = v_i$. Then show by example (Exercise 13.1!) that for symmetric A with a negative eigenvalue, singular values are the absolute values of eigenvalues.

Exercise 13.4 (Proof). Using Corollary 13.2(b) applied to A and to A^{-1} (square invertible case), show $\|Ax\|/\|x\|$ ranges over $[\sigma_n, \sigma_1]$, and interpret $\kappa = \sigma_1/\sigma_n$ as the worst-case ratio by which A distorts relative lengths. (κ is the *condition number*; `np.linalg.cond` computes it, and it governs how much rounding error `solve` can amplify.)

Exercise 13.5 (NumPy). Load any grayscale image as a matrix (or synthesize one with structure), compute its SVD, and plot reconstruction error versus k for rank- k truncations $k = 1, 5, 20, 50$. Report how many components you need for the image to “look fine” and what compression ratio that represents.

Deeper reading. LADR 7E–7F (SVD and its consequences — Axler's proof is the one given here), 7C (positive operators and square roots); Strang, ch. “The Singular Value Decomposition.”

14 Determinants and Trace

NEEDED FOR: EE261 (stretch theorem); general background. Not needed for CS221.

Axler famously postpones determinants to his final chapter because nothing earlier needed them — and indeed nothing earlier *here* needed them. But they earn their place: as the volume-scaling factor of a linear map (which is exactly how they enter multivariable integrals and multidimensional Fourier transforms) and as a compact certificate of invertibility.

14.1 Theory

Definition 14.1 (Determinant, characterized). The determinant is the unique function $\det : (n \times n \text{ matrices}) \rightarrow \mathbb{F}$ that, viewed as a function of the n columns, is

- (i) *multilinear*: linear in each column separately, the others held fixed;
- (ii) *alternating*: $\det = 0$ whenever two columns are equal;
- (iii) *normalized*: $\det I = 1$.

Existence and uniqueness are LADR 9B–9C; uniqueness forces the explicit permutation formula

$$\det A = \sum_{\pi} \text{sign}(\pi) A_{\pi(1),1} \cdots A_{\pi(n),n},$$

summed over the $n!$ permutations π of $\{1, \dots, n\}$. For $n = 2, 3$ this yields the familiar

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc, \quad \det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bfg + cdh - ceg - bdi - afh.$$

Two consequences of (i)–(ii), each one line: swapping two columns flips the sign of $\det$ (expand $\det$ with both columns replaced by their sum, and cancel using (ii)); and adding a multiple of one column to another leaves $\det$ unchanged (multilinearity splits it into $\det A$ plus a determinant with a repeated column). The same statements hold for rows, since $\det A = \det A^T$ (visible from the permutation formula: transposing reindexes the sum by inverse permutations, which have the same signs).

Theorem 14.2 (Multiplicativity). $\det(AB) = \det(A) \det(B)$.

Proof. Fix A and consider $f(B) = \det(AB)$ as a function of the columns of B . Column j of AB is Ab_j (Lesson 4), so f is multilinear in the columns of B (because $b_j \mapsto Ab_j$ is linear and $\det$ is multilinear) and alternating (equal columns $b_i = b_j$ give equal columns $Ab_i = Ab_j$). A short argument from uniqueness — any multilinear alternating function is a scalar multiple of $\det$, the scalar being its value at I — gives $f(B) = f(I) \det(B) = \det(A) \det(B)$. $\square$

Theorem 14.3 (Determinant as invertibility certificate and eigenvalue product). (a) A is invertible $\iff \det A \neq 0$; in that case $\det(A^{-1}) = 1/\det A$.

(b) If A (over $\mathbb{C}$) has eigenvalues $\lambda_1, \dots, \lambda_n$ listed with multiplicity, then $\det A = \lambda_1 \cdots \lambda_n$; consequently λ is an eigenvalue of A iff $\det(A - \lambda I) = 0$ — the characteristic polynomial route used informally in Example 10.7.

Proof. (a) If A is invertible, $1 = \det I = \det(AA^{-1}) = \det A \det A^{-1}$ by Theorem 14.2, so $\det A \neq 0$ and the reciprocal formula holds. If A is not invertible, its columns are dependent (Lesson 5), so some column is a combination of the others (dependence lemma); subtracting that combination from it — which does not change $\det$ — produces a zero column, and multilinearity with the zero column gives $\det A = 0$.

(b) Over $\mathbb{C}$, every matrix is similar to an upper-triangular one: $A = STS^{-1}$ with T upper triangular whose diagonal lists the eigenvalues with multiplicity (LADR 5C, from Theorem 10.4 by induction; statement used here without the inductive details). By multiplicativity, $\det A = \det S \det T \det S^{-1} = \det T$, and the determinant of a triangular matrix is the product of its diagonal entries — expand by the permutation formula: only the identity permutation avoids all the zero entries below the diagonal. Hence $\det A = \lambda_1 \cdots \lambda_n$. Applying this to $A - \lambda I$, whose eigenvalues are $\lambda_i - \lambda$: it is singular iff some $\lambda_i = \lambda$ iff its determinant $\prod_i (\lambda_i - \lambda)$ vanishes. $\square$

Volume. $|\det A|$ is the factor by which the map $x \mapsto Ax$ scales n -dimensional volume: the unit square (cube) maps to the parallelogram (parallelepiped) spanned by the columns of A , whose volume is $|\det A|$. Properties (i)–(iii) are exactly the axioms signed volume must satisfy (scale one edge, the volume scales; degenerate flat shapes have volume zero; the unit cube has volume 1), which is the honest reason the determinant is defined this way. The sign records orientation: negative when the map flips handedness.

Definition 14.4 (Trace). $\operatorname{tr} A = \sum_i A_{i,i}$, the sum of diagonal entries.

Proposition 14.5. (a) $\operatorname{tr}(AB) = \operatorname{tr}(BA)$ for any A ($m \times n$) and B ($n \times m$). (b) Over $\mathbb{C}$, $\operatorname{tr} A = \lambda_1 + \cdots + \lambda_n$ (eigenvalues with multiplicity).

Proof. (a) Both sides equal $\sum_i \sum_j A_{i,j} B_{j,i}$ — expand each definition and swap the order of summation. (b) With $A = STS^{-1}$ as above: $\operatorname{tr} A = \operatorname{tr}((ST)S^{-1}) = \operatorname{tr}(S^{-1}(ST)) = \operatorname{tr} T = \sum_i \lambda_i$, using (a) with the grouping shown. $\square$

Trace and determinant are the two coefficients of the characteristic polynomial you can read off instantly; for 2×2 : $\det(A - \lambda I) = \lambda^2 - (\operatorname{tr} A)\lambda + \det A$, so eigenvalues of a 2×2 are $\frac{\operatorname{tr} A \pm \sqrt{(\operatorname{tr} A)^2 - 4 \det A}}{2}$ — worth memorizing.

14.2 Practice: by hand

Example 14.6. $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: $\operatorname{tr} A = 4$, $\det A = 3$; eigenvalues solve $\lambda^2 - 4\lambda + 3 = 0$: $\lambda = 1, 3$. Matches Example 10.7, with product $3 = \det$ and sum $4 = \operatorname{tr}$. $\checkmark$

Example 14.7 (Volume scaling). The map $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ sends the unit square to the parallelogram spanned by $(2, 0)$ and $(1, 3)$: base 2, height 3, area $6 = \det A$. The shear part (the 1) moved the top edge sideways without changing the area — exactly the “add a multiple of one column to another” invariance.

EE261 connection. Determinants enter EE261 exactly once, but at a memorable moment: the higher-dimensional Fourier transform. For $f : \mathbb{R}^n \rightarrow \mathbb{C}$ and an invertible A , the transform of the stretched signal $f(Ax)$ is $\frac{1}{|\det A|} \hat{f}(A^{-T}\xi)$ — the general stretch theorem. The $|\det A|$ is the volume-scaling factor from the change of variables in the integral, and A^{-T} is why frequency space transforms “contragradiently” (images stretched in space compress in

frequency, along dual axes). When Osgood derives this for 2-D images and CT scans, you will know exactly where every symbol comes from.

14.3 NumPy

```
import numpy as np
A = np.array([[2., 1.], [1., 2.]])
np.linalg.det(A)          # 3.0000000000000004
np.trace(A)               # 4.0
lam = np.linalg.eigvals(A)
np.allclose(lam.prod(), np.linalg.det(A)) # True
np.allclose(lam.sum(), np.trace(A))      # True

# Never test invertibility with det in floating point at scale:
B = 1e-3 * np.eye(300)
np.linalg.det(B)          # 1e-900 -> underflows to 0.0
np.linalg.cond(B)         # 1.0    -> perfectly well-conditioned!
```

`det` costs $O(n^3)$ via elimination (product of pivots, sign-tracked). The last snippet is a real-world warning: determinants scale like $(\text{size})^n$, so a perfectly invertible matrix can have an astronomically tiny determinant. Rank, condition number, or singular values are the numerically meaningful invertibility tests; the determinant is a tool of theory (and of hand computation on small matrices), which is precisely how Axler's ordering treats it.

14.4 Exercises

Exercise 14.1 (Hand). Compute by row reduction, tracking swaps and column operations'

effects: $\det \begin{pmatrix} 1 & 2 & 0 \\ 2 & 4 & 1 \\ 1 & 0 & 3 \end{pmatrix}$. Cross-check with the 3×3 formula.

Exercise 14.2 (Hand). Using trace and det only, find the eigenvalues of $\begin{pmatrix} 5 & 4 \\ 1 & 2 \end{pmatrix}$ and $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ (the latter over $\mathbb{C}$; interpret).

Exercise 14.3 (Proof, ★). From the three defining properties alone (no permutation formula), derive $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$: expand $\det((a, c), (b, d))$ by multilinearity into four determinants of standard-basis columns, and evaluate each using alternation and normalization.

Exercise 14.4 (Proof). Prove that similar matrices ($B = S^{-1}AS$) have the same determinant, trace, and eigenvalues — so all three are properties of the underlying linear map, independent of basis.

Exercise 14.5 (NumPy). Sample 10^5 random 2×2 matrices with standard normal entries and estimate the probability that $\det > 0$. Then verify the volume interpretation: for a fixed A , map 10^5 uniform points in the unit square through A and compare the area of the image (estimate via a bounding-box Monte Carlo or the convex hull) with $|\det A|$.

Deeper reading. LADR 9A (bilinear and quadratic forms), 9B (alternating multilinear forms), 9C (determinants — the characterization above); for the skipped Chapter 8 (generalized eigenvectors, i.e. what replaces diagonalization when it fails, and the trace’s operator-theoretic role in 8D): read after EE261 if the shear example left you curious — no Fourier topic requires it. Strang, ch. “Determinants.”

15 Capstone II: The DFT in NumPy

NEEDED FOR: EE261

Forty-five minutes; everything in Part II at once. Type it.

Task. Build the DFT matrix from scratch, certify it is (scaled-)unitary, use it to denoise a signal, and confirm the convolution theorem against a hand-rolled circular convolution.

```
import numpy as np
rng = np.random.default_rng(0)

# --- 1. The matrix and its unitarity (Lessons 9, 12) ---
N = 128
n = np.arange(N)
W = np.exp(-2j * np.pi * np.outer(n, n) / N)
F = W / np.sqrt(N)
assert np.allclose(F.conj().T @ F, np.eye(N), atol=1e-10) # unitary

# --- 2. A noisy two-tone signal ---
t = n / N
clean = np.sin(2*np.pi*5*t) + 0.5*np.sin(2*np.pi*12*t)
x = clean + 0.4 * rng.standard_normal(N)

# --- 3. Analyze: Parseval + peak-finding (Lessons 11, 12) ---
xh = W @ x # same as np.fft.fft(x)
assert np.allclose((np.abs(x)**2).sum(),
                    (np.abs(xh)**2).sum() / N, atol=1e-8) # Parseval
peaks = np.argsort(np.abs(xh[:N//2]))[-2:]
print("dominant frequencies:", sorted(peaks)) # [5, 12]

# --- 4. Filter: keep low frequencies + mirror band ---
keep = (n <= 15) | (n >= N - 15)
y = np.real(np.conj(W).T @ (xh * keep) / N) # inverse DFT
print("noise before/after:",
      np.linalg.norm(x - clean), np.linalg.norm(y - clean))

# --- 5. Convolution theorem (Lesson 12) ---
c = np.zeros(N); c[:4] = 0.25 # moving-average filter
conv_direct = np.array([(c * x[(k - n) % N]).sum() for k in range(N)])
conv_fft = np.real(np.fft.ifft(np.fft.fft(c) * np.fft.fft(x)))
assert np.allclose(conv_direct, conv_fft, atol=1e-9)
```

Checkpoints.

1. The unitarity assertion passes: columns of F are orthonormal in the complex inner product — Lemma 9.2 at work (change a `conj()` to see it fail).
2. Parseval holds to 10^{-8} : the DFT moves energy between representations without creating or destroying it (Theorem 12.4).
3. The two injected tones are the two largest $|\hat{x}_k|$ despite the noise — orthogonality is what lets each frequency's coefficient ignore the others.
4. Denoising by zeroing high-frequency coefficients is a *projection* (Theorem 6.3) onto the span of 31 Fourier basis vectors — Part I's least-squares geometry, in Part II's basis. The output is real only because the mirror band was kept (Exercise 12.5).
5. Direct circular convolution ($O(N^2)$ as written) and the FFT route ($O(N \log N)$) agree to 10^{-9} : Theorem 12.6, numerically certified. Re-run with $N = 4096$ and time both.

Final self-check list for Part II

From memory: compute with complex exponentials and state the roots-of-unity lemma; find eigenvalues/eigenvectors of a 2×2 and diagonalize it; state and prove that self-adjoint matrices have real eigenvalues and orthogonal eigenspaces; state the spectral theorem and the three equivalent faces of unitarity; write the DFT matrix, prove $W^*W = NI$, and state the convolution theorem; explain what the SVD's rotation–stretch–rotation says about an arbitrary matrix; compute 2×2 and 3×3 determinants and eigenvalues via trace/det. That is LADR Chapters 5–7 and 9 at working strength — and the linear-algebra spine of EE261.

Part III: The CS221 Probability Core

CS221 lists CS109-level probability as a required prerequisite, and it collects on the debt: Bayes' rule and conditional independence power the Bayesian-network and HMM lectures, expectations define the utilities that MDPs and expectimax maximize, and every training loss is an expectation over data. This part is the minimum working dose, following Bertsekas & Tsitsiklis (B&T), *Introduction to Probability* (2nd ed.), Chapters 1–3, in the same three-layer style as Part I. EE278 assumes all of this as a floor, so nothing here is wasted on the longer road either.

16 Probability Models, Conditioning, and Bayes' Rule

NEEDED FOR: CS221 (core: Bayesian networks, HMMs); EE278 (assumed)

16.1 Theory

Definition 16.1 (Probability model). A *probabilistic model* consists of a *sample space* Ω — the set of all possible outcomes of an experiment, exactly one of which occurs — and a *probability law* $\mathbf{P}$ assigning to each event $A \subseteq \Omega$ a number $\mathbf{P}(A)$, satisfying the axioms:

- (i) *nonnegativity*: $\mathbf{P}(A) \geq 0$ for every event A ;
- (ii) *additivity*: if $A_1, A_2, \dots$ are disjoint events, then $\mathbf{P}(A_1 \cup A_2 \cup \dots) = \mathbf{P}(A_1) + \mathbf{P}(A_2) + \dots$;
- (iii) *normalization*: $\mathbf{P}(\Omega) = 1$.

Everything else in Parts III–IV is a consequence of these three axioms — the same game as Lesson 1's vector-space axioms, and worth playing for the same reason.

Proposition 16.2 (First consequences). *For events A, B :*

- (a) $\mathbf{P}(\emptyset) = 0$ and $\mathbf{P}(A^c) = 1 - \mathbf{P}(A)$;
- (b) if $A \subseteq B$ then $\mathbf{P}(A) \leq \mathbf{P}(B)$ (*monotonicity*);
- (c) $\mathbf{P}(A \cup B) = \mathbf{P}(A) + \mathbf{P}(B) - \mathbf{P}(A \cap B)$, hence $\mathbf{P}(A \cup B) \leq \mathbf{P}(A) + \mathbf{P}(B)$ (*the union bound*).

Proof. (a) Ω and $\emptyset$ are disjoint with union Ω , so $1 = \mathbf{P}(\Omega) = \mathbf{P}(\Omega) + \mathbf{P}(\emptyset)$, giving $\mathbf{P}(\emptyset) = 0$; A and A^c are disjoint with union Ω , so $\mathbf{P}(A) + \mathbf{P}(A^c) = 1$.

(b) $B = A \cup (B \cap A^c)$, a disjoint union, so $\mathbf{P}(B) = \mathbf{P}(A) + \mathbf{P}(B \cap A^c) \geq \mathbf{P}(A)$ by nonnegativity.

(c) Write $A \cup B$ as the disjoint union $A \cup (B \cap A^c)$ and B as the disjoint union $(A \cap B) \cup (B \cap A^c)$. Then

$$\mathbf{P}(A \cup B) = \mathbf{P}(A) + \mathbf{P}(B \cap A^c) = \mathbf{P}(A) + (\mathbf{P}(B) - \mathbf{P}(A \cap B)),$$

and the inequality follows since $\mathbf{P}(A \cap B) \geq 0$. □

When Ω is finite with equally likely outcomes, $\mathbf{P}(A) = |A|/|\Omega|$: probability is counting. (B&T Chapter 1.6 develops the counting toolkit — permutations, combinations $\binom{n}{k}$ — which CS109 drills; we use $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ freely below.)

Definition 16.3 (Conditional probability). For events A, B with $\mathbf{P}(B) > 0$, the *conditional probability of A given B* is

$$\mathbf{P}(A | B) = \frac{\mathbf{P}(A \cap B)}{\mathbf{P}(B)}.$$

Conditioning is re-normalization: the outcomes in B become the new sample space, with probabilities scaled up by $1/\mathbf{P}(B)$ so they again sum to 1. One checks directly that $\mathbf{P}(\cdot | B)$ satisfies the three axioms — so *every* theorem about probability laws applies verbatim to conditional laws. This innocuous remark is used constantly.

Theorem 16.4 (Multiplication rule, total probability, Bayes' rule). Let $A_1, \dots, A_n$ be disjoint events whose union is Ω (a partition), each with $\mathbf{P}(A_i) > 0$, and let B be an event with $\mathbf{P}(B) > 0$.

(a) Multiplication rule: $\mathbf{P}(A \cap B) = \mathbf{P}(B) \mathbf{P}(A | B) = \mathbf{P}(A) \mathbf{P}(B | A)$ (whenever the conditioning events have positive probability); more generally

$$\mathbf{P}(A_1 \cap \dots \cap A_n) = \mathbf{P}(A_1) \mathbf{P}(A_2 | A_1) \dots \mathbf{P}(A_n | A_1 \cap \dots \cap A_{n-1}).$$

(b) Total probability theorem: $\mathbf{P}(B) = \sum_{i=1}^n \mathbf{P}(A_i) \mathbf{P}(B | A_i)$.

(c) Bayes' rule:

$$\mathbf{P}(A_i | B) = \frac{\mathbf{P}(A_i) \mathbf{P}(B | A_i)}{\sum_{j=1}^n \mathbf{P}(A_j) \mathbf{P}(B | A_j)}.$$

Proof. (a) is the definition of conditional probability rearranged; the general form follows by induction, multiplying one more conditional probability at each step (each factor's numerator cancels the previous factor's intersection).

(b) The sets $B \cap A_1, \dots, B \cap A_n$ are disjoint (the A_i are) and their union is B (the A_i cover Ω). By additivity and then the multiplication rule,

$$\mathbf{P}(B) = \sum_i \mathbf{P}(B \cap A_i) = \sum_i \mathbf{P}(A_i) \mathbf{P}(B | A_i).$$

(c) By definition and then (a) for the numerator and (b) for the denominator:

$$\mathbf{P}(A_i | B) = \frac{\mathbf{P}(A_i \cap B)}{\mathbf{P}(B)} = \frac{\mathbf{P}(A_i) \mathbf{P}(B | A_i)}{\sum_j \mathbf{P}(A_j) \mathbf{P}(B | A_j)}. \quad \square$$

Bayes' rule is *inference*: the A_i are competing hypotheses (“causes”), B is the observed evidence (“effect”), and the rule converts the easy direction $\mathbf{P}(\text{effect} | \text{cause})$ — usually given by a model — into the wanted direction $\mathbf{P}(\text{cause} | \text{effect})$. The $\mathbf{P}(A_i)$ are *priors*, the $\mathbf{P}(A_i | B)$ *posteriors*.

Definition 16.5 (Independence). Events A and B are *independent* if $\mathbf{P}(A \cap B) = \mathbf{P}(A) \mathbf{P}(B)$ (equivalently, when $\mathbf{P}(B) > 0$: $\mathbf{P}(A | B) = \mathbf{P}(A)$ — learning B tells you nothing about A). Events A and B are *conditionally independent given C* if $\mathbf{P}(A \cap B | C) = \mathbf{P}(A | C) \mathbf{P}(B | C)$.

Two warnings, both of which CS221's graphical-models lectures weaponize: independence does *not* survive conditioning (two independent coin flips become dependent given “the two flips differ”), and conditional independence does not imply independence (nor conversely). Independence is a modeling *assumption* about the world, not something you can usually read off the numbers.

16.2 Practice: by hand

Example 16.6 (Two rolls of a fair die). $\Omega = \{(i, j) : 1 \leq i, j \leq 6\}$, all 36 outcomes equally likely. Let $A = \text{“the sum is 8”} = \{(2, 6), (3, 5), (4, 4), (5, 3), (6, 2)\}$, so $\mathbf{P}(A) = 5/36$. Let $B = \text{“the first roll is even”}$ (18 outcomes, $\mathbf{P}(B) = 1/2$). Then $A \cap B = \{(2, 6), (4, 4), (6, 2)\}$, so

$$\mathbf{P}(A \mid B) = \frac{3/36}{18/36} = \frac{1}{6} > \frac{5}{36} = \mathbf{P}(A) :$$

evidence that the first roll is even makes a sum of 8 slightly *more* likely — A and B are dependent.

Example 16.7 (The false-positive puzzle — Bayes’ rule you must own). A test for a condition affecting 1 in 1000 people is 99% accurate in both directions: $\mathbf{P}(+ \mid \text{sick}) = 0.99$ and $\mathbf{P}(- \mid \text{healthy}) = 0.99$. You test positive. The probability you are sick is *not* 0.99; by Bayes’ rule with the partition {sick, healthy}:

$$\mathbf{P}(\text{sick} \mid +) = \frac{0.001 \times 0.99}{0.001 \times 0.99 + 0.999 \times 0.01} = \frac{0.00099}{0.00099 + 0.00999} \approx 0.09.$$

About 9%: the false positives from the huge healthy population swamp the true positives from the tiny sick one. The prior matters as much as the test.

Example 16.8 (Conditional independence, concretely). Two coins: one fair, one with $\mathbf{P}(\text{heads}) = 0.9$. Pick one at random and flip it twice. Given *which* coin was picked, the flips are independent. Unconditionally they are not: a first head is evidence for the biased coin, which makes a second head more likely. Compute: $\mathbf{P}(H_1) = \frac{1}{2}(0.5) + \frac{1}{2}(0.9) = 0.7$, while

$$\mathbf{P}(H_2 \mid H_1) = \frac{\mathbf{P}(H_1 \cap H_2)}{\mathbf{P}(H_1)} = \frac{\frac{1}{2}(0.25) + \frac{1}{2}(0.81)}{0.7} = \frac{0.53}{0.7} \approx 0.757 > 0.7.$$

This two-coin experiment is a two-node Bayesian network, and the calculation you just did is exactly CS221’s “inference by enumeration.”

CS221 connection. A Bayesian network *is* the multiplication rule with a conditional-independence assumption: the joint distribution factors as $\mathbf{P}(X_1, \dots, X_n) = \prod_i \mathbf{P}(X_i \mid \text{Parents}(X_i))$. Inference in the network — filtering in an HMM, diagnosing from symptoms — is Bayes’ rule plus total probability, applied systematically. Example 16.7 *is* a two-variable Bayesian network. When CS221 says “condition on the evidence and renormalize,” it is invoking the fact that $\mathbf{P}(\cdot \mid B)$ is a probability law.

16.3 NumPy

Monte Carlo estimation — simulate the experiment many times, count — is both a sanity check on hand calculations and CS221’s fallback when exact inference is too expensive (particle filtering, Gibbs sampling).

```
import numpy as np
rng = np.random.default_rng(0)

# The false-positive puzzle, simulated:
n = 1_000_000
```

```

sick = rng.random(n) < 0.001
test_pos = np.where(sick, rng.random(n) < 0.99,
                    rng.random(n) < 0.01)
print(test_pos[sick].mean())      # ~0.99 = P(+ | sick)
print(sick[test_pos].mean())      # ~0.09 = P(sick | +)   Bayes!

# Conditioning = filter the samples, then take the mean:
# P(A | B) ~ (A & B).mean() / B.mean() -- the definition, empirically.

```

The estimate of a probability p from n samples has standard deviation $\sqrt{p(1-p)/n}$ (Lesson 24 proves the machinery behind this), so 10^6 samples give about 3 decimal places. Note the idiom: conditioning is *boolean indexing* — `sick[test_pos]` restricts attention to the samples where the evidence holds, which is precisely “ B becomes the new sample space.”

16.4 Exercises

Exercise 16.1 (Hand). Three fair coin flips. List the sample space. Let A = “at least two heads” and B = “the first flip is heads.” Compute $\mathbf{P}(A)$ and $\mathbf{P}(A | B)$, and check whether A and B are independent.

Exercise 16.2 (Hand). A spam filter knows: 30% of email is spam, the word “free” appears in 40% of spam and 5% of non-spam. Apply Bayes’ rule to find $\mathbf{P}(\text{spam} | \text{“free”})$. Then recompute with a prior of 80% spam and note how the posterior moves — this is Capstone III in miniature.

Exercise 16.3 (Proof, ★). Prove the general union bound $\mathbf{P}(\bigcup_{i=1}^n A_i) \leq \sum_{i=1}^n \mathbf{P}(A_i)$ by induction from Proposition 16.2(c). (This one-line tool underlies half of the analyses in machine learning theory.)

Exercise 16.4 (Proof). Verify that $\mathbf{P}(\cdot | B)$ satisfies the three probability axioms. Then prove the “chain rule with a spectator”: $\mathbf{P}(A \cap C | B) = \mathbf{P}(C | B) \mathbf{P}(A | B \cap C)$ whenever the conditioning events have positive probability.

Exercise 16.5 (Proof). Show that if A and B are independent, so are A and B^c , and so are A^c and B^c .

Exercise 16.6 (NumPy). Simulate the two-coin experiment (fair coin and 0.9-heads coin, pick one uniformly, flip twice) with 10^6 trials. Estimate $\mathbf{P}(H_2 | H_1)$ by boolean indexing and confirm the hand answer ≈ 0.757 . Then estimate $\mathbf{P}(H_2 | H_1, \text{coin} = \text{fair})$ and confirm conditional independence: it should be ≈ 0.5 .

Deeper reading. B&T 1.1–1.2 (models and axioms), 1.3–1.4 (conditioning, total probability, Bayes), 1.5 (independence), 1.6 (counting); the CS221 “Bayesian networks” module notes re-derive Theorem 16.4 in network notation.

17 Discrete Random Variables and Expectation

NEEDED FOR: CS221 (core: utilities, expectimax, losses); EE278 (assumed)

17.1 Theory

Definition 17.1 (Random variable, PMF). A *random variable* is a function $X : \Omega \rightarrow \mathbb{R}$ — a number determined by the outcome. It is *discrete* if its range is finite or countable. The *probability mass function* (PMF) of a discrete X is

$$p_X(x) = \mathbf{P}(X = x) \quad (\text{shorthand for } \mathbf{P}(\{\omega \in \Omega : X(\omega) = x\})),$$

and satisfies $p_X(x) \geq 0$ and $\sum_x p_X(x) = 1$ (additivity over the disjoint events $\{X = x\}$).

The four discrete distributions to know cold:

- *Bernoulli*(p): $X \in \{0, 1\}$ with $p_X(1) = p$. Models one yes/no trial.
- *Binomial*(n, p): X = number of successes in n independent Bernoulli(p) trials; $p_X(k) = \binom{n}{k} p^k (1-p)^{n-k}$ for $k = 0, \dots, n$.
- *Geometric*(p): X = number of trials up to and including the first success; $p_X(k) = (1-p)^{k-1} p$ for $k = 1, 2, \dots$.
- *Poisson*(λ): $p_X(k) = e^{-\lambda} \lambda^k / k!$ for $k = 0, 1, 2, \dots$; the $n \rightarrow \infty, np \rightarrow \lambda$ limit of the binomial (Lesson 25 uses this).

Definition 17.2 (Expectation, variance). The *expectation* (mean) of a discrete random variable X is

$$\mathbf{E}[X] = \sum_x x p_X(x)$$

(assuming the sum converges absolutely), and its *variance* is

$$\text{var}(X) = \mathbf{E}[(X - \mathbf{E}[X])^2] \geq 0, \quad \text{with standard deviation } \sigma_X = \sqrt{\text{var}(X)}.$$

Theorem 17.3 (Expected value rule / LOTUS). For any function $g : \mathbb{R} \rightarrow \mathbb{R}$,

$$\mathbf{E}[g(X)] = \sum_x g(x) p_X(x).$$

Proof. Let $Y = g(X)$; Y is discrete with $p_Y(y) = \sum_{x: g(x)=y} p_X(x)$ (additivity over the disjoint events $\{X = x\}$ composing $\{Y = y\}$). Then

$$\mathbf{E}[Y] = \sum_y y p_Y(y) = \sum_y \sum_{x: g(x)=y} y p_X(x) = \sum_y \sum_{x: g(x)=y} g(x) p_X(x) = \sum_x g(x) p_X(x),$$

since every x appears in exactly one inner sum. □

The name LOTUS — “law of the unconscious statistician” — records that people apply it without noticing it needs proof: the naive move (average $g(x)$ against p_X , without ever computing $p_{g(X)}$) is correct.

Theorem 17.4 (Linearity of expectation). For random variables X, Y on the same experiment and constants a, b :

$$\mathbf{E}[aX + b] = a \mathbf{E}[X] + b, \quad \mathbf{E}[X + Y] = \mathbf{E}[X] + \mathbf{E}[Y].$$

The second identity requires no independence.

Proof. The first is LOTUS with $g(x) = ax + b$: $\sum_x (ax + b)p_X(x) = a \sum_x xp_X(x) + b \sum_x p_X(x) = a\mathbf{E}[X] + b$. For the second, work on the underlying sample space (for simplicity, countable Ω , writing $p(\omega) = \mathbf{P}(\{\omega\})$):

$$\mathbf{E}[X + Y] = \sum_{\omega} (X(\omega) + Y(\omega))p(\omega) = \sum_{\omega} X(\omega)p(\omega) + \sum_{\omega} Y(\omega)p(\omega) = \mathbf{E}[X] + \mathbf{E}[Y]. \quad \square$$

Proposition 17.5 (Variance identities). $\text{var}(X) = \mathbf{E}[X^2] - (\mathbf{E}[X])^2$, and $\text{var}(aX + b) = a^2 \text{var}(X)$.

Proof. Write $\mu = \mathbf{E}[X]$. Expanding the square and using linearity,

$$\text{var}(X) = \mathbf{E}[X^2 - 2\mu X + \mu^2] = \mathbf{E}[X^2] - 2\mu\mathbf{E}[X] + \mu^2 = \mathbf{E}[X^2] - \mu^2.$$

For the second: $aX + b$ has mean $a\mu + b$, so $\text{var}(aX + b) = \mathbf{E}[(aX + b - a\mu - b)^2] = \mathbf{E}[a^2(X - \mu)^2] = a^2 \text{var}(X)$ — the shift b disappears, as it should: shifting a distribution does not change its spread. $\square$

Example 17.6 (The catalog, computed). *Bernoulli*(p): $\mathbf{E}[X] = p$, and $\mathbf{E}[X^2] = p$ (since $X^2 = X$), so $\text{var}(X) = p - p^2 = p(1 - p)$ — maximized at $p = \frac{1}{2}$, the most unpredictable coin.

Binomial(n, p): write $X = X_1 + \dots + X_n$ as a sum of Bernoulli indicators. By linearity (no independence needed!): $\mathbf{E}[X] = np$. For the variance, independence *is* needed (Lesson 18): $\text{var}(X) = np(1 - p)$.

Geometric(p): $\mathbf{E}[X] = 1/p$. Slick derivation via the total expectation theorem (next lesson) or the derivative trick: for $|q| < 1$, differentiate $\sum_{k \geq 0} q^k = \frac{1}{1-q}$ to get $\sum_{k \geq 1} kq^{k-1} = \frac{1}{(1-q)^2}$, so with $q = 1 - p$: $\mathbf{E}[X] = p \sum_{k \geq 1} kq^{k-1} = p \cdot \frac{1}{p^2} = \frac{1}{p}$.

Poisson(λ): $\mathbf{E}[X] = \sum_{k \geq 1} k e^{-\lambda} \frac{\lambda^k}{k!} = \lambda e^{-\lambda} \sum_{k \geq 1} \frac{\lambda^{k-1}}{(k-1)!} = \lambda$; a similar computation gives $\text{var}(X) = \lambda$ too.

17.2 Practice: by hand

Example 17.7 (Expected utility, the CS221 shape). A game: pay 1 to play; roll a die; win 4 if you roll a six, else nothing. Your net gain G has $p_G(3) = \frac{1}{6}$, $p_G(-1) = \frac{5}{6}$, so

$$\mathbf{E}[G] = 3 \cdot \frac{1}{6} + (-1) \cdot \frac{5}{6} = -\frac{1}{3}.$$

Play 100 times and linearity says your expected total is -33.3 : no cleverness about dependence between rounds required. Decisions in CS221 (expectimax, MDP policies) are rankings of actions by exactly this kind of number.

Example 17.8 (Indicators make hard counts easy). n people throw their hats in a pile and each takes one back uniformly at random. How many get their own hat, on average? Let X_i be the indicator that person i succeeds: $\mathbf{E}[X_i] = \mathbf{P}(X_i = 1) = 1/n$. The total is $X = \sum_i X_i$, so by linearity

$$\mathbf{E}[X] = \sum_{i=1}^n \frac{1}{n} = 1,$$

for *every* n — even though the X_i are dependent (if $n - 1$ people got their own hats, so did the last). Linearity through dependence is the single most-used trick in average-case analysis.

CS221 connection. An MDP policy is evaluated by its *expected utility*; the value of a chance node in expectimax is $\sum_s \mathbf{P}(s) V(s)$ — an expectation; the training loss $\frac{1}{m} \sum_i \text{Loss}_i$ is the expectation of the loss under the empirical distribution of the data, and generalization asks how it relates to the expectation under the true distribution. When CS221 writes $V_\pi(s) = \sum_{s'} T(s, a, s') [\text{Reward} + \gamma V_\pi(s')]$, read it as: value = expected (reward + discounted future value). Expectation is the course’s unit of account.

17.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

# A PMF as two arrays; expectation as a dot product (Lesson 2!):
vals = np.array([3., -1.])
pmf = np.array([1/6, 5/6])
E = vals @ pmf                # -0.333...  E[X] = <values, probs>
E2 = (vals**2) @ pmf          # LOTUS with g(x)=x^2
var = E2 - E**2

# Sampling from a PMF, and the sample mean converging to E[X]:
draws = rng.choice(vals, size=100_000, p=pmf)
print(draws.mean(), draws.var()) # ~ -0.333, ~ var

# Built-in families:
rng.binomial(10, 0.3, size=5)    # Binomial(10, 0.3) draws
rng.geometric(0.2, size=5)      # Geometric(0.2)
rng.poisson(4.0, size=5)        # Poisson(4)
```

Note the punchline of the first block: *an expectation is a dot product* between the value vector and the probability vector — Lesson 2’s machinery reappears. Computing $\mathbf{E}[g(X)]$ for a discrete X with k values is $O(k)$; estimating it from n samples is $O(n)$ with error shrinking like $1/\sqrt{n}$ (Lesson 24).

17.4 Exercises

Exercise 17.1 (Hand). Let X be the number of heads in 3 fair flips. Tabulate p_X , compute $\mathbf{E}[X]$ and $\text{var}(X)$ from the table, and confirm both against the $\text{Binomial}(3, \frac{1}{2})$ formulas.

Exercise 17.2 (Hand). A quiz has 5 multiple-choice questions, 4 options each; you guess uniformly and independently. Let X = number correct. What are $\mathbf{E}[X]$ and $\text{var}(X)$? What is $\mathbf{P}(X \geq 1)$? (Complement trick.)

Exercise 17.3 (Proof, ★). Prove that $\mathbf{E}[X]$ minimizes the mean squared error: for any constant c , $\mathbf{E}[(X - c)^2] = \text{var}(X) + (c - \mathbf{E}[X])^2$, so the unique minimizing c is $\mathbf{E}[X]$. (Expand; linearity does the rest. Lesson 22 upgrades this from constants to functions of an observation — it becomes MMSE estimation.)

Exercise 17.4 (Proof). Derive $\mathbf{E}[X] = 1/p$ for the geometric distribution using the memoryless recursion: condition on the first trial to get $\mathbf{E}[X] = 1 + (1 - p)\mathbf{E}[X]$, and justify the conditioning

step. (This is the total expectation theorem of Lesson 18 in action; compare with the derivative trick of Example 17.6.)

Exercise 17.5 (NumPy). Estimate the hat-problem answer (Example 17.8) by simulation for $n = 3, 10, 100$ (use `rng.permutation` and count fixed points). Confirm the mean is ≈ 1 throughout, and report the observed variance. (It is also ≈ 1 — curious? The Poisson limit explains it.)

Deeper reading. B&T 2.1–2.4 (PMFs, expectation, variance; the distribution catalog); B&T 2.4 has the geometric-series variance computations spelled out.

18 Multiple Random Variables and Conditioning

NEEDED FOR: CS221 (core: naive Bayes, HMMs, MDP value computations); EE278 (assumed)

18.1 Theory

Definition 18.1 (Joint, marginal, conditional PMFs). For discrete random variables X, Y on the same experiment, the *joint PMF* is $p_{X,Y}(x, y) = \mathbf{P}(X = x, Y = y)$. The *marginals* recover each variable alone:

$$p_X(x) = \sum_y p_{X,Y}(x, y), \quad p_Y(y) = \sum_x p_{X,Y}(x, y)$$

(additivity over the partition by the other variable's value). For $p_Y(y) > 0$, the *conditional PMF* is

$$p_{X|Y}(x | y) = \frac{p_{X,Y}(x, y)}{p_Y(y)},$$

a genuine PMF in x for each fixed y . X and Y are *independent* if $p_{X,Y}(x, y) = p_X(x)p_Y(y)$ for all x, y .

All of Lesson 16 transfers: the multiplication rule reads $p_{X,Y}(x, y) = p_Y(y)p_{X|Y}(x | y)$, and Bayes' rule swaps which variable is conditioned on. Everything extends to three or more variables by adding coordinates.

Definition 18.2 (Conditional expectation). $\mathbf{E}[X | Y = y] = \sum_x x p_{X|Y}(x | y)$: the mean of X in the re-normalized world where $Y = y$ is known.

Theorem 18.3 (Total expectation theorem).

$$\mathbf{E}[X] = \sum_y p_Y(y) \mathbf{E}[X | Y = y].$$

Proof. Expand the right side using the definitions and the multiplication rule:

$$\sum_y p_Y(y) \sum_x x p_{X|Y}(x | y) = \sum_y \sum_x x p_{X,Y}(x, y) = \sum_x x \sum_y p_{X,Y}(x, y) = \sum_x x p_X(x) = \mathbf{E}[X],$$

where the swap of summation order is legal by absolute convergence. $\square$

Read it as *divide and conquer for averages*: average within each scenario y , then average the scenario averages, weighted by how likely each scenario is. This theorem is the engine of every MDP value calculation and of Lesson 22.

Theorem 18.4 (Expectations and products under independence). *If X and Y are independent, then*

$$\mathbf{E}[XY] = \mathbf{E}[X] \mathbf{E}[Y] \quad \text{and} \quad \text{var}(X + Y) = \text{var}(X) + \text{var}(Y).$$

Proof. For the product, using LOTUS on the pair (whose proof is identical to Theorem 17.3's) and factoring the joint PMF:

$$\mathbf{E}[XY] = \sum_{x,y} xy p_X(x) p_Y(y) = \left(\sum_x x p_X(x) \right) \left(\sum_y y p_Y(y) \right) = \mathbf{E}[X] \mathbf{E}[Y].$$

For the variance, let $\tilde{X} = X - \mathbf{E}[X]$ and $\tilde{Y} = Y - \mathbf{E}[Y]$ (still independent, with mean zero). Then

$$\text{var}(X + Y) = \mathbf{E}[(\tilde{X} + \tilde{Y})^2] = \mathbf{E}[\tilde{X}^2] + 2 \mathbf{E}[\tilde{X}\tilde{Y}] + \mathbf{E}[\tilde{Y}^2] = \text{var}(X) + 2 \mathbf{E}[\tilde{X}]\mathbf{E}[\tilde{Y}] + \text{var}(Y),$$

and the cross term is $2 \cdot 0 \cdot 0 = 0$. $\square$

Contrast with Theorem 17.4: means *always* add; variances add only under independence (Lesson 21 measures the failure by *covariance*). This completes the binomial variance claimed in Example 17.6: a sum of n independent Bernoulli(p)'s has variance $np(1 - p)$.

18.2 Practice: by hand

Example 18.5 (A full joint-PMF workout). Let (X, Y) have joint PMF given by the table (rows $x = 0, 1$; columns $y = 0, 1, 2$):

$p_{X,Y}$	$y = 0$	$y = 1$	$y = 2$
$x = 0$	0.1	0.2	0.1
$x = 1$	0.2	0.1	0.3

Marginals: $p_X(0) = 0.4$, $p_X(1) = 0.6$; $p_Y = (0.3, 0.3, 0.4)$. Conditional: $p_{X|Y}(1 | 2) = 0.3/0.4 = 0.75$. Independence fails: $p_{X,Y}(1, 2) = 0.3$ but $p_X(1)p_Y(2) = 0.24$. Conditional expectation: $\mathbf{E}[X | Y = 2] = 0.75$, while $\mathbf{E}[X] = 0.6$ — observing $Y = 2$ raises the estimate of X . Total expectation check: $0.3 \cdot \frac{0.2}{0.3} + 0.3 \cdot \frac{0.1}{0.3} + 0.4 \cdot 0.75 = 0.2 + 0.1 + 0.3 = 0.6 = \mathbf{E}[X]$. $\checkmark$

Example 18.6 (Naive Bayes, by hand). Classify an email as spam ($C = 1$) or not ($C = 0$) from two binary features: F_1 = contains “free,” F_2 = contains “meeting.” Model: $\mathbf{P}(C = 1) = 0.3$, and *given the class*, features are conditionally independent with

$$\begin{aligned} \mathbf{P}(F_1 = 1 | C = 1) &= 0.4, & \mathbf{P}(F_1 = 1 | C = 0) &= 0.05, \\ \mathbf{P}(F_2 = 1 | C = 1) &= 0.02, & \mathbf{P}(F_2 = 1 | C = 0) &= 0.3. \end{aligned}$$

Observe $F_1 = 1, F_2 = 0$. By Bayes' rule with the factored likelihood:

$$\mathbf{P}(C = 1 | F_1 = 1, F_2 = 0) = \frac{0.3 \times 0.4 \times 0.98}{0.3 \times 0.4 \times 0.98 + 0.7 \times 0.05 \times 0.7} = \frac{0.1176}{0.1176 + 0.0245} \approx 0.83.$$

“Free” without “meeting” pushes the posterior from 30% to 83% spam. The conditional-independence assumption is what let us multiply the per-feature likelihoods — with d features it reduces 2^d joint parameters to $2d$.

CS221 connection. Example 18.6 is a Bayesian network ($C \rightarrow F_1, C \rightarrow F_2$), and the computation is the course’s inference-by-enumeration algorithm. An HMM adds time: hidden states form a chain (Lesson 26), each emitting an observation, and filtering alternates the multiplication rule (predict) with Bayes’ rule (update). The total expectation theorem is also how Bellman equations average over next states in MDPs. Master the two examples above and the graphical-models unit becomes bookkeeping.

18.3 NumPy

```
import numpy as np

# The joint PMF of Example 18.5 as a 2x3 array:
P = np.array([[0.1, 0.2, 0.1],
              [0.2, 0.1, 0.3]])
pX = P.sum(axis=1)      # [0.4, 0.6]      marginalize out Y
pY = P.sum(axis=0)      # [0.3, 0.3, 0.4] marginalize out X

# Conditioning on Y=2: take a column, renormalize (Bayes in one line):
pX_given_Y2 = P[:, 2] / P[:, 2].sum()    # [0.25, 0.75]

# Independence check: does the joint equal the outer product?
np.allclose(P, np.outer(pX, pY))          # False -> dependent

# E[X | Y=y] for all y at once, then total expectation:
x = np.array([0., 1.])
E_X_given_Y = x @ (P / pY)                # [0.667, 0.333, 0.75]
print(E_X_given_Y @ pY)                   # 0.6 = E[X] (Theorem 18.3)
```

Everything is Part I linear algebra: marginalization is summing an axis, conditioning is slice-and-renormalize, the independence test is a rank-1 (outer product) check — Lesson 13’s language — and total expectation is a dot product. Joint tables over many variables blow up exponentially (2^d entries for d binary variables), which is exactly why CS221’s Bayesian networks exist: they store only the factors.

18.4 Exercises

Exercise 18.1 (Hand). Roll two fair dice; let X = the minimum, Y = the maximum. Tabulate the joint PMF for $X, Y \in \{1, 2, 3\}$ restricted to rolls of a 3-sided die (i.e. two fair 3-sided dice), find both marginals, and compute $\mathbf{E}[X | Y = 3]$.

Exercise 18.2 (Hand). Extend Example 18.6: observe $F_1 = 1, F_2 = 1$. Compute the posterior. Explain in one sentence why the two features pull in opposite directions.

Exercise 18.3 (Proof, ★). Prove the total expectation theorem in its conditional form: $\mathbf{E}[X | A] = \sum_y p_{Y|A}(y) \mathbf{E}[X | Y = y, A]$ for an event A with $\mathbf{P}(A) > 0$, by applying Theorem 18.3 to the conditional law $\mathbf{P}(\cdot | A)$ (recall: it satisfies the axioms, so every theorem applies to it).

Exercise 18.4 (Proof). Show that if X and Y are independent, then so are $g(X)$ and $h(Y)$ for any functions g, h ; conclude $\mathbf{E}[g(X)h(Y)] = \mathbf{E}[g(X)] \mathbf{E}[h(Y)]$.

Exercise 18.5 (NumPy). Build the joint PMF of (number of heads in first two flips, number of heads in all three flips) for three fair coin flips as a 3×4 array by enumerating the 8 outcomes. Verify the marginals against the binomial, check dependence, and compute $\mathbf{E}[\text{total} \mid \text{first two} = 2]$.

Deeper reading. B&T 2.5–2.7 (joint PMFs, conditioning, independence); B&T 4.3 revisits conditional expectation as a random variable, which is Lesson 22’s starting point.

19 Continuous Random Variables and the Normal

NEEDED FOR: CS221 (light: Gaussian noise, continuous features); EE278 (assumed, heavily)

19.1 Theory

Definition 19.1 (PDF, CDF). A random variable X is *continuous* if there is a nonnegative function f_X (the *probability density function*, PDF) with

$$\mathbf{P}(a \leq X \leq b) = \int_a^b f_X(x) dx \quad \text{for all } a \leq b, \quad \int_{-\infty}^{\infty} f_X(x) dx = 1.$$

The *cumulative distribution function* (CDF) of any random variable is $F_X(x) = \mathbf{P}(X \leq x)$; for continuous X , $F_X(x) = \int_{-\infty}^x f_X(t) dt$ and $f_X = F'_X$ wherever the derivative exists.

Two immediate consequences. First, $\mathbf{P}(X = x) = \int_x^x f_X = 0$ for every single point: individual values have probability zero, and only *intervals* carry probability — $f_X(x)$ is probability *per unit length* ($\mathbf{P}(x \leq X \leq x + \delta) \approx f_X(x) \delta$ for small δ), not a probability; it can exceed 1. Second, the CDF is the universal translator: it is defined for discrete and continuous variables alike, and differentiating (continuous) or differencing (discrete) recovers the local description.

Expectations are integrals now,

$$\mathbf{E}[X] = \int_{-\infty}^{\infty} x f_X(x) dx, \quad \mathbf{E}[g(X)] = \int_{-\infty}^{\infty} g(x) f_X(x) dx,$$

and *every* algebraic theorem of Lessons 17–18 (linearity, LOTUS, variance identities, total expectation, independence factoring) holds verbatim with sums replaced by integrals and PMFs by PDFs — the proofs transpose symbol-for-symbol. Joint PDFs $f_{X,Y}$, marginals ($\int f_{X,Y} dy$), and conditionals ($f_{X|Y} = f_{X,Y}/f_Y$) likewise mirror the discrete case.

The continuous catalog:

- *Uniform*(a, b): $f_X(x) = \frac{1}{b-a}$ on $[a, b]$; $\mathbf{E}[X] = \frac{a+b}{2}$, $\text{var}(X) = \frac{(b-a)^2}{12}$.
- *Exponential*(λ): $f_X(x) = \lambda e^{-\lambda x}$ for $x \geq 0$; $\mathbf{E}[X] = 1/\lambda$, $\text{var}(X) = 1/\lambda^2$; CDF $F_X(x) = 1 - e^{-\lambda x}$. The continuous geometric: memoryless (Lesson 25 proves it and builds the Poisson process on it).
- *Normal* (*Gaussian*) $\mathcal{N}(\mu, \sigma^2)$:

$$f_X(x) = \frac{1}{\sqrt{2\pi} \sigma} e^{-(x-\mu)^2/2\sigma^2}, \quad \mathbf{E}[X] = \mu, \quad \text{var}(X) = \sigma^2.$$

Proposition 19.2 (Standardization). *If $X \sim \mathcal{N}(\mu, \sigma^2)$ and $a \neq 0$, then $aX + b \sim \mathcal{N}(a\mu + b, a^2\sigma^2)$. In particular $Z = (X - \mu)/\sigma \sim \mathcal{N}(0, 1)$ (the standard normal), so every normal probability reduces to the standard CDF $\Phi(z) = \mathbf{P}(Z \leq z)$:*

$$\mathbf{P}(X \leq x) = \Phi\left(\frac{x - \mu}{\sigma}\right).$$

Proof. Let $Y = aX + b$ with $a > 0$ (the case $a < 0$ is symmetric). CDF first, then differentiate — the standard recipe for derived distributions (Lesson 21 makes it a method):

$$F_Y(y) = \mathbf{P}(aX + b \leq y) = \mathbf{P}\left(X \leq \frac{y - b}{a}\right) = F_X\left(\frac{y - b}{a}\right),$$

so by the chain rule

$$f_Y(y) = \frac{1}{a} f_X\left(\frac{y - b}{a}\right) = \frac{1}{\sqrt{2\pi} a \sigma} \exp\left(-\frac{(y - b - a\mu)^2}{2a^2\sigma^2}\right),$$

which is the $\mathcal{N}(a\mu + b, a^2\sigma^2)$ density. Taking $a = 1/\sigma$, $b = -\mu/\sigma$ gives $Z \sim \mathcal{N}(0, 1)$, and the displayed identity is the CDF computation just done. $\square$

Why this family rules: sums of many small independent effects are approximately normal (the central limit theorem, Lesson 24 — this is a theorem, not folklore), and the family is closed under the linear operations of Part I (Lesson 23 does the vector version). Useful numbers: $\mathbf{P}(|X - \mu| \leq \sigma) \approx 0.68$, $\leq 2\sigma \approx 0.95$, $\leq 3\sigma \approx 0.997$.

19.2 Practice: by hand

Example 19.3 (Working a normal probability). Scores are $\mathcal{N}(500, 100^2)$. What fraction exceeds 650?

$$\mathbf{P}(X > 650) = 1 - \Phi\left(\frac{650 - 500}{100}\right) = 1 - \Phi(1.5) \approx 1 - 0.9332 = 0.067.$$

No new integral was done — standardization plus one table/`scipy` lookup.

Example 19.4 (Exponential tail). Requests arrive with exponential($\lambda = 2$ per minute) gaps. Probability a gap exceeds 1 minute: $\mathbf{P}(X > 1) = e^{-2} \approx 0.135$. Given a gap has already lasted 1 minute, the chance it lasts past 2 minutes is $\mathbf{P}(X > 2 \mid X > 1) = e^{-4}/e^{-2} = e^{-2}$ — the same 0.135: memorylessness, proved in general in Lesson 25.

Example 19.5 (Continuous Bayes' rule). A signal Θ is $\mathcal{N}(0, 1)$; you observe $Y = \Theta + N$ where the noise N is $\mathcal{N}(0, 1)$, independent. Then $f_{Y|\Theta}(y \mid \theta)$ is the $\mathcal{N}(\theta, 1)$ density, and Bayes' rule for densities gives

$$f_{\Theta|Y}(\theta \mid y) = \frac{f_{\Theta}(\theta) f_{Y|\Theta}(y \mid \theta)}{\int f_{\Theta}(t) f_{Y|\Theta}(y \mid t) dt} \propto e^{-\theta^2/2} e^{-(y-\theta)^2/2} \propto e^{-(\theta - y/2)^2},$$

after completing the square in θ . So $\Theta \mid Y = y \sim \mathcal{N}(y/2, \frac{1}{2})$: the posterior mean $y/2$ splits the difference between the prior mean 0 and the data y , weighted by their precisions. This tiny computation is the seed of Kalman filtering (EE278) and of every “Gaussian prior $\times$ Gaussian likelihood” argument.

CS221 / EE278 connection. CS221 keeps continuous probability in a supporting role: Gaussian noise in continuous-state models and particle filtering, densities for continuous features. EE278 promotes it to the lead: signals are continuous random variables, noise is Gaussian, and Example 19.5 — estimate a Gaussian signal from a noisy observation — is the course’s central estimation problem, solved there in full generality (Lessons 22–23 build the machinery).

19.3 NumPy

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(0)

# Sampling and moments:
x = rng.normal(500, 100, size=1_000_000)
print((x > 650).mean())           # ~0.0668 = 1 - Phi(1.5)
print(x.mean(), x.std())          # ~500, ~100

# Exact answers via the CDF (no simulation):
1 - stats.norm.cdf(650, loc=500, scale=100) # 0.0668...
stats.norm.ppf(0.95, loc=500, scale=100)    # 664.5 (95th percentile)

# Histogram vs density -- the definition of a PDF, visually:
counts, edges = np.histogram(x, bins=100, density=True)
# counts now approximates f_X at the bin centers.

# Memorylessness of the exponential, empirically:
g = rng.exponential(scale=0.5, size=1_000_000) # lambda = 2
print((g > 1).mean(), (g > 2).mean() / (g > 1).mean()) # both ~0.135
```

`density=True` divides counts by $n \times \text{bin width}$ — turning a histogram into an estimate of probability *per unit length*, which is what a PDF is. For exact tail probabilities and quantiles use `scipy.stats`; simulate only when no closed form exists (which, from Part IV onward, is often).

19.4 Exercises

Exercise 19.1 (Hand). $X \sim \text{Uniform}(0, 2)$. Compute $\mathbf{E}[X]$, $\text{var}(X)$, and $\mathbf{E}[X^3]$ (LOTUS). Then find the CDF of $Y = X^2$ and differentiate to get f_Y . (CDF-then-differentiate — the derived-distribution recipe of Lesson 21.)

Exercise 19.2 (Hand). $X \sim \mathcal{N}(10, 4)$. Using Φ : compute $\mathbf{P}(X \leq 13)$, $\mathbf{P}(|X - 10| > 4)$, and the value c with $\mathbf{P}(X \leq c) = 0.99$ (use $\Phi^{-1}(0.99) \approx 2.33$). Careful: $\sigma = 2$, not 4.

Exercise 19.3 (Proof, ★). Prove that the normal density integrates to 1: let $I = \int_{-\infty}^{\infty} e^{-x^2/2} dx$, compute I^2 as a double integral, switch to polar coordinates, and conclude $I = \sqrt{2\pi}$. (One of the great tricks; EE261 meets it again when computing the Fourier transform of a Gaussian.)

Exercise 19.4 (Proof). Show the exponential is the *only* memoryless continuous positive distribution: if $\mathbf{P}(X > s + t) = \mathbf{P}(X > s)\mathbf{P}(X > t)$ for all $s, t \geq 0$, then the function

$g(t) = \ln \mathbf{P}(X > t)$ is additive, and (assuming right-continuity) $g(t) = -\lambda t$ for some $\lambda \geq 0$, i.e. $\mathbf{P}(X > t) = e^{-\lambda t}$.

Exercise 19.5 (NumPy). Verify Example 19.5 by simulation: draw 10^6 pairs (Θ, N) , form Y , select the samples with $Y \in (1.45, 1.55)$, and check that the selected Θ 's have mean ≈ 0.75 and variance ≈ 0.5 . (Conditioning on a zero-probability event $\{Y = 1.5\}$ is approximated by a thin slab — exactly how densities work.)

Deeper reading. B&T 3.1–3.3 (PDFs, CDFs, the normal), 3.4–3.6 (joint densities, conditioning, continuous Bayes); the polar-coordinates trick is B&T's Section 3.3 sidebar.

20 Capstone III: A Naive Bayes Classifier from Scratch

NEEDED FOR: CS221 (core)

Forty-five minutes; Lessons 16–18 at once, on the problem that made Bayes' rule famous in software: spam filtering. Type it, don't copy-paste.

Task. Generate a synthetic labeled dataset from a known naive Bayes model, learn the model's parameters back from the data by counting, classify with Bayes' rule in log space, and measure accuracy against the truth.

```
import numpy as np
rng = np.random.default_rng(0)

# --- 1. The true model: 6 binary features, 2 classes ---
d = 6
prior = 0.3 # P(class = spam)
p_spam = np.array([.60, .40, .25, .05, .02, .20]) # P(F_j=1 | spam)
p_ham = np.array([.05, .08, .10, .30, .25, .15]) # P(F_j=1 | ham)

# --- 2. Sample a dataset (Lesson 16: multiplication rule) ---
n = 5000
y = rng.random(n) < prior # class labels
probs = np.where(y[:, None], p_spam, p_ham) # per-row feature probs
X = rng.random((n, d)) < probs # conditionally indep. features

# --- 3. "Train": estimate parameters by counting (Lesson 17) ---
prior_hat = y.mean()
p_spam_hat = X[y].mean(axis=0) # P(F_j=1 | spam), empirical
p_ham_hat = X[~y].mean(axis=0)

# --- 4. Classify with Bayes' rule in log space (Lessons 16, 18) ---
def log_lik(X, p): # sum_j log P(F_j = x_j | class)
    return X @ np.log(p) + (1 - X) @ np.log(1 - p)

log_post_spam = np.log(prior_hat) + log_lik(X, p_spam_hat)
log_post_ham = np.log(1 - prior_hat) + log_lik(X, p_ham_hat)
y_hat = log_post_spam > log_post_ham
print("accuracy:", (y_hat == y).mean()) # ~0.9
```

```
# --- 5. A posterior probability, not just a decision ---
m = np.maximum(log_post_spam, log_post_ham) # for numerical stability
post = np.exp(log_post_spam - m) / (
    np.exp(log_post_spam - m) + np.exp(log_post_ham - m))
print(post[:3]) # P(spam | features), per email
```

Checkpoints (do all of these).

1. `prior_hat`, `p_spam_hat`, `p_ham_hat` land near the true parameters — counting *is* estimation, and the $1/\sqrt{n}$ error is Lesson 24’s law of large numbers in action. Re-run with `n = 500` and watch the estimates (and accuracy) degrade.
2. Step 4 is Example 18.6 vectorized: the log of the product $\mathbf{P}(C) \prod_j \mathbf{P}(F_j | C)$ becomes a sum, and the sum is a matrix–vector product (Lesson 4). Verify one email by hand against the code’s answer.
3. Why logs? Multiply 100 probabilities of size ~ 0.1 and you underflow float64. Confirm: `np.prod(np.full(400, 0.1))` is 0.0, but the log-sum is finite.
4. The decision rule compares $\log \mathbf{P}(C = 1) + \sum_j \log \mathbf{P}(F_j | C = 1)$ against the $C = 0$ version: a *linear* function of the feature vector! Compute the implied weight vector w and bias, and check `y_hat = (X@w + b > 0)` agrees. Naive Bayes is a linear classifier (Part I, Lesson 2’s score $w \cdot \phi(x)$) whose weights were set by counting instead of gradient descent.
5. Break the model: regenerate the data with feature 2 *copied* from feature 1 (perfectly dependent), retrain, and watch accuracy drop as the conditional-independence assumption double-counts evidence. This failure mode is why CS221 discusses when naive Bayes is and isn’t appropriate.

If every checkpoint makes sense, you are ready for CS221’s probability-flavored units — Bayesian networks, HMMs, and the probabilistic view of learning.

Final self-check list for Part III

You should be able to, from memory: state the three axioms and derive the complement and union-bound rules; define conditional probability and prove total probability and Bayes’ rule; work a false-positive-style Bayes problem numerically; define PMF, expectation, variance and compute them for Bernoulli, binomial, geometric, Poisson; state and prove linearity of expectation and use indicators; manipulate joint/marginal/conditional PMFs and prove the total expectation theorem; define PDF and CDF, standardize a normal, and run the CDF-then-differentiate recipe; implement naive Bayes with logs in NumPy. That is B&T Chapters 1–3 at working strength — the probability CS221 assumes on day one.

Part IV: Toward EE278

EE278 (*Probability and Statistical Inference*) opens where Part III stops and moves fast: random vectors, convergence and limit theorems, random processes (IID, Markov, Gaussian), stationarity and autocorrelation, MMSE and linear estimation, Kalman filtering, hypothesis testing, PCA. This part is the on-ramp: B&T Chapters 4–8, with the linear-algebra connections (Lessons 6, 11, 13) made explicit, because in EE278 probability and linear algebra stop being separate subjects. *None of this part is required for CS221* except as noted for Lesson 26 (Markov chains, which make CS221’s MDP and HMM lectures feel like review).

21 Derived Distributions, Covariance, and Correlation

NEEDED FOR: EE278. Not needed for CS221.

21.1 Theory

Theorem 21.1 (Derived distributions: the CDF method and the monotone formula). *Let X be continuous with PDF f_X and let $Y = g(X)$.*

(a) *Always: $F_Y(y) = \mathbf{P}(g(X) \leq y)$, computed as an integral of f_X over $\{x : g(x) \leq y\}$; differentiate to get f_Y .*

(b) *If g is strictly monotone and differentiable with inverse h (so $X = h(Y)$), then*

$$f_Y(y) = f_X(h(y)) |h'(y)|.$$

Proof. (a) is the definition of the CDF plus the definition of a PDF. For (b) with g increasing:

$$F_Y(y) = \mathbf{P}(g(X) \leq y) = \mathbf{P}(X \leq h(y)) = F_X(h(y)),$$

and the chain rule gives $f_Y(y) = f_X(h(y)) h'(y)$, with $h' > 0$. For g decreasing the event flips to $\{X \geq h(y)\}$, so $F_Y(y) = 1 - F_X(h(y))$ and $f_Y(y) = -f_X(h(y))h'(y)$ with $h' < 0$; both cases are covered by $|h'(y)|$. $\square$

The factor $|h'(y)|$ is one-dimensional volume scaling — the same role $|\det A|$ played in Lesson 14’s change of variables, and the preview of the multivariate formula (Jacobian determinant) EE278 uses. Proposition 19.2 was this theorem applied to $g(x) = ax + b$.

Definition 21.2 (Covariance, correlation). For random variables X, Y with means μ_X, μ_Y :

$$\text{cov}(X, Y) = \mathbf{E}[(X - \mu_X)(Y - \mu_Y)] = \mathbf{E}[XY] - \mu_X\mu_Y,$$

(the second form by expanding and linearity), and the *correlation coefficient* is

$$\rho(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_X\sigma_Y} \quad (\sigma_X, \sigma_Y > 0).$$

X, Y are *uncorrelated* if $\text{cov}(X, Y) = 0$.

Theorem 21.3 (Covariance is an inner product; $|\rho| \leq 1$). *On the space of zero-mean random variables (with finite variance), define $\langle X, Y \rangle = \mathbf{E}[XY]$. This satisfies the three inner-product properties of Lesson 2 (positivity — with $\langle X, X \rangle = 0$ forcing $X = 0$ with probability 1 — symmetry, and linearity in each slot). Consequently, by the Cauchy–Schwarz proof of Theorem 2.5, transcribed verbatim:*

$$|\mathbf{E}[XY]| \leq \sqrt{\mathbf{E}[X^2]} \sqrt{\mathbf{E}[Y^2]},$$

i.e. $|\text{cov}(X, Y)| \leq \sigma_X \sigma_Y$, i.e. $-1 \leq \rho \leq 1$, with $|\rho| = 1$ exactly when Y is (with probability 1) a scalar multiple of X .

Proof. Positivity: $\mathbf{E}[X^2] \geq 0$ since $X^2 \geq 0$; and $\mathbf{E}[X^2] = 0$ forces $\mathbf{P}(|X| > \epsilon) = 0$ for every $\epsilon > 0$ (else the expectation would exceed $\epsilon^2 \mathbf{P}(|X| > \epsilon) > 0$), so $X = 0$ with probability 1. Symmetry is $XY = YX$; linearity in each slot is linearity of expectation (Theorem 17.4). With the axioms verified, Lesson 2’s proof of Cauchy–Schwarz applies word for word — it used nothing but the axioms. Applying it to the centered variables $X - \mu_X$, $Y - \mu_Y$ gives the covariance statement, and the equality case (one variable a multiple of the other) translates to $Y - \mu_Y = c(X - \mu_X)$: a perfect linear relationship. $\square$

This theorem is the hinge of Part IV: *random variables form an inner product space*, in which uncorrelated = orthogonal, standard deviation = norm, and $\rho = \cos \theta$. Lesson 22 will cash this in: best estimation becomes orthogonal projection.

Proposition 21.4 (Variance of a sum, in general).

$$\text{var}(X + Y) = \text{var}(X) + \text{var}(Y) + 2\text{cov}(X, Y),$$

and independent $\Rightarrow$ uncorrelated (by Theorem 18.4), but not conversely.

Proof. Expand $\mathbf{E}[(\tilde{X} + \tilde{Y})^2]$ with $\tilde{X}, \tilde{Y}$ centered, as in Theorem 18.4; the cross term is now $2\text{cov}(X, Y)$, kept rather than killed. For the non-converse, one example suffices: let X be uniform on $\{-1, 0, 1\}$ and $Y = X^2$. Then $\mathbf{E}[XY] = \mathbf{E}[X^3] = 0 = \mathbf{E}[X]\mathbf{E}[Y]$: uncorrelated. But $\mathbf{P}(X = 0, Y = 0) = \frac{1}{3} \neq \frac{1}{3} \cdot \frac{1}{3}$: dependent — Y is a deterministic function of X ! Covariance detects only *linear* relationships. $\square$

21.2 Practice: by hand

Example 21.5 (Derived distribution, start to finish). $X \sim \text{Uniform}(0, 1)$; find the PDF of $Y = -\ln X$. The inverse of $g(x) = -\ln x$ is $h(y) = e^{-y}$, with $|h'(y)| = e^{-y}$, so for $y > 0$:

$$f_Y(y) = f_X(e^{-y}) \cdot e^{-y} = 1 \cdot e^{-y} = e^{-y} :$$

exponential(1). This is how computers generate exponential random variables from uniform ones (*inverse transform sampling*) — Capstone IV uses it.

Example 21.6 (Covariance from a joint table). For the joint PMF of Example 18.5: $\mathbf{E}[X] = 0.6$, $\mathbf{E}[Y] = 0.3 \cdot 0 + 0.3 \cdot 1 + 0.4 \cdot 2 = 1.1$, and

$$\mathbf{E}[XY] = \sum_{x,y} xy p_{X,Y}(x, y) = 1 \cdot 1 \cdot 0.1 + 1 \cdot 2 \cdot 0.3 = 0.7,$$

so $\text{cov}(X, Y) = 0.7 - 0.6 \times 1.1 = 0.04 > 0$: weakly positively associated, matching Lesson 18’s observation that seeing $Y = 2$ raised the estimate of X . With $\text{var}(X) = 0.24$ and $\text{var}(Y) = 0.69$: $\rho = 0.04 / \sqrt{0.24 \times 0.69} \approx 0.098$.

EE278 connection. Replace “two random variables X, Y ” by “one random signal sampled at two times X_t, X_s ” and covariance becomes the *autocorrelation function* $R_X(t, s) = \mathbf{E}[X_t X_s]$ — EE278’s central object, whose Fourier transform (for stationary processes) is the power spectral density. Every property proved here ($|R| \leq \sigma^2$ via Cauchy–Schwarz, bilinearity) is quoted there in the first two weeks.

21.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

# Inverse transform sampling: uniform -> exponential
u = rng.random(1_000_000)
y = -np.log(u)
print(y.mean(), y.var())      # ~1, ~1 (exponential(1))

# Covariance and correlation, empirically:
x = rng.normal(size=100_000)
z = 0.6 * x + 0.8 * rng.normal(size=100_000)  # corr 0.6 by design
C = np.cov(x, z)                        # 2x2 covariance matrix
print(np.corrcoef(x, z)[0, 1])  # ~0.6

# Uncorrelated but dependent:
x3 = rng.choice([-1., 0., 1.], size=100_000)
print(np.corrcoef(x3, x3**2)[0, 1])  # ~0 despite Y = X^2
```

`np.cov` returns the 2×2 matrix $\begin{pmatrix} \text{var}(X) & \text{cov}(X, Z) \\ \text{cov}(Z, X) & \text{var}(Z) \end{pmatrix}$ — the object Lesson 23 studies in n dimensions. Estimating a covariance from n samples costs $O(n)$ and carries $O(1/\sqrt{n})$ error, like every Monte Carlo average.

21.4 Exercises

Exercise 21.1 (Hand). $X \sim \text{Uniform}(0, 1)$. Find the PDF of (a) $Y = X^2$ (CDF method — careful, g is monotone here, so check both routes agree) and (b) $Y = 3X + 1$ (monotone formula).

Exercise 21.2 (Hand). Two fair dice, $S = \text{sum}$, $D = \text{difference}$ (first minus second). Compute $\text{cov}(S, D)$. (Slick route: $S = X + Y$, $D = X - Y$; expand by bilinearity of covariance.) Are S and D independent? (Check a corner case, e.g. $S = 12$ vs $D = 5$.)

Exercise 21.3 (Proof, ★). Prove the bilinearity of covariance,

$$\text{cov}\left(\sum_i a_i X_i, \sum_j b_j Y_j\right) = \sum_i \sum_j a_i b_j \text{cov}(X_i, Y_j),$$

and deduce the general variance-of-a-sum formula

$$\text{var}\left(\sum_i X_i\right) = \sum_i \text{var}(X_i) + 2 \sum_{i < j} \text{cov}(X_i, X_j).$$

Exercise 21.4 (Proof). Let $Y = aX + b$ with $a \neq 0$. Show $\rho(X, Y) = \text{sign}(a)$, confirming $|\rho| = 1$ iff the relationship is exactly linear, with the sign recording direction.

Exercise 21.5 (NumPy). The hat problem of Example 17.8 has $\text{var}(X) = 1$: verify by simulation, then explain it with the general variance-of-a-sum formula. ($\text{var}(X_i) = \frac{1}{n}(1 - \frac{1}{n})$; $\text{cov}(X_i, X_j) = \frac{1}{n(n-1)} - \frac{1}{n^2}$ for $i \neq j$ — derive these, sum, and watch everything cancel to 1 as n grows.)

Deeper reading. B&T 4.1 (derived distributions), 4.2 (covariance and correlation); B&T 4.4–4.5 (transforms and random sums — worth a skim: EE278 uses moment generating functions in its concentration-inequality unit).

22 Conditional Expectation and Least Mean Squares

NEEDED FOR: EE278 (core: MMSE, linear estimation, Kalman lead-in). Not needed for CS221.

22.1 Theory

Lesson 18 defined the *number* $\mathbf{E}[X | Y = y]$. The upgrade that powers estimation theory is to see it as a *random variable*: define $\mathbf{E}[X | Y]$ to be the random variable that takes the value $\mathbf{E}[X | Y = y]$ when $Y = y$ — a function of Y .

Theorem 22.1 (Law of iterated expectations). $\mathbf{E}[\mathbf{E}[X | Y]] = \mathbf{E}[X]$.

Proof. $\mathbf{E}[X | Y]$ is a function of Y , say $g(Y)$ with $g(y) = \mathbf{E}[X | Y = y]$. By LOTUS and then the total expectation theorem (Theorem 18.3):

$$\mathbf{E}[g(Y)] = \sum_y g(y) p_Y(y) = \sum_y \mathbf{E}[X | Y = y] p_Y(y) = \mathbf{E}[X].$$

(Continuous case: same lines with integrals.) □

Theorem 22.2 (Conditional expectation is the least-mean-squares estimator). *Among all functions g , the mean squared error $\mathbf{E}[(X - g(Y))^2]$ is minimized by $g(Y) = \mathbf{E}[X | Y]$.*

Proof. Fix y and work in the conditional world given $Y = y$ (a legitimate probability law, Lesson 16). There, $g(y)$ is just a constant c , and Exercise 17.3 proved $\mathbf{E}[(X - c)^2 | Y = y]$ is minimized uniquely by $c = \mathbf{E}[X | Y = y]$. So for every y ,

$$\mathbf{E}[(X - g(Y))^2 | Y = y] \geq \mathbf{E}[(X - \mathbf{E}[X | Y = y])^2 | Y = y],$$

and averaging both sides over y (total expectation, Theorem 18.3) gives $\mathbf{E}[(X - g(Y))^2] \geq \mathbf{E}[(X - \mathbf{E}[X | Y])^2]$. □

Theorem 22.3 (Orthogonality principle). *Let $\tilde{X} = X - \mathbf{E}[X | Y]$ be the estimation error. Then for every function h ,*

$$\mathbf{E}[\tilde{X} h(Y)] = 0 :$$

the error is uncorrelated with every function of the data. In the inner product of Theorem 21.3, the error is orthogonal to the space of estimators, so $\mathbf{E}[X | Y]$ is the orthogonal projection of X onto the space of functions of Y .

Proof. Condition on Y and iterate (Theorem 22.1):

$$\mathbf{E}[\tilde{X}h(Y)] = \mathbf{E}\left[\mathbf{E}[\tilde{X}h(Y) \mid Y]\right] = \mathbf{E}\left[h(Y) \mathbf{E}[\tilde{X} \mid Y]\right],$$

where $h(Y)$ slides out of the inner conditional expectation because, given Y , it is a constant. And $\mathbf{E}[\tilde{X} \mid Y] = \mathbf{E}[X \mid Y] - \mathbf{E}[X] = 0$. $\square$

This is Lesson 6, re-lived: Theorem 6.3 said the projection is the closest point and its residual is orthogonal to the subspace; Theorems 22.2 and 22.3 say it again with $\|\cdot\|^2 = \mathbf{E}[(\cdot)^2]$. Same geometry, new inner product.

Linear least mean squares. $\mathbf{E}[X \mid Y]$ can be a complicated nonlinear function, and computing it needs the full joint distribution. EE278's workhorse compromise: project onto the smaller subspace of *linear* (affine) estimators $\hat{X} = aY + b$ — computable from means, variances, and one covariance alone.

Theorem 22.4 (Linear LMS). *The minimizers of $\mathbf{E}[(X - aY - b)^2]$ are*

$$a = \frac{\text{cov}(X, Y)}{\text{var}(Y)}, \quad b = \mu_X - a\mu_Y,$$

so the best linear estimator and its error variance are

$$\hat{X}_L = \mu_X + \frac{\text{cov}(X, Y)}{\text{var}(Y)}(Y - \mu_Y), \quad \mathbf{E}[(X - \hat{X}_L)^2] = (1 - \rho^2)\sigma_X^2.$$

Proof. For fixed a , the best constant shift is $b = \mathbf{E}[X - aY] = \mu_X - a\mu_Y$ (Exercise 17.3 again), leaving $\mathbf{E}[(\tilde{X} - a\tilde{Y})^2]$ with centered variables. Expand:

$$J(a) = \sigma_X^2 - 2a \text{cov}(X, Y) + a^2 \sigma_Y^2,$$

a parabola in a minimized at $a = \text{cov}(X, Y)/\sigma_Y^2$ (set $J'(a) = 0$; $J'' = 2\sigma_Y^2 > 0$). Substituting back:

$$J(a^*) = \sigma_X^2 - \frac{\text{cov}(X, Y)^2}{\sigma_Y^2} = \sigma_X^2 \left(1 - \frac{\text{cov}(X, Y)^2}{\sigma_X^2 \sigma_Y^2}\right) = (1 - \rho^2)\sigma_X^2. \quad \square$$

Read the error formula as a scoreboard for ρ : correlation is exactly the fraction of standard deviation that linear prediction can remove (ρ^2 is the “variance explained”). And notice: Theorem 22.4 is Lesson 6's least-squares projection onto the two-“vector” span $\{1, Y\}$, with the normal equations $\mathbf{E}[(X - aY - b) \cdot 1] = 0$, $\mathbf{E}[(X - aY - b) \cdot Y] = 0$ — residual orthogonal to each spanning vector.

22.2 Practice: by hand

Example 22.5 (Nonlinear vs. linear estimator). Let $Y \sim \text{Uniform}(-1, 1)$ and $X = Y^2$ (no noise!). The best estimator is $\mathbf{E}[X \mid Y] = Y^2$: zero error. The best *linear* estimator: $\text{cov}(X, Y) = \mathbf{E}[Y^3] - \mathbf{E}[Y^2]\mathbf{E}[Y] = 0$, so $a = 0$ and $\hat{X}_L = \mu_X = \mathbf{E}[Y^2] = \frac{1}{3}$ — a constant, with error variance $\text{var}(Y^2) = \mathbf{E}[Y^4] - \frac{1}{9} = \frac{1}{5} - \frac{1}{9} = \frac{4}{45}$. Linear estimation is blind to this perfectly predictable X because the dependence is purely nonlinear ($\rho = 0$). Moral: LLMS is powerful exactly when correlation captures the relationship — which Lesson 23 shows is always the case for Gaussians.

Example 22.6 (Signal plus noise, the EE278 primal example). X (signal) has mean 0, variance σ_X^2 ; N (noise) has mean 0, variance σ_N^2 , uncorrelated with X ; observe $Y = X + N$. Then $\text{cov}(X, Y) = \sigma_X^2$ and $\text{var}(Y) = \sigma_X^2 + \sigma_N^2$, so

$$\hat{X}_L = \frac{\sigma_X^2}{\sigma_X^2 + \sigma_N^2} Y, \quad \text{error variance} = \frac{\sigma_X^2 \sigma_N^2}{\sigma_X^2 + \sigma_N^2}.$$

The estimator shrinks the observation by the *signal-to-noise ratio*: trust Y when the signal dominates, shrink toward the prior mean 0 when the noise does. With $\sigma_X^2 = \sigma_N^2 = 1$ this is $\hat{X} = Y/2$ — exactly the posterior mean of Example 19.5, where everything was Gaussian and the linear estimator was fully optimal. Kalman filtering (EE278) is this formula applied recursively as observations stream in.

EE278 connection. EE278’s estimation unit is these two theorems at scale: MMSE = conditional expectation; LLMS = projection using covariances; the orthogonality principle is the certificate for both, and for the Wiener and Kalman filters built from them. The habit to carry in: *when someone says “best estimate,” ask “projection onto what subspace, in which inner product?”*

22.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

# Signal plus noise: X ~ N(0,1), Y = X + N, N ~ N(0,1)
n = 1_000_000
X = rng.normal(size=n)
Y = X + rng.normal(size=n)

a = np.cov(X, Y)[0, 1] / Y.var()      # ~0.5, Theorem 22.5
Xhat = a * Y
print(((X - Xhat)**2).mean())          # ~0.5 = error variance

# Orthogonality principle, empirically: error uncorrelated with
# Y, Y^2, sin(Y) -- any function of the data:
err = X - Xhat
for h in (Y, Y**2, np.sin(Y)):
    print(np.corrcoef(err, h)[0, 1]) # all ~0 (Y exactly; others: Gaussian)

# Nonlinear example: X = Y^2 -- linear fails, conditional mean wins
Y2 = rng.uniform(-1, 1, size=n); X2 = Y2**2
print(np.cov(X2, Y2)[0, 1])          # ~0: LLMS is the constant 1/3
```

Estimating the LLMS coefficients from data is exactly fitting least squares on samples (Capstone I!) — `np.polyfit(Y, X, 1)` returns (a, b) . That closes a loop: Lesson 6 projected *data vectors* in $\mathbb{R}^m$; this lesson projects *random variables*; the sample version of the second is the first.

22.4 Exercises

Exercise 22.1 (Hand). For the joint table of Example 18.5: compute $\mathbf{E}[X | Y]$ (a value for each y), verify the law of iterated expectations numerically, and find the LLMS estimator $\hat{X}_L = aY + b$ from the moments computed in Lesson 21. Compare its MSE with that of $\mathbf{E}[X | Y]$.

Exercise 22.2 (Hand). In Example 22.6 with $\sigma_X = 2, \sigma_N = 1$: compute $\hat{X}_L$ as a function of Y and the error variance. What fraction of the signal's variance does the observation remove?

Exercise 22.3 (Proof, ★). (Law of total variance.) Define $\text{var}(X | Y)$ as the variance computed in the conditional world, a function of Y . Prove

$$\text{var}(X) = \mathbf{E}[\text{var}(X | Y)] + \text{var}(\mathbf{E}[X | Y]),$$

by applying $\text{var} = \mathbf{E}[\square^2] - (\mathbf{E}[\square])^2$ twice and the law of iterated expectations. Interpret: total uncertainty = average leftover uncertainty + uncertainty explained by the data.

Exercise 22.4 (Proof). Derive Theorem 22.4 from the orthogonality principle instead of calculus: require $\mathbf{E}[(X - aY - b) \cdot 1] = 0$ and $\mathbf{E}[(X - aY - b) \cdot Y] = 0$ and solve the two linear equations. (These are normal equations, Lesson 6 — with $\mathbf{E}[\cdot \cdot]$ as the inner product.)

Exercise 22.5 (NumPy). Let $Y \sim \mathcal{N}(0, 1)$ and $X = Y^3 + 0.1\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$ independent. Estimate from 10^6 samples: the LLMS line, its MSE, and the MSE of the binned conditional mean (average X within narrow Y -bins — an empirical $\mathbf{E}[X | Y]$). How much better is the nonlinear estimator? What is ρ here?

Deeper reading. B&T 4.3 (conditional expectation and variance revisited — iterated expectations, total variance); B&T 8.3–8.4 (Bayesian least mean squares and *linear* LMS estimation — the two theorems above in their inference setting); then EE278's linear-estimation notes will read as review.

23 Random Vectors and Gaussian Vectors

NEEDED FOR: EE278 (core: PCA, Gaussian processes, Kalman). Uses Lessons 11 & 13. Not needed for CS221.

23.1 Theory

Definition 23.1 (Random vector, mean, covariance matrix). A *random vector* $X = (X_1, \dots, X_n)$ is a vector of random variables on one experiment. Its *mean vector* is $\mu = \mathbf{E}[X] = (\mathbf{E}[X_1], \dots, \mathbf{E}[X_n])$, and its *covariance matrix* is the $n \times n$ matrix

$$\Sigma = \mathbf{E}[(X - \mu)(X - \mu)^\top], \quad \Sigma_{ij} = \text{cov}(X_i, X_j),$$

with variances on the diagonal.

Proposition 23.2 (Covariance matrices transform like quadratic forms). *For any (deterministic) $m \times n$ matrix A and $b \in \mathbb{R}^m$, the random vector $AX + b$ has mean $A\mu + b$ and covariance matrix $A\Sigma A^\top$. In particular, for a single linear functional $w \in \mathbb{R}^n$:*

$$\text{var}(w^\top X) = w^\top \Sigma w.$$

Proof. The mean is linearity of expectation, coordinate by coordinate. For the covariance, let $\tilde{X} = X - \mu$; then $AX + b$ centered is $A\tilde{X}$, and

$$\mathbf{E}[(A\tilde{X})(A\tilde{X})^\top] = \mathbf{E}[A\tilde{X}\tilde{X}^\top A^\top] = A\mathbf{E}[\tilde{X}\tilde{X}^\top]A^\top = A\Sigma A^\top,$$

pulling the constant matrices through the (entrywise) expectation. The scalar case is $w^\top$ as a $1 \times n$ matrix. $\square$

Theorem 23.3 (Covariance matrices are exactly the PSD matrices). *Every covariance matrix is symmetric positive semidefinite. Conversely, every symmetric PSD matrix is the covariance matrix of some random vector.*

Proof. Symmetry: $\text{cov}(X_i, X_j) = \text{cov}(X_j, X_i)$. PSD: for any $w \in \mathbb{R}^n$, $w^\top \Sigma w = \text{var}(w^\top X) \geq 0$ by Proposition 23.2. Conversely, given symmetric PSD Σ , the spectral theorem (Theorem 11.4) gives $\Sigma = Q\Lambda Q^\top$ with $\Lambda = \text{diag}(\lambda_i)$, $\lambda_i \geq 0$; set $\Sigma^{1/2} = Q\Lambda^{1/2}Q^\top$ (a symmetric matrix with $\Sigma^{1/2}\Sigma^{1/2} = \Sigma$). Take Z with independent, zero-mean, unit-variance coordinates — so $\text{cov}(Z) = I$ — and set $X = \Sigma^{1/2}Z$: by Proposition 23.2, $\text{cov}(X) = \Sigma^{1/2}I\Sigma^{1/2} = \Sigma$. $\square$

Lesson 11's abstract PSD theory just became concrete: *every* covariance matrix has an orthonormal eigenbasis with nonnegative eigenvalues, the eigenvectors are the *principal components* (directions of uncorrelated variation), and the eigenvalues are the variances along them. Changing to the eigenbasis ($Y = Q^\top \tilde{X}$, which has covariance $Q^\top \Sigma Q = \Lambda$, diagonal) *decorrelates* the vector — that transformation is PCA, and the top- k eigenvalue truncation is Lesson 13's Eckart–Young story applied to data.

Definition 23.4 (Gaussian random vector). X is a *Gaussian (normal) random vector*, $X \sim \mathcal{N}(\mu, \Sigma)$, if it can be written $X = \mu + BZ$ where Z has independent $\mathcal{N}(0, 1)$ coordinates and B is any matrix with $BB^\top = \Sigma$. Equivalently (the equivalence is proved in EE278 via characteristic functions; we adopt the constructive definition): every linear combination $w^\top X$ is a (one-dimensional) normal random variable. For invertible Σ , the density is

$$f_X(x) = \frac{1}{(2\pi)^{n/2}(\det \Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right).$$

The $\sqrt{\det \Sigma}$ is Lesson 14's volume factor (the map B stretches the unit Gaussian ball into the ellipsoid of the distribution), and the level sets of f_X are the ellipsoids $x^\top \Sigma^{-1}x = \text{const}$ whose axes are — again — the eigenvectors of Σ (Lesson 11's quadratic-form picture).

Theorem 23.5 (The Gaussian miracles). *Let $X \sim \mathcal{N}(\mu, \Sigma)$.*

- (a) Linearity: $AX + b \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$ — *Gaussians stay Gaussian under every linear map.*
- (b) Uncorrelated $\Rightarrow$ independent: *if two coordinates (or blocks) of a jointly Gaussian vector are uncorrelated, they are independent.*

Proof. (a) Write $X = \mu + BZ$ with $BB^\top = \Sigma$; then $AX + b = (A\mu + b) + (AB)Z$ with $(AB)(AB)^\top = A\Sigma A^\top$ — literally the definition, plus Proposition 23.2 for the parameters.

(b) In the density case: if Σ is block-diagonal $\begin{pmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{pmatrix}$, then Σ^{-1} is block-diagonal too, $\det \Sigma = \det \Sigma_1 \det \Sigma_2$ (Lesson 14 on the block-triangular form), and the exponent splits as a sum of a term in x_1 and a term in x_2 — so f_X *factors* into the two marginal Gaussian densities, which is independence. $\square$

Part (b) is a miracle worth staring at: Proposition 21.4 warned that uncorrelated is far weaker than independent (recall X and X^2) — *except* in the jointly Gaussian world, where covariance captures all dependence. This is also why Lesson 22’s *linear* LMS estimator is fully optimal for Gaussians (Example 22.6): no nonlinear function can see structure that covariances don’t already report.

23.2 Practice: by hand

Example 23.6 (A 2-D Gaussian, fully worked). Let $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $\mu = 0$ — Example 10.7’s matrix as a covariance. Eigen-decomposition (known): eigenvalues 3, 1 with orthonormal eigenvectors $\frac{1}{\sqrt{2}}(1, 1)$, $\frac{1}{\sqrt{2}}(1, -1)$. Read off:

- The density’s ellipses have long axis along $(1, 1)$ (variance 3) and short axis along $(1, -1)$ (variance 1): the cloud is a tilted ellipse.
- PCA: the first principal component is $(1, 1)/\sqrt{2}$, “explaining” $3/(3 + 1) = 75\%$ of the total variance $\text{tr } \Sigma = 4$ (trace = sum of eigenvalues, Lesson 14).
- Whitening: $Y = \Lambda^{-1/2} Q^T X$ has covariance I (check with Proposition 23.2) — rotate to the eigenbasis, divide each axis by its standard deviation.
- $\text{var}(X_1 - X_2) = w^T \Sigma w$ with $w = (1, -1)$: $2 + 2 - 2 \cdot 1 = 2$ — the quadratic form doing what Lesson 11 promised.

EE278 connection. A *Gaussian process* is this lesson with infinitely many coordinates: any finite set of samples $(X_{t_1}, \dots, X_{t_n})$ is a Gaussian vector, so the whole process is specified by a mean function and a covariance (autocorrelation) function. Kalman filtering is Theorem 23.5(a) plus Lesson 22’s estimator, iterated. PCA of data is Theorem 23.3’s eigen-story with Σ estimated from samples. When EE278 says “jointly Gaussian, hence independent given uncorrelated,” that is miracle (b).

23.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

mu = np.zeros(2)
Sigma = np.array([[2., 1.], [1., 2.]])

# Sample by the constructive definition X = mu + B Z:
lam, Q = np.linalg.eigh(Sigma)           # Lesson 11
B = Q @ np.diag(np.sqrt(lam)) @ Q.T      # Sigma^{1/2}
Z = rng.normal(size=(1_000_000, 2))
X = Z @ B.T + mu                         # (built-in: rng.multivariate_normal)

print(np.cov(X.T))                       # ~Sigma

# PCA on the samples: eigenvectors of the empirical covariance
```

```

lam_hat, Q_hat = np.linalg.eigh(np.cov(X.T))
print(lam_hat)                                # ~[1, 3]
print(lam_hat[1] / lam_hat.sum())             # ~0.75 variance explained

# Whitening, and miracle (b): decorrelated Gaussians are independent
Y = (X - mu) @ Q @ np.diag(1/np.sqrt(lam))
print(np.cov(Y.T))                           # ~I
# empirical check of independence:  $P(Y_1 > 1, Y_2 > 1) \sim P(Y_1 > 1) P(Y_2 > 1)$ 
print((( Y[:,0]>1) & (Y[:,1]>1)).mean(), (Y[:,0]>1).mean()*2)

```

Sampling costs $O(n^3)$ once for the factorization, then $O(n^2)$ per draw. In practice the factor B is computed by Cholesky (`np.linalg.cholesky`, $O(n^3)$ but a third of `eigh`'s constant); the eigen-route shown keeps the PCA story visible.

23.4 Exercises

Exercise 23.1 (Hand). Let X_1, X_2 be uncorrelated with variances 1, 4, and $Y_1 = X_1 + X_2$, $Y_2 = X_1 - X_2$. Write the matrix A with $Y = AX$, compute $\text{cov}(Y) = A\Sigma A^\top$, and read off $\text{var}(Y_1)$, $\text{var}(Y_2)$, $\text{cov}(Y_1, Y_2)$. When would Y_1, Y_2 be uncorrelated?

Exercise 23.2 (Hand). For $\Sigma = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$: find eigenvalues and orthonormal eigenvectors, the fraction of variance along the first principal component, and the whitening transform. Sketch (in words) the density's ellipses.

Exercise 23.3 (Proof, ★). Prove that $\Sigma^{1/2} = Q\Lambda^{1/2}Q^\top$ is the *unique* symmetric PSD square root of a symmetric PSD Σ . (Uniqueness: if $S^2 = \Sigma$ with S symmetric PSD, show S and Σ have the same eigenvectors — diagonalize S and square. LADR 7C proves this as the “positive operators have unique positive square roots” theorem.)

Exercise 23.4 (Proof). Using Theorem 23.5(a), show that if $X \sim \mathcal{N}(\mu, \Sigma)$ with Σ invertible, the whitened vector $\Lambda^{-1/2}Q^\top(X - \mu)$ is $\mathcal{N}(0, I)$ — i.e. its coordinates are independent standard normals. (This is why “every Gaussian is a linearly transformed standard one” can be run in both directions.)

Exercise 23.5 (NumPy). Generate 10^5 samples of a 5-D Gaussian whose covariance you build as $\Sigma = RR^\top + 0.1I$ for a random 5×2 matrix R (nearly rank-2!). Run PCA on the empirical covariance; confirm two eigenvalues dominate, and that projecting onto the top-2 eigenvectors preserves $\approx$ the fraction $(\lambda_1 + \lambda_2)/\sum \lambda_i$ of total variance. You have re-derived dimensionality reduction.

Deeper reading. B&T 4.2 (correlation matrices, in the covariance section); the bivariate normal supplement on the B&T website; LADR 7C (positive operators, square roots) for the linear-algebra half; EE278's random-vectors notes then formalize the characteristic-function view.

24 Limit Theorems

NEEDED FOR: EE278 (core: convergence, concentration); CS221 (background: why Monte Carlo works)

24.1 Theory

Throughout, $X_1, X_2, \dots$ are independent identically distributed (i.i.d.) with mean μ and variance $\sigma^2 < \infty$, and

$$M_n = \frac{X_1 + \dots + X_n}{n}$$

is the *sample mean*. By linearity and independence: $\mathbf{E}[M_n] = \mu$ and $\text{var}(M_n) = \sigma^2/n$ — averaging keeps the target and shrinks the spread. The limit theorems sharpen “shrinks” into guarantees.

Theorem 24.1 (Markov and Chebyshev inequalities). (a) If $X \geq 0$, then for any $a > 0$:

$$\mathbf{P}(X \geq a) \leq \frac{\mathbf{E}[X]}{a}.$$

(b) For any random variable Y with mean μ and variance σ^2 , and any $\epsilon > 0$:

$$\mathbf{P}(|Y - \mu| \geq \epsilon) \leq \frac{\sigma^2}{\epsilon^2}.$$

Proof. (a) Let I be the indicator of $\{X \geq a\}$. Then $aI \leq X$ always: if the event occurs, $aI = a \leq X$; if not, $aI = 0 \leq X$. Taking expectations (monotonicity of $\mathbf{E}$, which follows from nonnegativity of $\mathbf{E}$ applied to $X - aI$): $a\mathbf{P}(X \geq a) = \mathbf{E}[aI] \leq \mathbf{E}[X]$.

(b) Apply (a) to the nonnegative variable $(Y - \mu)^2$ with threshold ϵ^2 :

$$\mathbf{P}(|Y - \mu| \geq \epsilon) = \mathbf{P}((Y - \mu)^2 \geq \epsilon^2) \leq \frac{\mathbf{E}[(Y - \mu)^2]}{\epsilon^2} = \frac{\sigma^2}{\epsilon^2}. \quad \square$$

Theorem 24.2 (Weak law of large numbers). For every $\epsilon > 0$,

$$\mathbf{P}(|M_n - \mu| \geq \epsilon) \leq \frac{\sigma^2}{n\epsilon^2} \xrightarrow{n \rightarrow \infty} 0.$$

We say M_n converges in probability to μ .

Proof. Chebyshev applied to M_n , whose mean is μ and variance is σ^2/n . $\square$

Three remarks. (1) The bound is quantitative: to pin μ within ϵ with 95% confidence, $n = 20\sigma^2/\epsilon^2$ samples suffice — crude but honest, and the pattern (accuracy costs $1/\epsilon^2$) is right. (2) Chebyshev is distribution-free and correspondingly loose; EE278’s concentration unit (Chernoff/Hoeffding bounds, built from transforms, B&T 5.4’s sidebar and Chapter 4.4) trades generality for exponentially better bounds. (3) The *strong* law (B&T 5.5) upgrades the conclusion to $\mathbf{P}(M_n \rightarrow \mu) = 1$; the distinction between the two convergence modes is an EE278 lecture of its own.

Theorem 24.3 (Central limit theorem). Let $Z_n = \frac{X_1 + \dots + X_n - n\mu}{\sigma\sqrt{n}}$ (the sum, standardized to mean 0 and variance 1). Then for every z ,

$$\mathbf{P}(Z_n \leq z) \xrightarrow{n \rightarrow \infty} \Phi(z),$$

the standard normal CDF — regardless of the distribution of the X_i .

The proof (via transforms: the moment generating function of Z_n converges to the Gaussian's, B&T 5.4 sketches it) is the one omission in this part; the statement and its use are non-negotiable. The practical reading: for large n ,

$$X_1 + \cdots + X_n \approx \mathcal{N}(n\mu, n\sigma^2), \quad M_n \approx \mathcal{N}(\mu, \sigma^2/n),$$

which converts “how unlikely is this deviation?” into a Φ lookup (Lesson 19). The CLT is why the Gaussian owns Lesson 23 and EE278: sums of many small independent contributions — thermal noise being the canonical example — are Gaussian by theorem.

24.2 Practice: by hand

Example 24.4 (Polling). Poll n voters; each answer is Bernoulli(p), and M_n estimates p . Since $\sigma^2 = p(1-p) \leq \frac{1}{4}$: Chebyshev guarantees $\mathbf{P}(|M_n - p| \geq 0.03) \leq \frac{1/4}{n(0.03)^2}$, which is $\leq 5\%$ once $n \geq 5556$. The CLT sharpens this: $\mathbf{P}(|M_n - p| \geq 0.03) \approx 2(1 - \Phi(0.03\sqrt{n}/\sqrt{p(1-p)})) \leq 5\%$ once $0.03\sqrt{4n} \geq 1.96$, i.e. $n \geq 1068$ — the famous “ $\pm 3\%$ needs about a thousand people,” independent of the population size.

Example 24.5 (CLT for a die). Sum 100 fair-die rolls. $\mu = 3.5$, $\sigma^2 = \frac{35}{12} \approx 2.92$ per roll, so the sum is approximately $\mathcal{N}(350, 292)$ and

$$\mathbf{P}(\text{sum} > 380) \approx 1 - \Phi\left(\frac{380-350}{\sqrt{292}}\right) = 1 - \Phi(1.76) \approx 0.04.$$

An exact computation would need a 100-fold convolution; the CLT needs two moments.

EE278 / CS221 connection. EE278 devotes a unit to exactly this ladder — Markov $\rightarrow$ Chebyshev $\rightarrow$ Chernoff, then modes of convergence, then the CLT — and uses it to reason about detection error rates and generalization in learning. For CS221 the payoff is quieter but constant: every Monte Carlo estimate (particle filters, MCTS rollouts, sampled gradients in SGD) is a sample mean whose $\sigma/\sqrt{n}$ error bar and approximate normality come from these theorems.

24.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

# CLT visually: sums of a very non-Gaussian distribution
n, trials = 100, 100_000
draws = rng.exponential(size=(trials, n))      # skewed!
Z = (draws.sum(axis=1) - n) / np.sqrt(n)        # mu = sigma = 1
print(Z.mean(), Z.std())                       # ~0, ~1
print((Z <= 1.0).mean())                       # ~0.841 = Phi(1)

# LLN + the 1/sqrt(n) error law:
for n in (100, 10_000, 1_000_000):
    m = rng.exponential(size=n).mean()
    print(n, abs(m - 1))                       # shrinks ~ 1/sqrt(n)
```

```
# Chebyshev vs truth for the sample mean, n=100:
eps = 0.2
M = draws[:, :100].mean(axis=1)
print((np.abs(M - 1) >= eps).mean())           # actual: ~0.046
print(1 / (100 * eps**2))                       # Chebyshev bound: 0.25
```

The last comparison is the honest picture: Chebyshev’s 0.25 versus reality’s ≈ 0.05 (and the CLT’s prediction $2(1 - \Phi(2)) \approx 0.046$ — nearly exact). Bounds are for guarantees; the CLT is for estimates.

24.4 Exercises

Exercise 24.1 (Hand). A server handles requests with mean processing time 2 ms and standard deviation 1 ms (distribution unknown). Bound $\mathbf{P}(\text{total time for 400 requests} > 900 \text{ ms})$ two ways: Chebyshev, then CLT approximation. Compare.

Exercise 24.2 (Hand). How many fair-coin flips guarantee, via Chebyshev, that the observed fraction of heads is within 0.01 of $\frac{1}{2}$ with probability at least 0.99? What does the CLT suggest instead?

Exercise 24.3 (Proof, $\star$). Prove the “one-sided Chebyshev” warm-up: for any Y with mean 0 and variance σ^2 and any $a > 0$, $\mathbf{P}(Y \geq a) \leq \frac{\sigma^2}{\sigma^2 + a^2}$. (Hint: for any $t \geq 0$, $\{Y \geq a\} \subseteq \{(Y + t)^2 \geq (a + t)^2\}$; apply Markov and minimize over t .)

Exercise 24.4 (Proof). Show that convergence in probability to μ does *not* require $\text{var}(M_n) \rightarrow 0$ in general: construct Y_n with $Y_n \rightarrow 0$ in probability but $\text{var}(Y_n) \rightarrow \infty$. (Take $Y_n = n$ with probability $1/n^1$... adjust the exponent until it works, and say why.)

Exercise 24.5 (NumPy). Estimate π by Monte Carlo (fraction of uniform points in the unit square landing in the quarter disk, times 4). Plot the error versus n on log–log axes for $n = 10^2, \dots, 10^7$ and confirm the slope is $-\frac{1}{2}$. Add the CLT-predicted 95% band $\pm 1.96\sigma/\sqrt{n}$ (compute σ for the Bernoulli involved) and check the empirical error stays inside it about 95% of the time.

Deeper reading. B&T 5.1–5.4 (inequalities, WLLN, convergence in probability, CLT), 5.5 (strong law); B&T 4.4 (transforms) is the doorway to the Chernoff bounds EE278 adds on top.

25 The Bernoulli and Poisson Processes

NEEDED FOR: EE278 (first random processes; independent increments). Not needed for CS221.

25.1 Theory

A *random process* is a collection of random variables indexed by time. The two canonical arrival processes — coin flips in discrete time, and its continuous-time limit — are where EE278 starts building intuition for the general theory.

Definition 25.1 (Bernoulli process). A *Bernoulli process* with rate p is a sequence $X_1, X_2, \dots$ of i.i.d. Bernoulli(p) random variables: independent trials, each a success (“arrival”) with probability p .

Everything about it follows from Part III: the number of arrivals in n slots is $\text{Binomial}(n, p)$; the time to the first arrival is $\text{Geometric}(p)$; and the process is *memoryless* — whatever has happened through time n , the future $X_{n+1}, X_{n+2}, \dots$ is a fresh Bernoulli process, by independence. Consequently the *interarrival times* (gaps between successive arrivals) are i.i.d. $\text{Geometric}(p)$: after each arrival the process restarts.

Definition 25.2 (Poisson process). A *Poisson process* with rate $\lambda > 0$ counts arrivals in continuous time: with $N(t)$ = number of arrivals in $(0, t]$, the process satisfies

- (i) (*Poisson marginals*) $N(t) \sim \text{Poisson}(\lambda t)$: $\mathbf{P}(N(t) = k) = e^{-\lambda t}(\lambda t)^k / k!$;
- (ii) (*independent, stationary increments*) arrival counts in disjoint intervals are independent, and the count in an interval depends only on its length.

Theorem 25.3 (The Poisson process is the limit of Bernoulli processes). *Chop $(0, t]$ into n slots of width t/n and run a Bernoulli process with per-slot probability $p_n = \lambda t/n$. Then the number of successes $S_n \sim \text{Binomial}(n, p_n)$ satisfies, for each fixed k ,*

$$\mathbf{P}(S_n = k) \xrightarrow{n \rightarrow \infty} e^{-\lambda t} \frac{(\lambda t)^k}{k!}.$$

Proof. Write $a = \lambda t$. Then

$$\mathbf{P}(S_n = k) = \binom{n}{k} \left(\frac{a}{n}\right)^k \left(1 - \frac{a}{n}\right)^{n-k} = \frac{a^k}{k!} \cdot \underbrace{\frac{n(n-1) \cdots (n-k+1)}{n^k}}_{\rightarrow 1} \cdot \underbrace{\left(1 - \frac{a}{n}\right)^n}_{\rightarrow e^{-a}} \cdot \underbrace{\left(1 - \frac{a}{n}\right)^{-k}}_{\rightarrow 1},$$

using $(1 - a/n)^n \rightarrow e^{-a}$ and the fact that the first fraction is a product of k factors each tending to 1. $\square$

Theorem 25.4 (Interarrival times are exponential). *In a Poisson process of rate λ , the time T_1 to the first arrival is $\text{exponential}(\lambda)$, and successive interarrival times are i.i.d. $\text{exponential}(\lambda)$.*

Proof. $\mathbf{P}(T_1 > t) = \mathbf{P}(N(t) = 0) = e^{-\lambda t}$: exactly the exponential tail, so $T_1 \sim \text{exponential}(\lambda)$. For the later gaps: by independent and stationary increments, the process after any fixed time s is a fresh Poisson process independent of the past (this is the memorylessness of Lesson 19's exponential, now at the process level; the full argument that this also holds after the *random* time of an arrival is B&T 6.2), so each gap has the same exponential distribution, independently. $\square$

Two standard consequences, both provable in a few lines from the definitions (Exercises): *merging* two independent Poisson processes of rates λ_1, λ_2 yields a Poisson process of rate $\lambda_1 + \lambda_2$, and *splitting* a rate- λ process by an independent coin ($p / 1 - p$) yields two *independent* Poisson processes of rates $p\lambda, (1 - p)\lambda$.

25.2 Practice: by hand

Example 25.5 (A packet link). Packets arrive as a Poisson process at $\lambda = 3$ per second.

$$\mathbf{P}(\text{exactly 2 in one second}) = e^{-3} \frac{3^2}{2!} \approx 0.224, \quad \mathbf{P}(\text{none in 2 seconds}) = e^{-6} \approx 0.0025.$$

Expected arrivals in a minute: $\lambda t = 180$; and by Lesson 24's CLT (a Poisson is a sum of many thin-slice Bernoullis), the count over a minute is approximately $\mathcal{N}(180, 180)$ — Poisson variance equals its mean.

Example 25.6 (Merging and the memoryless race). Emails arrive at rate 2/hour, texts at rate 4/hour, independent Poisson. Interruptions of either kind: Poisson at rate 6/hour, so the wait for the next interruption is exponential(6) — mean 10 minutes. Given an interruption occurs, it is a text with probability 4/6: in a merged process, each arrival came from stream i with probability $\lambda_i / \sum_j \lambda_j$, independent of the past.

EE278 connection. These are your first *random processes*, and they carry the vocabulary EE278 runs on: independent increments, stationarity (increment distributions depend only on interval length — the first example of a stationarity property), memorylessness, and the interplay between the counting view $N(t)$ and the interarrival view $T_1, T_2, \dots$. Shot noise in electronics is modeled directly as a Poisson process; the Wiener process (Brownian motion) that EE278 introduces later is the same independent-stationary-increments idea with Gaussian increments in place of Poisson.

25.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

lam, T = 3.0, 1000.0

# Simulate by exponential gaps (Theorem 25.4):
gaps = rng.exponential(scale=1/lam, size=int(lam*T*2))
arrivals = np.cumsum(gaps)
arrivals = arrivals[arrivals < T]

# Check Poisson marginals: counts in unit intervals
counts = np.histogram(arrivals, bins=np.arange(0, T+1))[0]
print(counts.mean(), counts.var())      # ~3, ~3 (mean = variance)
print((counts == 2).mean())              # ~0.224 = e^-3 3^2/2!

# Independent increments: counts in disjoint intervals, correlation ~ 0
print(np.corrcoef(counts[:-1], counts[1:])[0, 1]) # ~0

# Splitting: thin each arrival with p=1/3; rate should be 1
keep = rng.random(arrivals.size) < 1/3
sub = np.histogram(arrivals[keep], bins=np.arange(0, T+1))[0]
print(sub.mean())                        # ~1
```

Simulating by cumulative exponential gaps is the standard method and is exactly Theorem 25.4 run forward. Note the two “fingerprints” checked: mean = variance (Poisson) and zero correlation across disjoint intervals (independent increments) — the same two checks EE278 problem sets ask for.

25.4 Exercises

Exercise 25.1 (Hand). A Bernoulli process has $p = 0.1$ per slot. Compute: the probability of no arrival in 10 slots; the expected time to the third arrival (sum of three geometrics); the

probability the first arrival is in slot 5 given none in the first 3 (memorylessness makes this instant).

Exercise 25.2 (Hand). Buses (rate 4/hour) and taxis (rate 8/hour) pass a corner as independent Poisson processes. You wait for either. Find: the mean wait; the probability the first three vehicles are all taxis; the probability exactly 2 buses pass in 30 minutes.

Exercise 25.3 (Proof, ★). Prove the merging theorem for marginals: if $N_1(t), N_2(t)$ are independent Poisson($\lambda_1 t$), Poisson($\lambda_2 t$), then $N_1(t) + N_2(t)$ is Poisson($(\lambda_1 + \lambda_2)t$). (Convolve the PMFs and recognize the binomial theorem inside the sum.)

Exercise 25.4 (Proof). For a Poisson process, show $\text{cov}(N(s), N(t)) = \lambda \min(s, t)$ for $s \leq t$ by writing $N(t) = N(s) + (N(t) - N(s))$ and using independent increments. (This “min” covariance reappears verbatim for the Wiener process in EE278.)

Exercise 25.5 (NumPy). Verify the splitting theorem’s independence claim: from one simulated rate-3 process, split with $p = \frac{1}{3}$; show the two sub-processes’ unit-interval counts have correlation ≈ 0 and each is Poisson with the right rate. Surprised? (Naively the two pieces “share” the same arrivals; conditioning on the split makes them independent.)

Deeper reading. B&T 6.1 (Bernoulli process), 6.2 (Poisson process — including the random-incidence paradox, a classic EE278 exam topic worth reading now).

26 Markov Chains

NEEDED FOR: EE278 (core: Markov processes); CS221 (helpful: the substrate of MDPs and HMMs). Uses Lesson 10.

26.1 Theory

Definition 26.1 (Discrete-time Markov chain). A *Markov chain* on states $\{1, \dots, m\}$ is a sequence $X_0, X_1, \dots$ such that the next state depends on the past only through the present (the *Markov property*):

$$\mathbf{P}(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = \mathbf{P}(X_{n+1} = j \mid X_n = i) = P_{ij},$$

where the *transition probabilities* P_{ij} form the $m \times m$ *transition matrix* P : each row is a PMF, $P_{ij} \geq 0$, $\sum_j P_{ij} = 1$.

Theorem 26.2 (Chapman–Kolmogorov: n -step transitions are matrix powers). Let $r_{ij}(n) = \mathbf{P}(X_n = j \mid X_0 = i)$. Then $r_{ij}(n) = (P^n)_{ij}$; and if the row vector $\pi^{(0)}$ is the PMF of X_0 , the PMF of X_n is $\pi^{(0)} P^n$.

Proof. Induct on n ; the case $n = 1$ is the definition. For the step, condition on the state at time n (total probability, Theorem 16.4(b)) and use the Markov property:

$$r_{ij}(n+1) = \sum_{k=1}^m \mathbf{P}(X_n = k \mid X_0 = i) \mathbf{P}(X_{n+1} = j \mid X_n = k) = \sum_k r_{ik}(n) P_{kj},$$

which is exactly the (i, j) entry of $P^n P = P^{n+1}$. The distribution statement is the same computation with the initial PMF weighting row i . $\square$

Probability question → Lesson 4 matrix multiplication; long-run behavior → Lesson 10 eigenvalues. That is the whole design.

Definition 26.3 (Stationary distribution). A row vector π with $\pi_j \geq 0$, $\sum_j \pi_j = 1$ is a *stationary distribution* if

$$\pi P = \pi \quad (\text{the balance equations } \pi_j = \sum_k \pi_k P_{kj}, \text{ plus normalization}).$$

Started from π , the chain keeps distribution π forever (Theorem 26.2).

$\pi P = \pi$ says: $\pi^\top$ is an eigenvector of $P^\top$ with eigenvalue 1 — Exercise 10.5’s “population matrix” was this in disguise. Eigenvalue 1 always exists (P has row sums 1, so $P\mathbf{1} = \mathbf{1}$; P and $P^\top$ have the same eigenvalues by Lesson 14, $\det(P - \lambda I) = \det(P^\top - \lambda I)$). The substantive question is convergence:

Theorem 26.4 (Steady-state convergence — statement). *Suppose the chain is irreducible (every state can reach every state) and aperiodic (no forced cycling; e.g. some state has $P_{ii} > 0$). Then there is a unique stationary π , with all $\pi_j > 0$, and for every starting state i :*

$$r_{ij}(n) \xrightarrow{n \rightarrow \infty} \pi_j.$$

Moreover π_j is the long-run fraction of time spent in state j .

B&T 7.3 proves this; the eigen-story explains it (for the diagonalizable case): the other eigenvalues of P have $|\lambda| < 1$ under these hypotheses, so in the eigen-expansion of $\pi^{(0)}P^n$ every component except the $\lambda = 1$ one dies geometrically — Lesson 10’s “ $|\lambda| < 1$ decays” as a theorem about forgetting the initial condition. The convergence *rate* is set by the second-largest $|\lambda|$ (the “spectral gap”), the number EE278 and MCMC practitioners both watch.

26.2 Practice: by hand

Example 26.5 (A two-state weather chain, end to end). Sunny (1) or rainy (2): $P = \begin{pmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{pmatrix}$.

Balance: $\pi_1 = 0.9\pi_1 + 0.5\pi_2$ gives $0.1\pi_1 = 0.5\pi_2$, so $\pi = (\frac{5}{6}, \frac{1}{6})$: sunny five days in six, whatever today is. Two-step check from rain: $r_{21}(2) = (P^2)_{21} = 0.5 \cdot 0.9 + 0.5 \cdot 0.5 = 0.70$, already climbing toward $\frac{5}{6} \approx 0.83$. Eigenvalues: $\text{tr } P = 1.4$, $\det P = 0.4$, so $\lambda = 1, 0.4$ (Lesson 14’s trace/det shortcut) — errors shrink by a factor 0.4 per step, and after n steps the deviation from steady state is proportional to 0.4^n .

Example 26.6 (Absorption — a gambler). States 0, 1, 2, 3: you hold $\$k$, bet $\$1$ at fair odds each round, stop at $\$0$ (ruin) or $\$3$ (goal). States 0 and 3 are *absorbing* ($P_{00} = P_{33} = 1$). Let a_k = probability of reaching 3 from k . Conditioning on one step (total probability):

$$a_k = \frac{1}{2}a_{k-1} + \frac{1}{2}a_{k+1}, \quad a_0 = 0, \quad a_3 = 1,$$

so a is linear in k : $a_k = k/3$. Fair game, fair chances — proportional to your stake. (B&T 7.4 develops absorption probabilities and expected absorption times in general; the same first-step conditioning gives both.)

CS221 / EE278 connection. A Markov chain is the “environment half” of CS221’s MDPs: add actions that choose which row of transition probabilities applies, plus rewards, and value iteration is first-step conditioning (the gambler calculation) iterated to a fixed point. An HMM is a Markov chain observed through noise, and its filtering updates ride on Theorem 26.2. In EE278, Markov chains are the first structured random process — and the eigen-analysis of P (stationarity, spectral gap, reversibility) is a recurring exam theme. This is the one Part IV lesson worth doing *before* CS221 if time permits.

26.3 NumPy

```
import numpy as np
rng = np.random.default_rng(0)

P = np.array([[0.9, 0.1],
              [0.5, 0.5]])

# Distribution evolution:  $\pi_n = \pi_0 P^n$  (row vector convention)
pi = np.array([0., 1.]) # start: rainy
for n in range(5):
    pi = pi @ P
print(pi) # -> [0.827, 0.173], nearing [5/6, 1/6]

# Stationary distribution via the eigenvector of  $P.T$  (Lesson 10):
lam, V = np.linalg.eig(P.T)
v = np.real(V[:, np.argmax(np.real(lam))])
print(v / v.sum()) # [0.833..., 0.166...]

# Long-run fraction of time, by simulation:
x, visits = 0, np.zeros(2)
for t in range(200_000):
    x = rng.choice(2, p=P[x])
    visits[x] += 1
print(visits / visits.sum()) # ~[5/6, 1/6]
```

Three computations of the same π — power iteration, eigenvector, simulation — agreeing is Theorem 26.4 certified numerically. Note the cost structure: πP is $O(m^2)$ per step with geometric convergence (rate = spectral gap), the full eigendecomposition is $O(m^3)$ once; for the enormous implicit chains of MCMC, simulation is the only option that fits in memory.

26.4 Exercises

Exercise 26.1 (Hand). A machine is up (1) or down (2): $P_{12} = 0.05$, $P_{21} = 0.6$. Find the stationary distribution, the long-run fraction of downtime, and both eigenvalues of P . Roughly how many steps until the initial condition is forgotten (deviation shrunk by $\sim 0.4^n$ -style factor — compute the actual base)?

Exercise 26.2 (Hand). For the gambler with unfair odds (win probability 0.4): re-derive the first-step equations and solve for a_1, a_2 with $a_0 = 0, a_3 = 1$. (The general solution is a geometric progression in $(0.6/0.4)^k$; or just solve the 2×2 linear system — Lesson 5.)

Exercise 26.3 (Proof, ★). Prove that if π satisfies the *detailed balance* equations $\pi_i P_{ij} = \pi_j P_{ji}$ for all i, j , then π is stationary. (Sum over i .) Verify detailed balance holds for Example 26.5 and exhibit a 3-state chain where stationarity holds but detailed balance fails (hint: probability flowing around a cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$).

Exercise 26.4 (Proof). Show that if P and Q are transition matrices, so is PQ ; conclude P^n is a transition matrix for every n (rows of $r_{ij}(n)$ are PMFs, as they must be).

Exercise 26.5 (NumPy). Build the 9×9 transition matrix of a random walk on a 3×3 grid (move to a uniformly random neighbor). Compute the stationary distribution via `eig` and confirm $\pi_j \propto \deg(j)$ (corners 2, edges 3, center 4). Estimate the spectral gap and check the observed convergence rate of $\pi^{(0)} P^n$ against it.

Deeper reading. B&T 7.1–7.3 (Markov chains, classification, steady state), 7.4 (absorption — the MDP-adjacent part), 7.5 (continuous time, EE278-relevant); CS221’s MDP notes re-derive first-step analysis as the Bellman equation.

27 Capstone IV: Estimation and Simulation in NumPy

NEEDED FOR: EE278

Forty-five minutes; Part IV at once, in EE278’s own shape: a hidden Gaussian signal, a noisy sensor, and the estimation machinery, checked against simulation.

Task. Track a slowly drifting Gaussian signal through noisy observations: build the model, derive the LLMS estimator from covariances, verify the orthogonality principle and the error formula, and watch the CLT and a Markov chain appear in the same program.

```
import numpy as np
rng = np.random.default_rng(0)

# --- 1. A Gauss-Markov signal (Lessons 23, 26) ---
# S_t = a S_{t-1} + W_t : a Markov chain with Gaussian steps
n, a, sw = 200_000, 0.95, 0.3
S = np.zeros(n)
W = sw * rng.normal(size=n)
for t in range(1, n):
    S[t] = a * S[t-1] + W[t]
# stationary variance: var = a^2 var + sw^2 => var = sw^2/(1-a^2)
print(S.var(), sw**2 / (1 - a**2))          # ~0.92 both

# --- 2. Noisy observations (Lesson 22's signal-plus-noise) ---
sn = 1.0
Y = S + sn * rng.normal(size=n)

# --- 3. LLMS from first principles (Lesson 22) ---
c_sy = np.cov(S, Y)[0, 1]                  # ~ var(S)
Shat = (c_sy / Y.var()) * Y
mse = ((S - Shat)**2).mean()
predicted = S.var() * sn**2 / (S.var() + sn**2)
```

```

print(mse, predicted)                                # agree (Example 22.3)

# --- 4. Orthogonality principle (Lesson 22) ---
err = S - S_hat
print(np.corrcoef(err, Y)[0, 1])                     # ~0

# --- 5. Autocorrelation of the signal (Lesson 21 -> EE278) ---
for k in (1, 5, 20):
    print(k, np.corrcoef(S[:-k], S[k:])[0, 1], a**k) # rho(k) = a^k

# --- 6. CLT check on the time average (Lesson 24) ---
means = S.reshape(200, 1000).mean(axis=1)           # 200 window averages
print(means.mean(), means.std())                     # ~0, small

```

Checkpoints (do all of these).

1. Step 1's variance identity is Proposition 21.4 at steady state: $\text{var}(S_t) = a^2 \text{var}(S_{t-1}) + \sigma_W^2$, solved at the fixed point. The process is a Markov chain (Lesson 26) with a continuous state space — EE278 calls it a *Gauss–Markov* or AR(1) process.
2. Step 3's measured MSE matches Example 22.6's formula $\sigma_S^2 \sigma_N^2 / (\sigma_S^2 + \sigma_N^2)$. Change `sn` to 0.3 and to 3.0 and confirm the estimator trusts/distrusts Y accordingly.
3. Step 4 is Theorem 22.3 empirically; also check `np.corrcoef(err, Y**2)` — near zero because everything is (jointly) Gaussian, Lesson 23's miracle (b).
4. Step 5: the autocorrelation $\rho(k) = a^k$ decays geometrically — the spectral-gap story of Lesson 26 (a is the second eigenvalue's role) meeting Lesson 21's covariance. This function $R_S(k)$ is exactly what EE278 will Fourier-transform into a power spectral density (Part II's DFT, waiting).
5. Step 6: window averages of a *dependent* process still concentrate, but more slowly than i.i.d. (var of the mean scales like $\frac{\sigma^2}{n} \cdot \frac{1+a}{1-a}$ — compute and compare). The i.i.d. CLT is the floor, not the whole story: EE278's ergodic-theory unit begins here.
6. Optional (the Kalman itch): the true $\mathbf{E}[S_t | Y_1, \dots, Y_t]$ uses all past observations, not just Y_t . Implement the two-line recursive update (predict with a , correct with the signal-to-noise gain) and watch the MSE drop below step 3's — you have written a scalar Kalman filter, EE278's finale, from this booklet's parts.

Final self-check list for Part IV

From memory: run the CDF-then-differentiate recipe and the monotone-change-of-variables formula; define covariance/correlation and prove $|\rho| \leq 1$ via Cauchy–Schwarz in the random-variable inner product; state and prove iterated expectations, the MMSE property of $\mathbf{E}[X | Y]$, the orthogonality principle, and the LMS formulas; define the covariance matrix, prove it is PSD and transform it through $A\Sigma A^T$; state the two Gaussian miracles and whiten a 2-D Gaussian by hand; prove Markov, Chebyshev, and the WLLN, and state the CLT with its standardization; define the Bernoulli and Poisson processes, prove the Poisson limit and exponential interarrivals; write balance equations, find a stationary distribution by hand and by eigenvector, and connect

convergence rate to the spectral gap. That is B&T Chapters 4–8 at working strength — and, with Parts II–III, the full prerequisite load-out for EE278.

A Hints and Answers to Computational Exercises

Lesson 1. 1.1: $3v + 2w = (8, 3)$; $v - w = (1, -4)$; $a = -1, b = 2$ (solve $2a + b = 0, -a + 3b = 7$). 1.2: the plane $x_1 - x_2 + x_3 = 0$ (combinations $(a, a + b, b)$ satisfy it); $(1, 0, 0)$ is not on it since $1 \neq 0$. 1.5: identical results; the loop pays Python interpreter overhead per element, NumPy runs one compiled loop.

Lesson 2. 2.1: $\langle x, y \rangle = 0$: orthogonal; $\|x\| = \|y\| = 3$; $\cos \theta = 0$. 2.2: $c = 9/5$, $cy = (18/5, 9/5)$, $z = (2/5, -4/5)$; $\|cy\|^2 + \|z\|^2 = 81/5 + 4/5 = 17 = \|x\|^2$. 2.4 hint: take $v = x$. 2.5: mean ≈ 0 , standard deviation $\approx 1/\sqrt{100} = 0.1$: random high-dimensional directions are nearly orthogonal, so distinct features/examples tend not to interfere.

Lesson 3. 3.1: (a) independent ($2 \cdot 3 - 1 \cdot 4 = 2 \neq 0$); (b) dependent: $(1, 2, 3) = (1, 1, 1) + (0, 1, 2)$; (c) always dependent: $1 \cdot 0 = 0$ is a nontrivial relation. 3.2: line: basis $\{(1, 2, -1)\}$, dimension 1; plane: basis $\{(1, 0, 1), (0, 1, 0)\}$, dimension 2. 3.3 hint: the standard basis spans $\mathbb{R}^n$ with length n ; apply Theorem 3.4. 3.5: rank of a random 3×5 is 3 (5 vectors in $\mathbb{R}^3$ cannot be independent; generic ones span). For rank 2: e.g. `np.column_stack([u, v, u+v, 2*u, u-3*v])` for random independent $u, v \in \mathbb{R}^3$.

Lesson 4. 4.1: (a) $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$; (b) $\begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$; (c) $\begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}$. 4.2: $AB = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$, $BA = \begin{pmatrix} 2 & 2 \\ 0 & 1 \end{pmatrix}$, $A^T B^T = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} = (BA)^T$. ✓ 4.5: $R^T R = I$ means $\langle Rx, Ry \rangle = \langle x, y \rangle$: rotations preserve dot products, hence lengths and angles.

Lesson 5. 5.1: first system: $x = (2, 1)$, two pivots, unique. Second: rank 1, b in the column space; solutions $(3 - 2s, s)$, a line. Third: rank of augmented matrix exceeds rank of A ($3 \neq 7/2$): no solution. 5.2: null space basis $\{(1, -2, 1)\}$; column space $= \mathbb{R}^2$, basis $\{(1, 0), (0, 1)\}$; $1 + 2 = 3$. ✓ 5.6: solvable iff the two ranks agree; for random b with this rank-1 A , almost surely not solvable.

Lesson 6. 6.1: $e_1 = \frac{1}{\sqrt{2}}(1, 1, 0)$; $f_2 = v_2 - \frac{1}{2}(1, 1, 0) = (\frac{1}{2}, -\frac{1}{2}, 1)$, $e_2 = \frac{1}{\sqrt{3/2}}(\frac{1}{2}, -\frac{1}{2}, 1)$. $P_U x = \langle x, e_2 \rangle e_2 = 2 \cdot \frac{(\frac{1}{2}, -\frac{1}{2}, 1)}{\sqrt{3/2}} = (1, -1, 2)$ (note $\langle x, e_1 \rangle = 0$). Residual $(-1, 1, 1)$: orthogonal to both. ✓ 6.2: $A^T A = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}$, $A^T b = (4, 9)$; $\hat{x} = (c, d) = (0.1, 0.6)$: line $y = 0.1 + 0.6t$. 6.5: with a duplicated column the rank stays 2 but A has 3 columns, so $A^T A$ is singular and `solve` raises `LinAlgError` (or returns garbage); `lstsq` still returns a minimizer — Theorem 6.4's uniqueness needs independent columns.

Lesson 7. 7.1: (a) $(1, -2, 3)$; (b) $2w$ (take $M = I$); (c) $2(w \cdot \phi - y)\phi$. 7.2: $\nabla f(w) = 2(2w_1 + w_2 - 3)(2, 1)$. Step 1: residual -3 , gradient $(-12, -6)$, $w^{(1)} = (1.2, 0.6)$, loss drops $9 \rightarrow 0.09$. Step 2: residual 0.3 , gradient $(1.2, 0.6)$, $w^{(2)} = (1.08, 0.54)$, loss $0.09 \rightarrow 0.0009$. Decreasing. ✓ 7.5: each SGD step touches one row ($O(n)$) instead of all m rows ($O(mn)$).

Lesson 9. 9.1: $r = 2$, $\theta = 2\pi/3$: $z = 2e^{2\pi i/3}$; $z^3 = 8e^{2\pi i} = 8$; direct expansion of $(-1 + i\sqrt{3})^3$ gives 8. ✓ 9.2: $1, \frac{1}{2} - \frac{\sqrt{3}}{2}i, -\frac{1}{2} - \frac{\sqrt{3}}{2}i, -1, -\frac{1}{2} + \frac{\sqrt{3}}{2}i, \frac{1}{2} + \frac{\sqrt{3}}{2}i$; both sums vanish — the squares case is Lemma 9.2 with $N = 6, m = 2$ ($6 \nmid 2$). 9.5: both inner products ≈ 0 (magnitude $\sim 10^{-16}$); sampled distinct-frequency exponentials are orthogonal — Lemma 9.2 again.

Lesson 10. 10.1: (a) $\lambda = 3$: e_1 ; $\lambda = -2$: e_2 . (b) $\lambda = 1$: $(1, 1)$ (the mirror line, fixed); $\lambda = -1$: $(1, -1)$ (perpendicular to it, flipped). (c) over $\mathbb{R}$: none; over $\mathbb{C}$: $\lambda = \pm i$ with eigenvectors $(1, \mp i)$. 10.2: eigenpairs $\lambda = 3, (1, 1)$ and $\lambda = -1, (1, -1)$; $A^k(1, 0) = \frac{1}{2}(3^k + (-1)^k, 3^k - (-1)^k)$. 10.5: $M^k x_0 \rightarrow (2/3, 1/3)$; eigenvalues 1 and 0.7; the 0.7-component dies geometrically and the limit is the $\lambda = 1$ eigenvector scaled to match the coordinates' conserved sum.

Lesson 11. 11.1: $\text{tr} = 3$, $\det = -4$: $\lambda = 4, -1$; eigenvectors $(1, 2)$ and $(2, -1)$ (orthogonal $\checkmark$); $q = 4y_1^2 - y_2^2$ in the normalized eigen-coordinates; not PSD (a negative eigenvalue). 11.2: $\lambda = 0, 2$; eigenvectors $(1, i)$ for 0 and $(1, -i)$ for 2; $\langle (1, i), (1, -i) \rangle = 1 \cdot 1 + i \cdot (-i) = 1 + i \cdot i = 0$. $\checkmark$ 11.5: for the non-symmetric case **eig**'s eigenvector matrix satisfies $Q^T Q \neq I$ in general — orthogonality of eigenvectors is a self-adjoint privilege.

Lesson 12. 12.1: rows of W_3 use $1, \omega, \omega^2$; $\hat{x} = (3, 0, 0)$ for the constant (all energy at frequency 0), and $\hat{x} = (0, 3, 0)$ for $x_n = \omega^{-n}$ (it is the frequency-1 basis vector, scaled). 12.2: C is the cyclic down-shift: $(Cx)_n = x_{n-1 \bmod 4}$. With $v_n = i^{-n}$: $(Cv)_n = i^{-(n-1)} = i v_n$, so $\lambda = i$. And $\hat{c}_k = \omega^k = (-i)^k$, so $\lambda = (-i)^3 = \hat{c}_3$ — consistent with Theorem 12.6, since $v_n = i^{-n} = i^{3n}$ is the frequency-3 column of W^{-1} . 12.5: peaks at $k = 5, 40$ (and mirrors 251, 216); keeping only $k \leq 10$ without the mirror band produces a complex-valued signal — the conjugate-symmetry proof: $\hat{x}_{N-k} = \sum_n x_n \omega^{(N-k)n} = \sum_n x_n \omega^{kn} = \overline{\hat{x}_k}$ for real x , since $\omega^{Nn} = 1$.

Lesson 13. 13.1: $\sigma_1 = 3$, $\sigma_2 = 2$; $v_1 = e_1$, $u_1 = e_1$; $v_2 = e_2$, $u_2 = -e_2$ (the sign moved into u_2 : $Av_2 = -2e_2 = 2(-e_2)$). 13.2: $A^T A = 5A$; $\sigma_1 = 5$, $v_1 = \frac{1}{\sqrt{5}}(1, 2) = u_1$; $A = 5 v_1 v_1^T$. 13.5: typically $k \approx 20$ of 50+ components looks clean; storage $k(m + n + 1)$ vs mn .

Lesson 14. 14.1: formula: $1 \cdot (4 \cdot 3 - 1 \cdot 0) - 2 \cdot (2 \cdot 3 - 1 \cdot 1) + 0 = 12 - 10 = 2$. Reduction: $R_2 \leftarrow 2R_1$, $R_3 \leftarrow R_1$ gives rows $(1, 2, 0)$, $(0, 0, 1)$, $(0, -2, 3)$; swap $R_2 \leftrightarrow R_3$ (sign flip); triangular pivots $1, -2, 1$: $\det = -(1)(-2)(1) = 2$. $\checkmark$ 14.2: first: $\text{tr} = 7$, $\det = 6$: $\lambda = 6, 1$. Rotation: $\text{tr} = 2 \cos \theta$, $\det = 1$: $\lambda = \cos \theta \pm i \sin \theta = e^{\pm i\theta}$ — unit-modulus, as Exercise 12.3 demands of any unitary map. 14.5: $\mathbf{P}(\det > 0) \approx \frac{1}{2}$ by symmetry ($\det$ is an odd function under swapping columns, which is measure-preserving); image area $\approx |\det A| \times (\text{unit area})$.

Lesson 16. 16.1: $\mathbf{P}(\geq 2H) = \frac{1}{2}$; $\mathbf{P}(\geq 2H \mid H_1) = \frac{3}{4}$; dependent ($\frac{3}{4} \neq \frac{1}{2}$). 16.2: $\mathbf{P}(\text{spam} \mid \text{free}) = \frac{0.3 \times 0.4}{0.3 \times 0.4 + 0.7 \times 0.05} = \frac{0.12}{0.155} \approx 0.774$; with prior 0.8: $\frac{0.32}{0.32 + 0.01} \approx 0.970$. 16.6: ≈ 0.757 unconditionally, ≈ 0.5 given the fair coin — conditional independence confirmed.

Lesson 17. 17.1: $p_X = (\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8})$ on $0..3$; $\mathbf{E}[X] = 1.5 = np$, $\text{var}(X) = 0.75 = np(1-p)$. 17.2: $\mathbf{E}[X] = 1.25$, $\text{var}(X) = 0.9375$; $\mathbf{P}(X \geq 1) = 1 - (3/4)^5 \approx 0.763$. 17.5: means all ≈ 1 ; variance also ≈ 1 ; for large n the number of fixed points is approximately Poisson(1), whose mean and variance are both 1.

Lesson 18. 18.1: (3-sided dice) joint on (x, y) , $x \leq y$: $p(x, x) = \frac{1}{9}$, $p(x, y) = \frac{2}{9}$ for $x < y$; $p_Y = (\frac{1}{9}, \frac{3}{9}, \frac{5}{9})$ for $y = 1, 2, 3$; $\mathbf{E}[X \mid Y = 3] = \frac{\frac{2}{9} \cdot 1 + \frac{2}{9} \cdot 2 + \frac{1}{9} \cdot 3}{\frac{5}{9}} = \frac{9}{5} = 1.8$. 18.2: posterior $\frac{0.3 \times 0.4 \times 0.02}{0.3 \times 0.4 \times 0.02 + 0.7 \times 0.05 \times 0.3} = \frac{0.0024}{0.0024 + 0.0105} \approx 0.186$: “free” says spam, “meeting” says ham, and ham wins. 18.5: marginals Binomial($2, \frac{1}{2}$) and Binomial($3, \frac{1}{2}$); dependent; $\mathbf{E}[\text{total} \mid \text{first two} = 2] = 2.5$.

Lesson 19. 19.1: $\mathbf{E}[X] = 1$, $\text{var}(X) = \frac{1}{3}$, $\mathbf{E}[X^3] = 2$; $F_Y(y) = \mathbf{P}(X^2 \leq y) = \frac{\sqrt{y}}{2}$ on $[0, 4]$, so $f_Y(y) = \frac{1}{4\sqrt{y}}$. 19.2: $\mathbf{P}(X \leq 13) = \Phi(1.5) \approx 0.933$; $\mathbf{P}(|X - 10| > 4) = 2(1 - \Phi(2)) \approx 0.046$; $c = 10 + 2 \times 2.33 = 14.66$. 19.5: selected Θ 's: mean $\approx 0.75 = y/2$, variance ≈ 0.5 . ✓

Lesson 21. 21.1: (a) $f_Y(y) = \frac{1}{2\sqrt{y}}$ on $(0, 1)$ both ways; (b) $f_Y(y) = \frac{1}{3}$ on $(1, 4)$. 21.2: $\text{cov}(S, D) = \text{var}(X) - \text{var}(Y) = 0$; *not* independent ($S = 12$ forces $D = 0$). 21.5: $\text{var}(X_i) = \frac{1}{n}(1 - \frac{1}{n})$; $\text{cov}(X_i, X_j) = \frac{1}{n(n-1)} - \frac{1}{n^2} = \frac{1}{n^2(n-1)}$; total $= n \cdot \frac{n-1}{n^2} + n(n-1) \cdot \frac{1}{n^2(n-1)} = \frac{n-1}{n} + \frac{1}{n} = 1$.

Lesson 22. 22.1: $\mathbf{E}[X \mid Y] = (\frac{2}{3}, \frac{1}{3}, \frac{3}{4})$ for $y = 0, 1, 2$; iterated: 0.6. ✓ LLMS: $a = \text{cov} / \text{var}(Y) = 0.04/0.69 \approx 0.058$, $b = 0.6 - a(1.1) \approx 0.536$. 22.2: $\hat{X}_L = \frac{4}{5}Y$; error variance $\frac{4}{5}$; fraction removed $= \rho^2 = \frac{4}{5}$. 22.5: LLMS MSE $\approx \text{var}(X)(1 - \rho^2)$ with $\rho = \text{cov}(Y^3, Y)/(\sigma_X \sigma_Y) = 3/\sqrt{15} \approx 0.775$ (using $\mathbf{E}[Y^4] = 3$, $\mathbf{E}[Y^6] = 15$); the binned conditional mean tracks y^3 and its MSE ≈ 0.01 — far better.

Lesson 23. 23.1: $A = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$; $\text{cov}(Y) = \begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix}$; uncorrelated iff $\text{var}(X_1) = \text{var}(X_2)$. 23.2: $\lambda = 6, 1$; eigenvectors $\frac{1}{\sqrt{5}}(2, 1)$, $\frac{1}{\sqrt{5}}(-1, 2)$; fraction $6/7 \approx 0.857$; ellipses elongated along $(2, 1)$. 23.5: two eigenvalues $\approx$ the two nonzero eigenvalues of RR^T plus 0.1, three ≈ 0.1 ; projection keeps $\approx (\lambda_1 + \lambda_2)/\text{tr}$ of the variance.

Lesson 24. 24.1: total: mean 800, sd 20; Chebyshev: $\mathbf{P} \leq 400/100^2 = 0.04$; CLT: $1 - \Phi(5) \approx 3 \times 10^{-7}$ — Chebyshev is loose. 24.2: Chebyshev: $n \geq \frac{1/4}{0.01 \times (0.01)^2} = 250,000$; CLT: $n \approx (2.58/0.02)^2 \times \dots \approx 16,600$. 24.5: slope $-\frac{1}{2}$; $\sigma = \sqrt{p(1-p)} \times 4$ with $p = \pi/4$.

Lesson 25. 25.1: $(0.9)^{10} \approx 0.349$; $3/0.1 = 30$ slots; $\mathbf{P} = 0.9 \cdot 0.1$ shifted: $(0.9)^1 \cdot 0.1 = 0.09$ (memorylessness: “first arrival in slot 5 given none in 1–3” = “first arrival in slot 2” = 0.9×0.1). 25.2: mean wait = $1/12$ hour = 5 min; $(8/12)^3 = \frac{8}{27}$; Poisson(2): $e^{-2}2^2/2! \approx 0.271$. 25.5: correlations ≈ 0 ; sub-rates ≈ 1 and ≈ 2 .

Lesson 26. 26.1: $\pi = (\frac{12}{13}, \frac{1}{13})$; downtime $\approx 7.7\%$; eigenvalues 1 and 0.35; deviations shrink like 0.35^n ($\approx$ forgotten within ~ 5 steps). 26.2: with $r = 0.6/0.4 = 1.5$: $a_k = \frac{1-r^k}{1-r^3}$, so $a_1 \approx 0.211$, $a_2 \approx 0.526$. 26.5: $\pi_j \propto$ degree: corners $\frac{2}{24}$, edges $\frac{3}{24}$, center $\frac{4}{24}$; spectral gap from the second-largest $|\lambda|$ of P .